

第四章 互斥同步与通讯

提纲

4. 1 并发进程 (concurrent processes)

4. 2 进程互斥 (mutual exclusion)

4. 3 进程同步 (synchronization)

4. 4 进程高级通讯 (communication)

提纲

4.1 并发进程(concurrent processes)

4.2 进程互斥(mutual exclusion)

4.3 进程同步(synchronization)

4.4 进程高级通讯(communication)

4.1 并发进程

■ 4.1.1 前趋图的定义

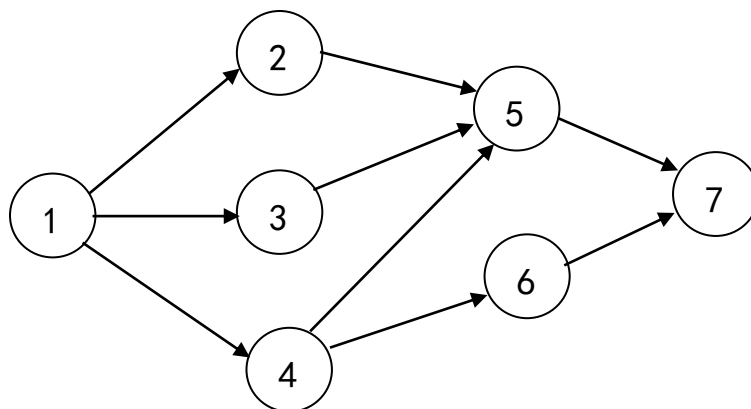
■ 前趋图 (precedence graph)

- 有向无环图，图中每个结点表示一个语句、一个计算步骤、或一个进程
- 结点间的有向边表示偏序或前趋 (precedence relation) 关系 “ $\rightarrow$ ”
- $\rightarrow = \{ (P_i, P_j) \mid P_j \text{ 启动之前 } P_i \text{ 必须已经完成} \}$
- $(P_i, P_j) \in \rightarrow$ 可记作 $P_i \rightarrow P_j$ ，称 P_i 是 P_j 的前趋， P_j 是 P_i 的后继
- 在前趋图中，没有前趋的结点称为初始结点，没有后继的结点称为终止结点
- 每个结点可以有一个权重 (weight)，它可以表示该结点所包含的程序量或计算时间

4.1.1 前趋图

- 前趋图的例子

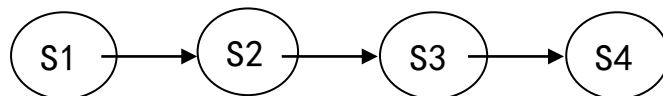
- $P1 \rightarrow P2$, $P1 \rightarrow P3$, $P1 \rightarrow P4$, $P2 \rightarrow P5$, $P3 \rightarrow P5$, $P4 \rightarrow P5$,
 $P4 \rightarrow P6$, $P5 \rightarrow P7$, $P6 \rightarrow P7$



- 前趋图满足传递性

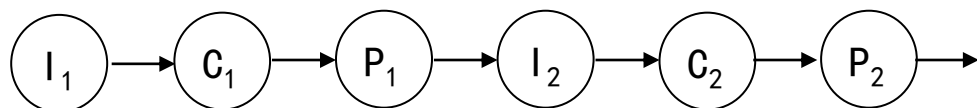
4.1.2 顺序程序及其特性

- 程序的顺序执行
- (1) 内部顺序性：对于一个进程来说，它的所有指令是按序执行的。
 - S1: $a := x + y$
 - S2: $b := a - z$
 - S3: $c := a + b$
 - S4: $d := c + 5$



4.1.2 顺序程序及其特性 (Cont.)

- (2) 外部顺序性：对于多个进程来说，所有进程的活动是依次执行的
 - 例：输入(I)、计算(C)、打印(P)三个活动构成的进程，每个进程的内部活动是顺序的，即 $I_i \rightarrow C_i \rightarrow P_i$ ，多个进程的活动也是顺序的



4.1.2 顺序程序及其特性 (Cont.)

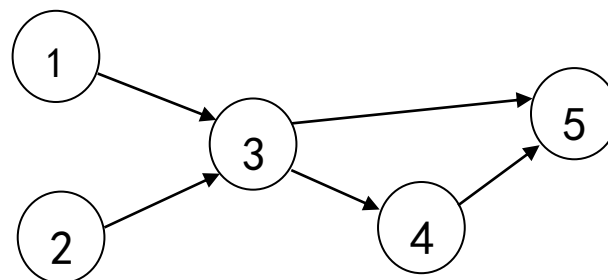
- 顺序程序特性
 - 连续性
 - 封闭性
 - 可再现性

4.1.3 并发程序及其特性

■ 程序的并发执行

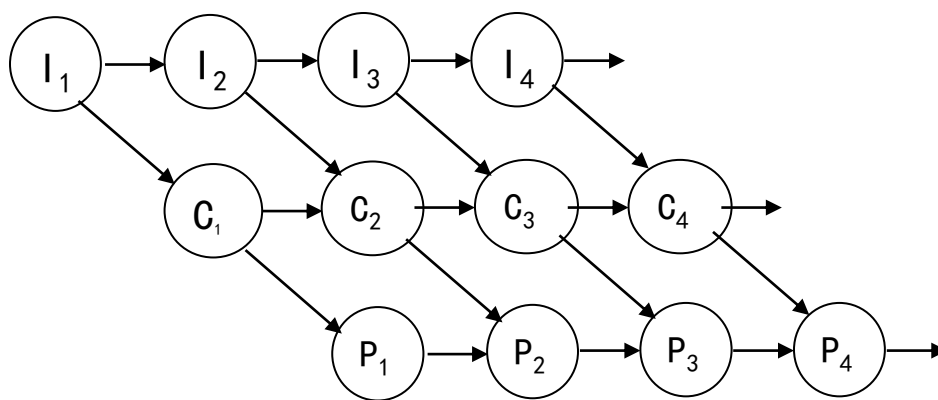
■ (1) 内部并发性：一个程序内部的并发性

- S1: $a := x + 2;$
- S2: $b := y + 4;$
- S3: $c := a + b;$
- S4: $d := c + 6;$
- S5: $e := c - d;$



4.1.3 并发程序及其特性 (Cont.)

- 程序的并发执行
 - (2) 外部并发性：多个程序之间的并发性



4.1.3 并发程序及其特性 (Cont.)

- 并发程序的特性
 - 间断性
 - 非封闭性
 - 不可再现性

4.1.3 并发程序及其特性 (Cont.)

- 不可再现性的例子

- 进程P:

A1: N:=0;

A2: N:=N+1;

A3: GOTO A2;

- 进程Q:

B1: PRINT (N) ;

B2: N:=0;

B3: GOTO B1;

- 两个进程并发执行时有许多可能的交叉

- 如进程P循环5次进程Q循环1次，然后进程P又循环3次进程Q循环1次，则输出结果是5， 3
 - 如进程P循环2次进程Q循环1次，然后进程P又循环6次进程Q循环1次，则输出结果是2， 6
 - 进程Q执行完B1后被中断，进程P对N执行加1操作，然后进程Q执行B2，在这种情况下进程P的累计数将被丢失

4.1.4 程序并发执行的条件

- 在失去封闭性的条件下，保持可再现性。
 - $R(p_i) = \{a_1, a_2, \dots, a_m\}$ 表示程序 p_i 在执行期间所需读取的所有变量的集合，称为“读集”
 - $W(p_i) = \{b_1, b_2, \dots, b_n\}$ 表示程序 p_i 在执行期间所需改变的所有变量的集合，称为“写集”
 - 若有两条语句 $c=a+b$ 和 $v=c-1$ ，则它们的“读集”和“写集”分别为：
 - $R(c:=a+b) = \{a, b\}$ ， $R(v:=c-1) = \{c\}$
 - $W(c:=a+b) = \{c\}$ ， $W(v:=c-1) = \{v\}$

4.1.4 程序并发执行的条件（Cont.）

- 若两个程序p1，p2满足如下条件，则能够保持可再现性，因而可以并发执行。该条件是1966年首先由Bernstein提出的，称为Bernstein条件。

$$R(p1) \cap W(p2) \cup R(p2) \cap W(p1) \cup W(p1) \cap W(p2) = \Phi$$

4.1.4 程序并发执行的条件 (Cont.)

- 例，有如下四条语句
 - S1: $a := x + y$
 - S2: $b := z + 1$
 - S3: $c := a - b$
 - S4: $w := c + 1$
- $R(S1) = \{x, y\}$, $R(S2) = \{z\}$, $R(S3) = \{a, b\}$, $R(S4) = \{c\}$
- $W(S1) = \{a\}$, $W(S2) = \{b\}$, $W(S3) = \{c\}$, $W(S4) = \{w\}$
- S1和S2可以并发执行, 因为 $R(S1) \cap W(S2) \cup R(S2) \cap W(S1) \cup W(S1) \cap W(S2) = \Phi$
- S1和S3不能并发执行, 因为 $W(S1) \cap R(S3) = \{a\}$
- S2和S3也不能并发执行, 因为 $W(S2) \cap R(S3) = \{b\}$
- S3和S4不能并发执行, 因为 $W(S3) \cap R(S4) = \{c\}$

4.1.5 并发程序的表示

- 并发程序的表示：
 - `cobegin S1; S2; ...; Sn coend;`
 - `parbegin S1; S2; ...; Sn parend;`
- 并发程序给操作系统带来一系列问题
 - 如互斥、死锁、饥饿等，增加了操作系统的复杂度
 - 但与并发所带来的好处相比，所付出的代价是值得的

4.1.6 与时间有关的错误

例：图书借阅系统 (x : 某种书册数, 设当前 $x=1$.)

终端1:

CYCLE

等待借书者;

IF $x \geq 1$ Then

Begin ← 1

$x := x - 1$;

借书 ← 3

End

Else 无书

End

终端2:

CYCLE

等待借书者;

IF $x \geq 1$ Then

Begin ← 2

$x := x - 1$;

借书 ← 4

End

Else 无书

End

交叉过程中改变了变量 x 的运行环境, 导致错误。

关于就绪队列的整队问题

设有A,B,C,D四个就绪进程,优先级依次为2,10,30,35,分析插队过程中出现的情况.分析:

- (1)若有E进程,优先数为25处于就绪状态,则要求加入就绪队列,调整队子程序执行,该程序判断E应加入B,C之间,则执行

$R:=C$; ① $B:=E$; $E:=R$

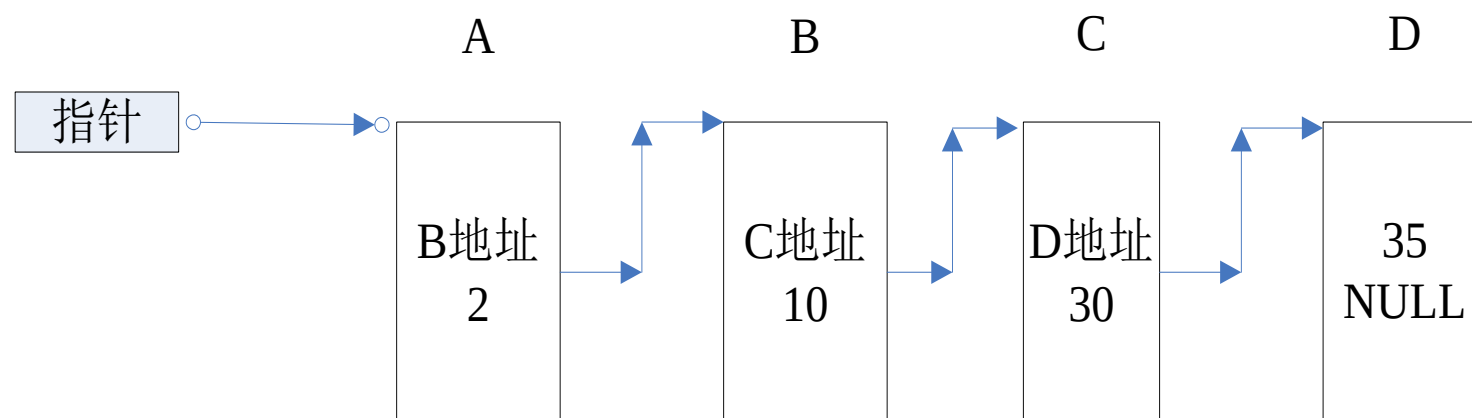
- (2)若在①时进程F因完成I/O由等待状态变为就绪,其优先级为15,整队子程序被中断,转入执行中服程序,则出现丢失一个进程

$R:=C$; $B:=F$; $F:=R$

结论:该错误因共享就绪队列引起,对就绪队列操作不当,也是与时间相关的错误导致结果不唯一.

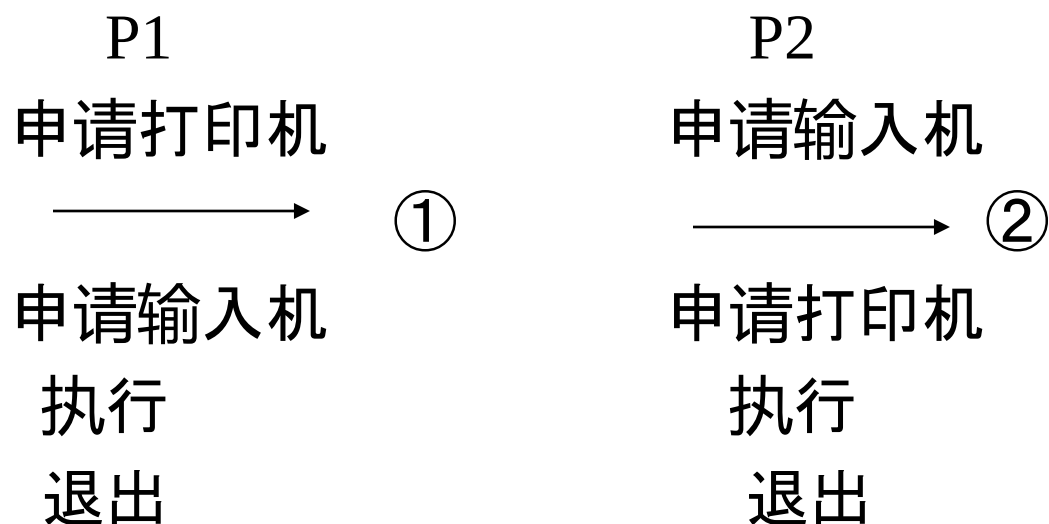
关于就绪队列的整队问题

$R := C;$ ① $B := E ; E := R$
 $R := C;$ $B := F; F := R$



两进程申请两个独占性资源

- 设有两个进程P1,P2,系统中有两类独占性资源如打印机和输入机,两进程的活动为:



分析: (1)若P1推进到①,P2推进,则P1,P2均等待对方释放资源形成永久等待.该错误也与推进速度有关是与时间相关的错误.

4.1.6 与时间有关的错误(Cont.)

错误原因之1:

进程执行交叉(interleave);

错误原因之2:

涉及公共变量(x)。

Remarks:

某些交叉结果不正确;

必须去掉导致不正确结果的交叉。

提纲

4.1 并发进程(concurrent processes)

4.2 进程互斥(mutual exclusion)

4.3 进程同步(synchronization)

4.4 进程高级通讯(communication)

4.2 进程互斥(mutual exclusion)

- 4.2.1 共享变量与临界区
- 4.2.2 临界区域与进程互斥
- 4.2.3 进程互斥的实现
- 4.2.4 多处理机环境下的互斥

4.2.1 共享变量与临界区域

- 共享变量 (shared variable)
 - 多个进程都需要访问的变量
- 临界区域 (critical region)
 - 访问共享变量的程序段



表示

共享变量: `shared <一组变量>`

临界区域: `region <一组变量> do <语句>`

例子: `shared B:array[0,...,n-1] of integer;`

```
    region B do
```

```
        begin
```

```
            ..... (访问B)
```

```
        end;
```

```
    region B do
```

```
        begin
```

```
            ... (访问B).
```

```
        end;
```

4.2.2 临界区域与进程互斥

定义：多个进程不能同时进入关于同一组共享变量的临界区域，否则可能发生与时间有关的错误，这种现象称为进程互斥。

二层涵义：

(1) 任何时刻最多只能有一个进程处于同一组共享变量的**相同**的临界区域

(2) 任何时刻最多只能有一个进程处于同一组共享变量的**不同**的临界区域

Remarks：互斥是相对于公共变量而言的

嵌套临界区域

```
shared x1, x2;
```

P1

```
region x1, x2 do
  begin
    ..... ①
    region y1, y2 do
      begin
        .....
      end
    end
  end
end
```

```
shared y1, y2;
```

P2

```
region y1, y2 do
  begin
    ..... ②
    region x1, x2 do
      begin
        .....
      end
    end
  end
end
```

嵌套临界区域(Cont.)

```
shared x1, x2, y1, y2;
```

P1

```
region x1, x2, y1, y2 do
  begin
    .....
  end;
```

```
shared x1, x2, y1, y2;
```

P2

```
region x1, x2, y1, y2 do
  begin
    .....
  end;
```

将临界区扩大，不能并发了，但是可以防止死锁。

4.2.3 进程互斥的实现

- Framework

Repeat

entry section

critical section

exit section

remainder section

Until false

4.2.3 进程互斥的实现 (Cont.)

- Requirements:

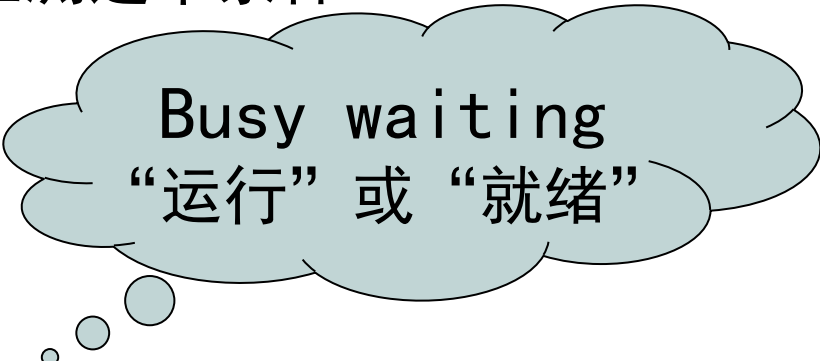
- mutual exclusion(互斥性): 任意时刻至多只能有一个进程处于关于同一组共享变量的临界区之中
- Progress(进展性): 临界区空闲时, 放行一个进入者
- bounded waiting(有限等待): 一个想要进入临界区的进程在等待有限个进程进入并离开临界区后获得进入临界区的机会

4.2.3 进程互斥的实现（Cont.）

- 临界资源 (critical resource): 一次只允许一个进程使用的资源
- 临界资源的调度原则
 - 当关于某一组共享变量的临界区均为空时，一个要求进入该组公共变量的某一临界区的进程应当能够立刻进入；
 - 当关于某一组共享变量的某一临界区被占用时，一个要求进入该组共享变量某一临界区的进程应当等待；
 - 当一个进程离开关于某一组共享变量的某一临界区时，应当允许某一个等待该组共享变量某一临界区的进程进入；

4.2.3.1 进程互斥的软件实现

- 完全用程序实现，不需特殊硬件指令支持
- 可用于单CPU
- 有忙式等待问题
 - 不进入等待状态
 - 它反复地检测一个条件，如果条件成立就反复地循环
 - 如果CPU被剥夺了，就回到就绪状态
 - 以后如果再分配CPU，还检测这个条件



Busy waiting
“运行”或“就绪”

4.2.3.1 进程互斥的软件实现（Cont.）

- 忙式等待与阻塞式等待
 - 相同之处
 - 在于进程都不具备继续向前推进的条件
 - 不同之处
 - 处于忙式等待的进程不主动放弃CPU，尽管CPU可能被剥夺，因而是低效的
 - 而处于阻塞状态的进程主动放弃CPU，因而是高效的

4.2.3.1 进程互斥的软件实现（Cont.）

- 软件实现方法
 - 在进入区设置和检查一些标志来标明是否有进程在临界区中
 - 如果已有进程在临界区，则在进入区通过循环检查进行等待
 - 进程离开临界区后则在退出区修改标志
- 问题：设置什么标志和如何检查标志

尝试1

```
int turn;
```

```
P0:
```

```
do{
```

```
    while (turn==1) ;
```

```
    临界区代码
```

```
    turn=1;
```

```
    其余代码
```

```
}while(1);
```

```
P1:
```

```
do{
```

```
    while(turn==0) ;
```

```
    临界区代码
```

```
    turn=0;
```

```
    其余代码
```

```
}while(1);
```

不满足**进展性**：P0和P1必须交替进入临界区。交替执行是不合理的，因为不知道一个进程是否想进入临界区。

尝试2

```
int flag[2]; (0, 0)
```

P0:

Do {

while (flag[1]) ;

flag[0]=1;

临界区

flag[0]=0;

其余代码

} while(1);

P1:

do {

while (flag[0]);

flag[1]=1;

临界区

flag[1]=0;

其余代码

} while(1);

不满足互斥性: flag[0]=flag[1]=0.

尝试3

```
int flag[2]; (0, 0)
```

```
do{  
    flag[0]=1;  
    while (flag[1]);  
    临界区代码  
    flag[0]=0;  
    其余代码  
}while(1);
```

```
do{  
    flag[1]=1;  
    while(flag[0]);  
    临界区  
    flag[1]=0;  
    其余代码  
}while(1);
```

不满足进展性：flag[0]=flag[1]=1，进程P1和进程P2都不能进入临界区。

Dekker算法

```
int flag[2]; (init 0)
int turn; (0 or 1)
```

P0:do{

```
    flag[0]=1;
    while(flag[1]) {
        if(turn==1) {
            flag[0]=0;
            while (turn==1)
                skip;
            flag[0]=1;
        }
    }
```

临界区

```
    turn=1;
    flag[0]=0;
    其余代码
```

}while(1);

P1: do{

```
    flag[1]=1;
    while(flag[0]) {
        if(turn==0) {
            flag[1]=0;
            while (turn==0)
                skip;
            flag[1]=1;
        }
    }
```

临界区

```
    turn=0;
    flag[1]=0;
    其余代码
```

}while(1);

尝试4 Peterson算法

(1) flag[1]=0 , turn==1
(2) flag[1]=1 , turn=0
(3) flag[1]=0 , turn=0

```
int flag[2];
```

```
Int turn;
```

```
P0:
```

```
Do{
```

```
    flag[0]=1; turn=1;
```

```
    while (flag[1] && turn==1);
```

```
    临界区
```

```
    flag[0]=0;
```

```
    其余代码
```

```
}while(1);
```

```
P1:
```

```
Do{
```

```
    flag[1]=1; turn=0;
```

```
    while (flag[0] && turn==0);
```

```
    临界区
```

```
    flag[1]=0;
```

```
    其余代码
```

```
}while(1);
```

迄今为止实现2个进程互斥的最简洁的算法

Lamport面包店算法

- **问题描述：**一个面包店，较小，一次只能进入一位顾客。顾客在进入面包店之前，首先抓一个号，然后按照号码由小到大的顺序依次进入面包店购买面包。
- **算法思想：**设置一个发号者，按1, 2, ..., 发号。想进入临界区的进程抓号，抓到号之后按由小到大的次序依次进入。

Lamport面包店算法

- Problem: 两个进程可能抓到相同的号。
- Why? 为保证抓到不同的号, 需要互斥机制。
- Resolution: 若抓到相同的号, 按进程编号依次进入。这样可以把要进入临界区的进程排一个全序
- Definition: $(a, b) < (c, d)$ iff $(a < c)$ or $(a == c \text{ and } b < d)$

Lamport面包店算法 (Cont.)

Boolean choosing[0, ..., n-1]; (false)

Int number[0, ..., n-1]; (0)

Pi 进入:

1.

2. number[i] = max {number[0], ..., number[n-1]} + 1;

3.

4. For (j=0; j<n; j++) {

5.

6. While (number[j] != 0) and
 (number[j], j) < (number[i], i) skip;

7. }

(1) P1抓到1未赋值

(2) P2抓到1赋值

(3) P2进入临界区

(4) P1进入临界区

Lamport面包店算法 (Cont.)

Boolean choosing[0, ..., n-1]; (false)

int number[0, ..., n-1]; (0)

P_i 进入:

1. choosing[i]=true; //正在抓号
2. number[i]=max{number[0], ..., number[n-1]}+1; //号递增
3. choosing[i]=false; //抓完号
4. For (j=0; j<n; j++) {
5. While (choosing[j]) skip; //若其他进程在抓, 等待
6. While (number[j] != 0) and
7. (number[j], j) < (number[i], i) skip;
8. }

Lamport面包店算法(Cont.)

P_i 离开:

`number[i]=0;`

Lamport满足临界区管理原则

- 互斥性 (mutual exclusion)
 - 多个进程竞争进入临界区时，抓到号且二元组 $(number[i], i)$ 最小的进程获准进入临界区，其它进程将在第一个while循环或第二个while循环处等待，因而满足互斥性
- 进展性 (progress)
 - 当仅有一个进程想进入临界区时，该进程可以立即进入；当有多个进程想进入临界区时，抓到号且二元组 $(number[i], i)$ 最小的进程获准进入，因而满足进展性
- 有限等待性 (bounded waiting)
 - 对任意一个想要进入临界区的进程 P_i ，设其抓到号码为 $number[i]$ ，按二元组 $(number[i], i)$ 次序排在 P_i 之前的竞争进程数量是有限的，在最坏情况下 P_i 将等待 $n-1$ 个排在其前面的进程进入并离开临界区后获准进入临界区，因而满足有限等待性

Eisenberg/Mcguire算法

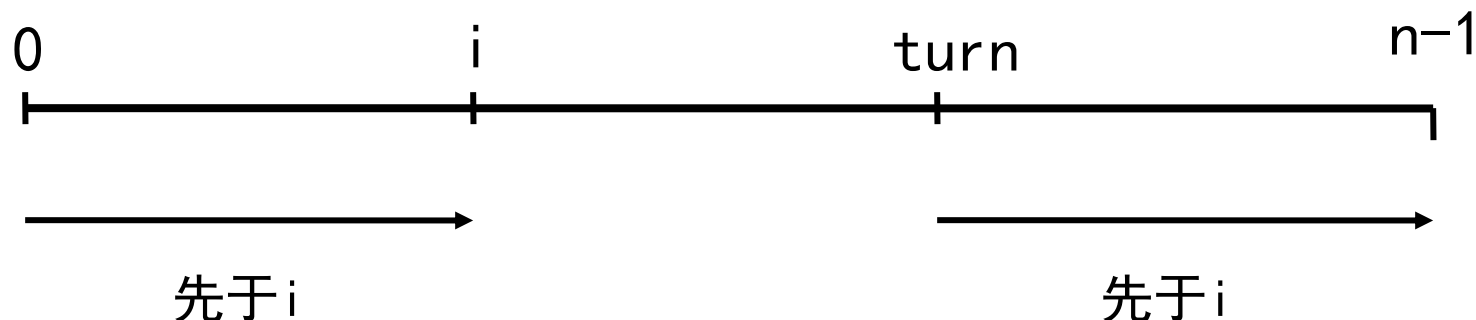
enum flag[0, ..., n-1] of (idle, want_in, in_cs);

int turn; //0..n-1; 初始任意

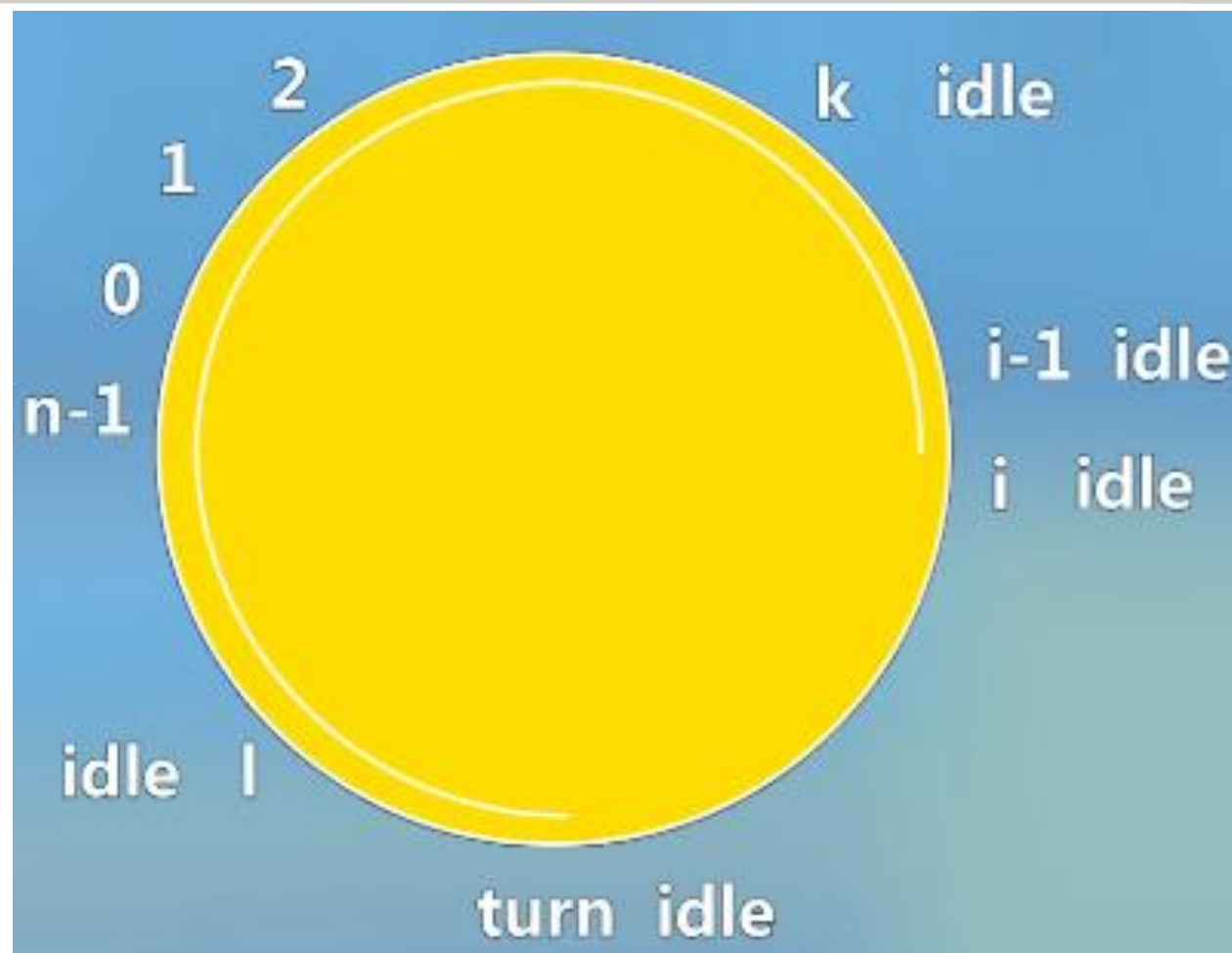
flag[i]==idle: 进程Pi不想进入临界区

flag[i]==want_in: 进程Pi想进入临界区

flag[i]==in_cs: 进程Pi想进入或已进入临界区

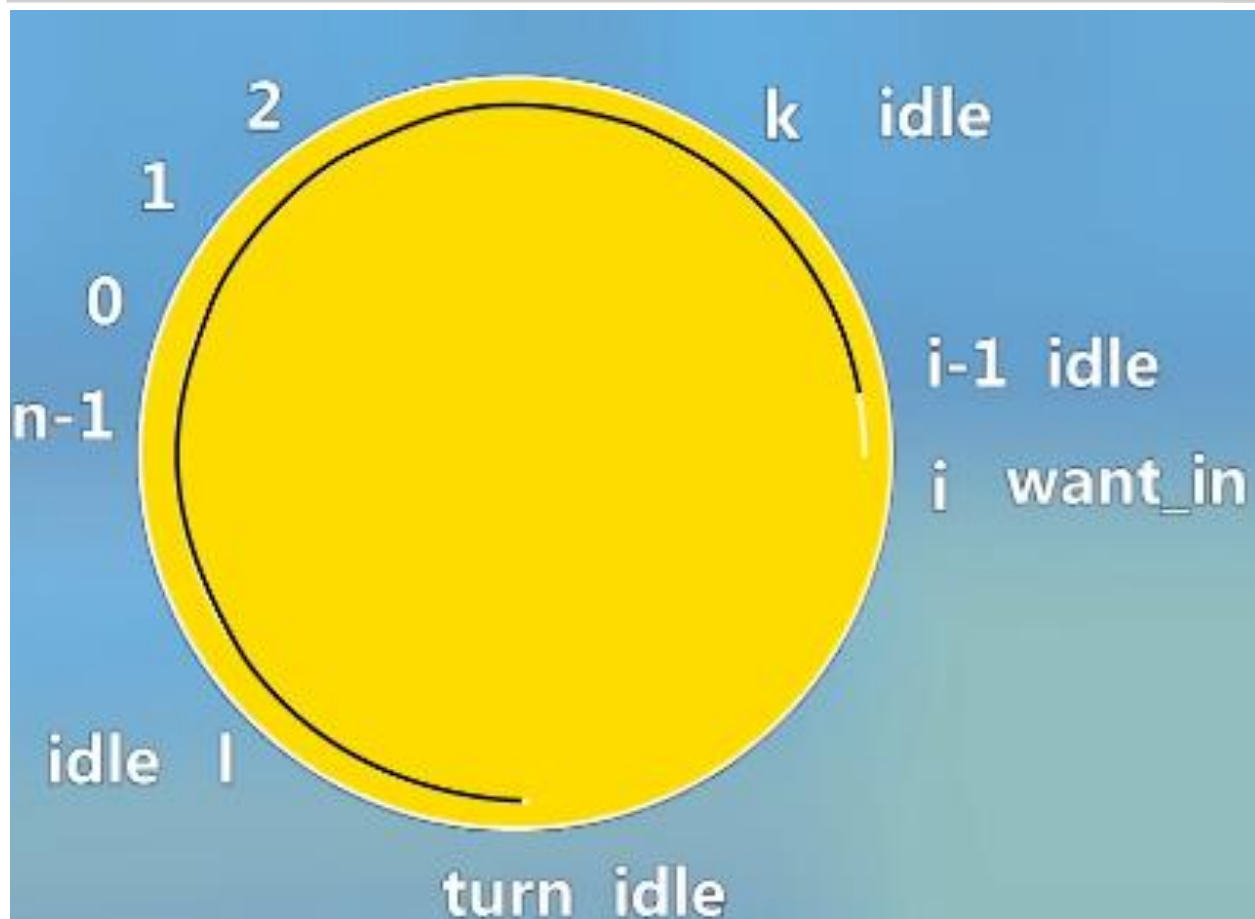


Eisenberg/Mcguire算法 (Cont.)



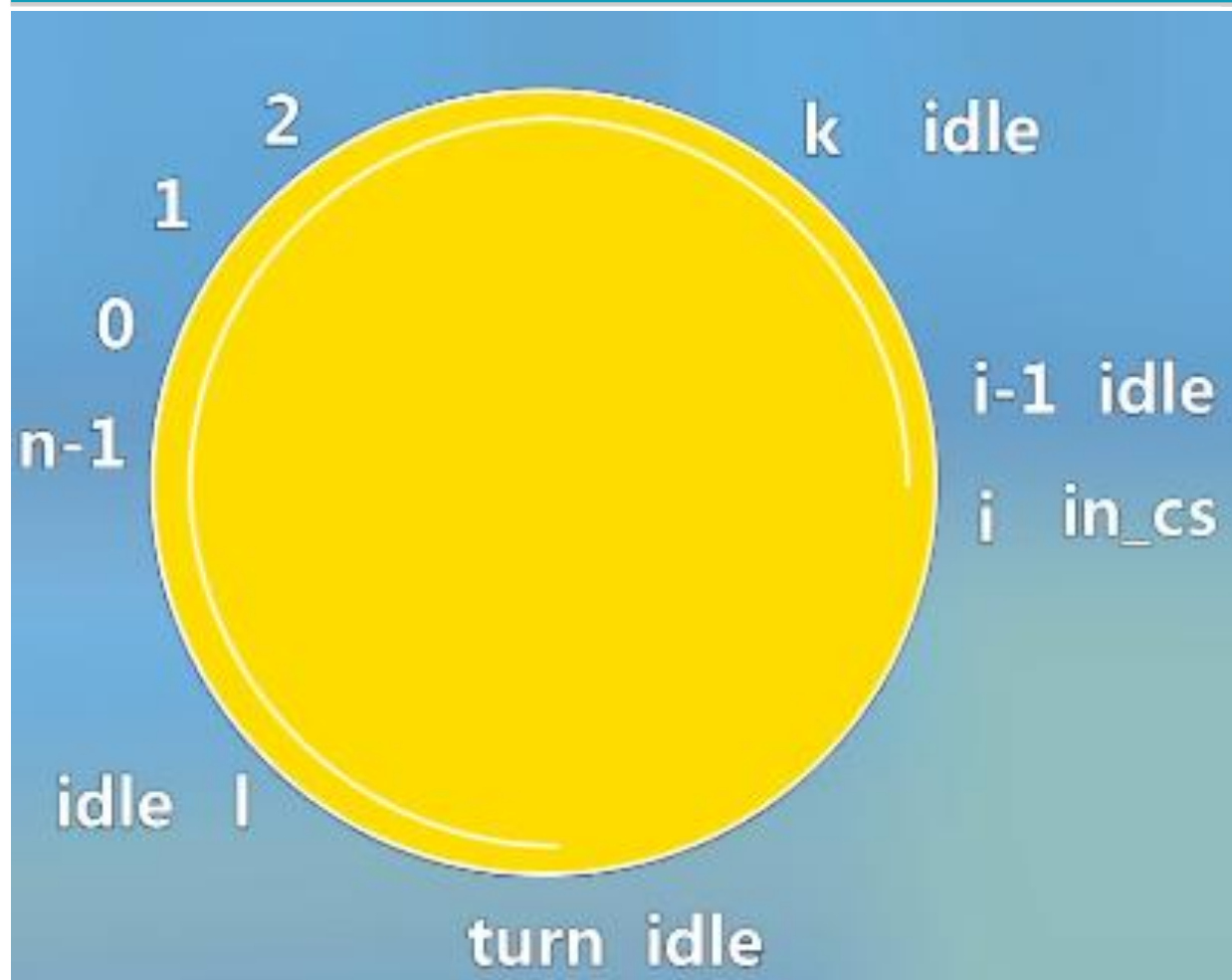
将n个进程
首尾相连

Eisenberg/Mcguire算法 (Cont.)



如果 P_i 想要进入临界区，将状态置为`want_in`，还需要看从`turn`开始到 $i-1$ 是否有想要进入临界区的进程。

Eisenberg/Mcguire算法 (Cont.)



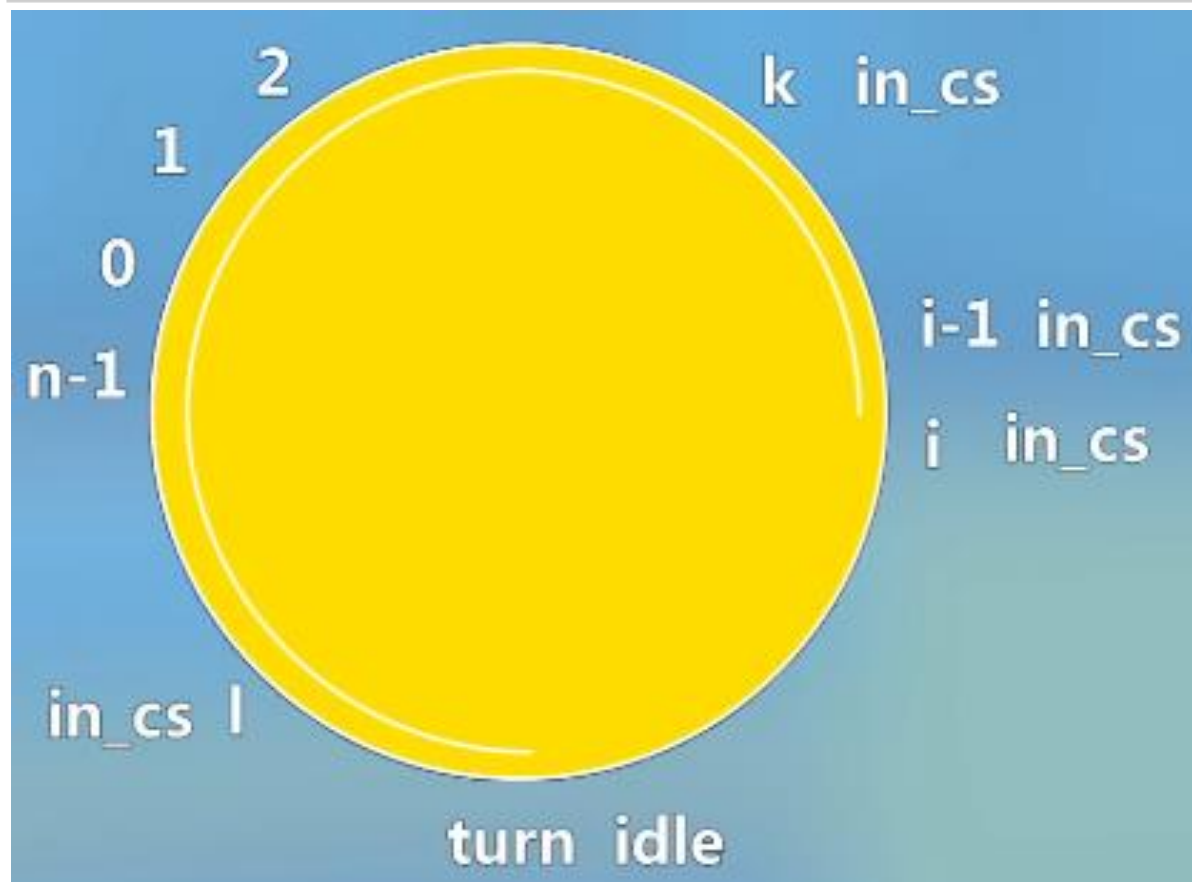
如果从turn开始到 $i-1$ ，这些进程都不想进入临界区，则 P_i 有机会进入临界区，将状态置为 in_cs。

Eisenberg/Mcguire算法 (Cont.)

问题：

- 进程 P_i 能否进入临界区？
 - 不一定
 - 如果当 i 号进程扫描 $i-1$ 号进程时是idle的，但扫描过去后， $i-1$ 号进程也想进入临界区，将自己的状态置为want_in，接下来， $i-1$ 号进程将扫描从turn开始到 $i-2$ ，如果这些进程状态都是idle的，也会将自己的状态置为in_cs
 - 同样，对于 l, k 号进程，也会将自己状态置为in_cs

Eisenberg/Mcguire算法 (Cont.)



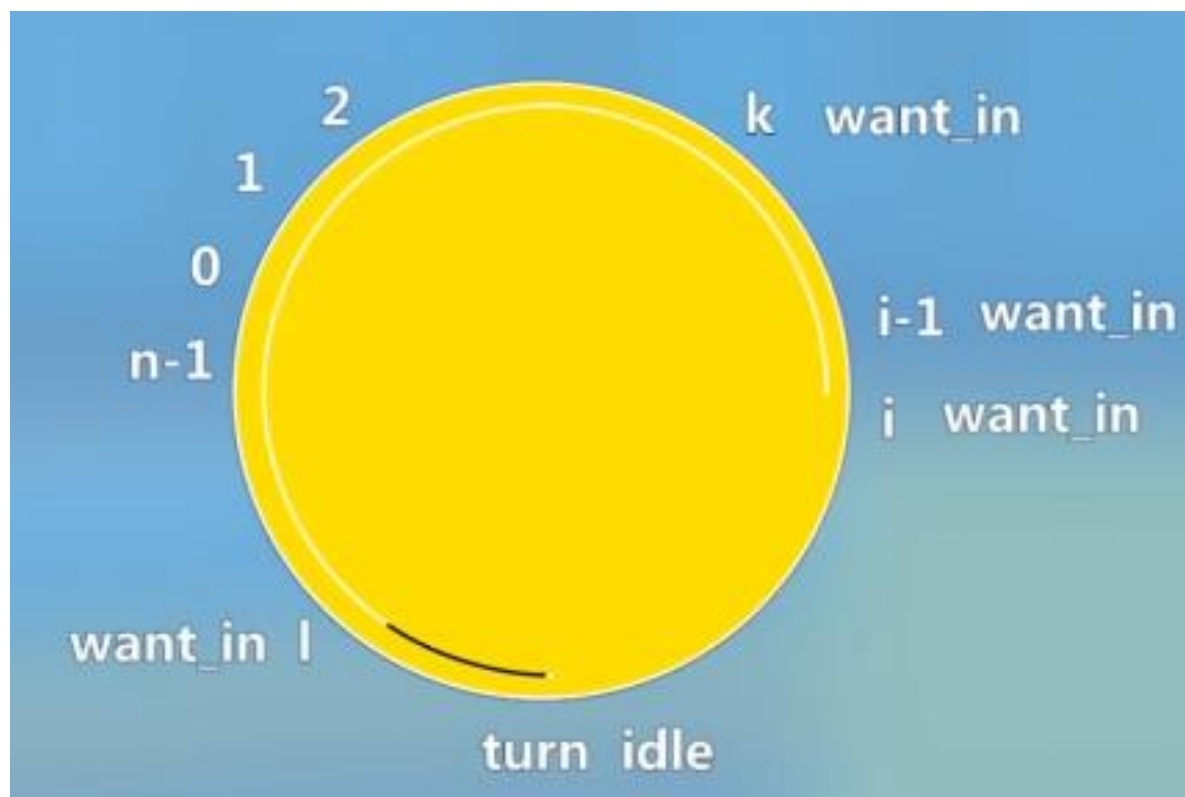
多个进程状态都是 in_cs。如何处理？

Eisenberg/Mcguire算法 (Cont.)



- 从头来。将进程状态置为want_in
- 从新扫描，那么对于i号进程，扫描turn到i-1时，所有进程状态不都是idle了，不能通过检查，不能将自己状态置为in_cs。
- 其它进程同理。

Eisenberg/Mcguire算法 (Cont.)



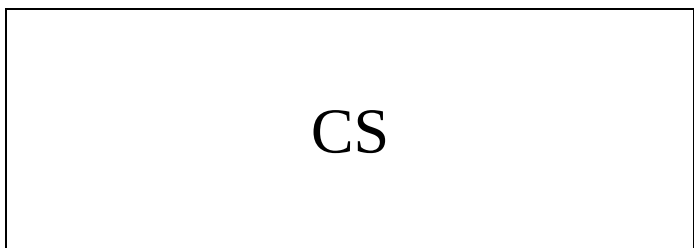
- 针对I号进程，如果在从turn到I-1，进程状态都为idle，则I号进程将自己状态置为in_cs。
- 思想：一个进程将自己状态置为in_cs后，要检测其它进程状态是否为in_cs，如果都不是in_cs，则可以进入临界区。反过来，如果除了自己外，还有其它进程状态为In_cs，则从头来。

Eisenberg/Mcguire算法 (Cont.)

Pi进入:

```
Do{
    flag[i]=want_in; // Pi想进入临界区
    j=turn; //全局变量，给出轮到哪个进程进入临界区了
    While (j!=i) //从j开始到i-1，如果所有进程状态都为idle，跳出循环
        //否则，重新扫描
        If (flag[j] != idle) j=turn //从头来，重新扫描
        Else j=(j+1)% n;
    flag[i]=in_cs;
    j=0;
    While (j<n)and(j==i or flag[j]!=in_cs) do //其它进程是否为in_cs
        j++;
}while (j!=n); //遍历一遍，j=n时表示无其它进程为in_cs， Pi可以进入
//如果还有进程状态是in_cs，则从头来
turn=i;
```

Eisenberg/Mcguire算法 (Cont.)



P_i 离开:

$j = (\text{turn} + 1) \% n;$

While ($\text{flag}[j] \neq \text{idle}$)

$j = (j + 1) \% n;$

$\text{turn} = j;$

$\text{flag}[i] = \text{idle};$

Eisenberg/Mcguire满足临界区管理原则

- 互斥性
 - 仅当 $\text{flag}[i] == \text{in_cs}$ ，且对所有 $j \neq i$ ， $\text{flag}[j] \neq \text{in_cs}$ 时，进程 P_i 才进入临界区域，因而满足互斥性
- 进展性
 - 临界区空闲时，排在序列 $\text{turn}, \text{turn}+1, \dots, n-1, 0, 1, 2, \dots, \text{turn}-1$ 最前面的申请进入临界区的进程获准进入临界区，因而满足进展性
- 有限等待性
 - 进程离开临界区时，按循环次序 $\text{turn}+1, \dots, n-1, 0, 1, 2, \dots, \text{turn}-1$ 确定唯一一个竞争进程为其后继，所以一个进程最多等待 $n-1$ 个进程进入并离开临界区后一定能进入临界区，因而满足有限等待性

4.2.3.2 进程互斥的硬件实现

■软件互斥算法一般适用于单处理机系统，多机环境下可并行执行的指令的效果可能是重排序执行的结果，会不满足互斥性

P0

turn=1; 2

Flag[0]=1 3

While(flag[1]&&turn==1); 4

临界区

P1

turn=0; 1

Flag[1]=1 5

While(flag[0]&&turn==0); 6

临界区

4.2.3.2 进程互斥的硬件实现

■ 硬件提供存储障碍语句

- 存储障碍（memory barrier）该指令出现在相继指令之间，保证前面指令先于后面指令执行

- 例如Peterson算法：flag[0]=1;

memory_barrier();

turn=1;

保证flag[0]=1和turn=1的次序不被重排序，从而保证算法在多处处理环境下的正确性

4.2.3.2 进程互斥的硬件实现

■ 硬件提供原子变量

- 原子变量 (atomic variable): 对一个基本变量的访问与修改不被分割的变量
- 例如, 将一个内存变量加1的操作一般是由以下指令完成:

`r1 ← count`

`r1++`

`count ← r1`

如果这三条指令中途被打断, 与其它对count的操作交替执行, 则可能不正确, 为此在现代计算机系统中都支持原子变量, 若上例中count被说明为原子变量, 则对count 的上述三条汇编指令就不会被打断。

- 有这样的硬件支持, 实现互斥会更容易, 但请注意, 原子变量并没有完全解决竞争条件的问题

4.2.3.2 进程互斥的硬件实现

1. 硬件提供“测试并设置”指令：将内存中一个单元的值取出，再送一个值。为原子操作。

```
int test_and_set(int *lock) {  
    int temp;  
    temp=lock;  
    *lock=1;  
    return(temp);  
}.
```

4.2.3.2 进程互斥的硬件实现（Cont.）

基于“测试并设置”指令的互斥算法

对一组公共变量，`int lock; (初始=0);`

`Pi` 进入：`do {`

`While test_and_set(&lock)`

`skip;`

临界区

`lock=0;`

其余代码

`} while(1);`

满足互斥性，进展性，不满足有限等待性。

4.2.3.2 进程互斥的硬件实现 (Cont.)

满足有限等待性:

全局变量:

```
int waiting[n];
```

```
int lock;
```

局部变量:

```
int j;
```

```
int key;
```

```
waiting[i]=1; key=1;
```

```
While(waiting[i]&&key)
```

```
    key=test_and_set(&lock);
```

```
waiting[i]=0;
```

临界区

```
J=(i+1)%n;
```

```
While(((j!=i)&&(!waiting[j])))
```

```
    j=(j+1)%n;
```

```
If(j==i) lock=0;
```

```
Else waiting[j]=0;
```

其余部分

4.2.3.2 进程互斥的硬件实现（Cont.）

2. 硬件提供“交换”指令：将内存中两个单元的内容相互交换，为原子操作。

```
void swap(int &a, &b) {  
    int temp;  
    temp:=*a; *a:=*b; *b:=temp;  
};
```

4.2.3.2 进程互斥的硬件实现（Cont.）

基于“交换”指令的互斥算法

```
int lock; int key;
```

```
Pi进入: do{
```

```
    key=1;
```

```
    do{
```

```
        swap(&lock, &key);
```

```
    } while (key==1);
```

```
    临界区
```

```
    lock=0;
```

```
    其余代码
```

```
    } while (1);
```

满足互斥性，进展性，不满足有限等待性。

4.2.3.2 进程互斥的硬件实现（Cont.）

Remarks:

- (1) test_and_set指令和swap指令是原子的，不可中断的（单CPU）
- (2) test_and_set实际上是：将内存中一个单元的内容取出，再送一个新值
- (3) swap实际上是：交换内存两个单元的内容

4.2.3.2 进程互斥的硬件实现 (Cont.)

3. 硬件提供“关中断”和“开中断”指令：

关中断

{ Critical Region }

开中断

Remarks:

- (1) 开关中断只在单CPU系统中有效；
- (2) 限制了处理器交替执行程序的能力，影响并发性。
- (3) 只能操作系统使用。

4.2.3.2 进程互斥的硬件实现

■ 开关中断特点

- 由于中断是进程切换的必要条件，所以关中断后不会发生进程切换，因而满足互斥性
- 实现效率高，没有忙式等待，简单易行
- 但会影响系统并发性

-
- 思考1：为何开关中断进程互斥方法仅在单CPU系统中是有效的？
 - 思考2：临界区大小与系统并发性之间的关系？

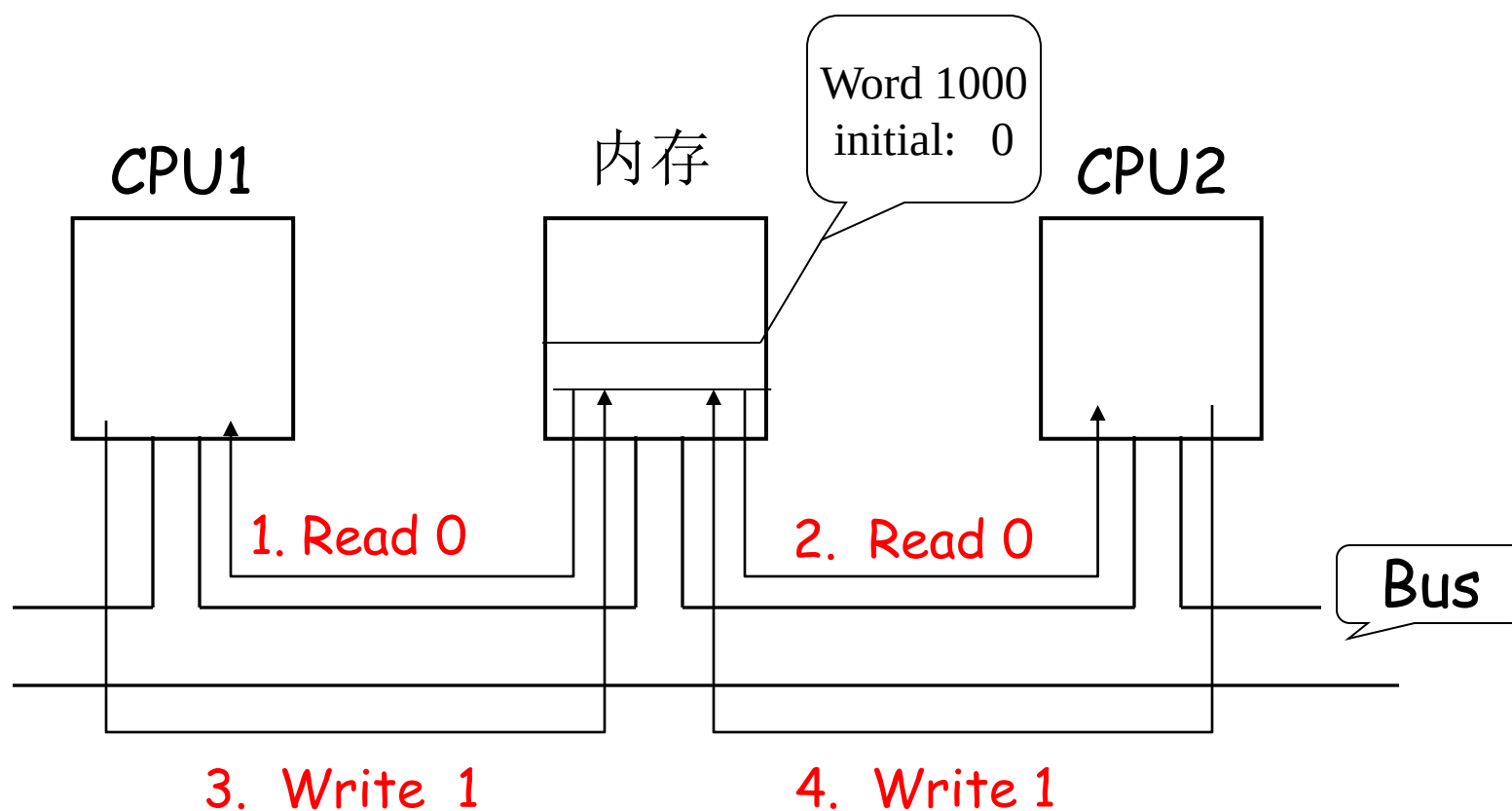
4.2.3.2 进程互斥的硬件实现（Cont.）

- 硬件互斥方法的优点
 - 适用范围广
 - 简单
 - 支持多个临界区
- 硬件互斥方法的缺点
 - 进程等待进入临界区时要消耗处理机时间，不能实现让权等待

4.2.4 多处理机环境下互斥

- 单CPU, 指令间交叉, `test_and_set`, `swap`指令是原子的
- 多CPU, 指令周期间交叉, `test_and_set`, `swap`指令不是原子的

4.2.4 多处理机环境下互斥 (Cont.)



4.2.4 多处理机环境下互斥 (Cont.)

TS指令, Swap指令: first lock the bus

bus request protocol:

modern buses have these facilities

earlier ones didn't

4.2.4 多处理机环境下互斥（Cont.）

```
do{  
    lock(bus);  
    b = test_and_set(&lock);  
    unlock(bus);  
}while(b)
```



```
lock=0;
```

提纲

4.1 并发进程(concurrent processes)

4.2 进程互斥(mutual exclusion)

4.3 进程同步(synchronization)

4.4 进程高级通讯(communication)

4.3 进程同步

例：司机-售票员问题

司机活动：

do{

启动车辆

正常行驶

到站停车

}while(1)

售票员活动：

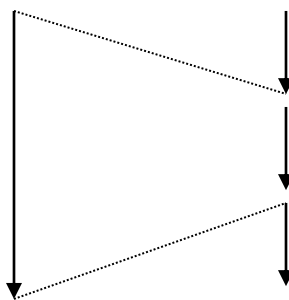
do{

关车门

售 票

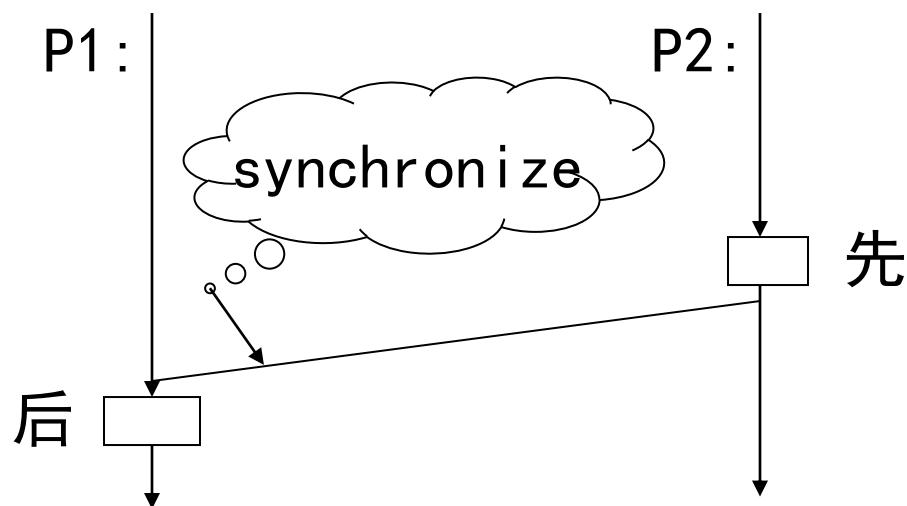
开车门

}while(1)



4.3.1 进程同步的概念

同步：一组进程，为了协调其推进速度，在某些点处需要相互等待或者相互唤醒，进程之间这种相互制约的关系称为进程同步。



进程同步是合作进程之间有意识的行为，只能发生在相关进程之间

4.3.2 进程同步机制

- **定义：** 用于实现进程同步的工具称为同步机制 (synchronization mechanism)
- **同步机制要求**
 - 描述能力够用
 - 可实现
 - 高效, 使用方便

典型同步机制

- 信号量与PV操作 (semaphore and PV operations)
- 管程 (monitor)
- 条件临界区 (conditional critical region)
- 会合 (rendezvous)
- 路径表达式 (path expression)
- 事件 (event, traditional UNIX).

4.3.3 信号量与PV操作

E.W.Dijkstra, 1965. P-Proberen, V-Vehogen

4.3.3.1 信号量与PV操作的定义

```
Typedef semaphore struct{  
    int value;  
    PCBpointer queue;  
};
```

semaphore s;

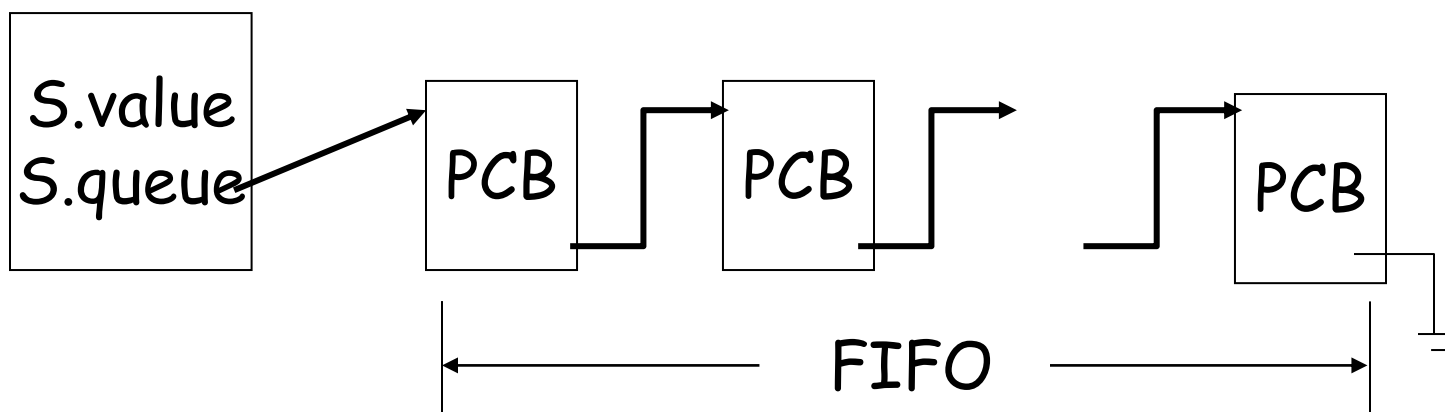
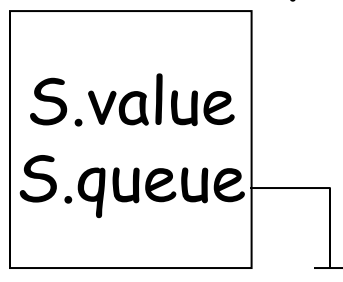
Remarks:

(1) *semaphore* is pre-defined data type

(2) *s* can be declared as needed, eg. semaphore s1,s2

信号量变量

Var S:semaphore;



定义了信号量变量，相当于定义了一个FIFO队列

P操作原语

P操作原语:

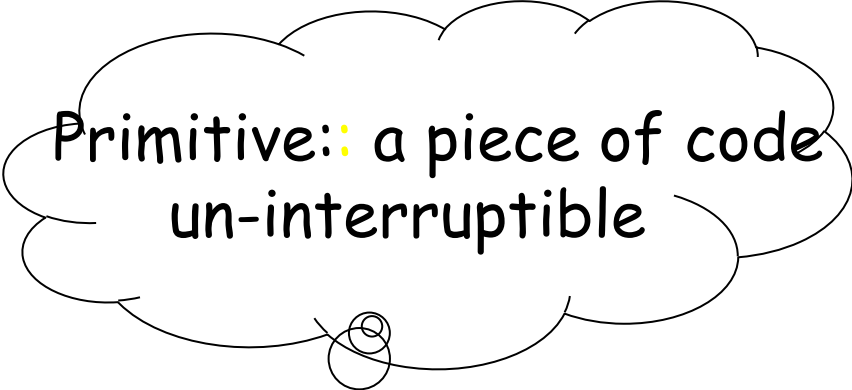
Procedure P(var s:semaphore)

 s.value:=s.value-1;

 If s.value<0 Then

 asleep(s.queue)

End



Primitive: a piece of code
un-interruptible

asleep(s.queue):

- (1) 执行此操作进程的PCB入s.queue尾（状态改为等待）
- (2) 转处理机调度程序，重新分派CPU

V操作原语

V操作原语：

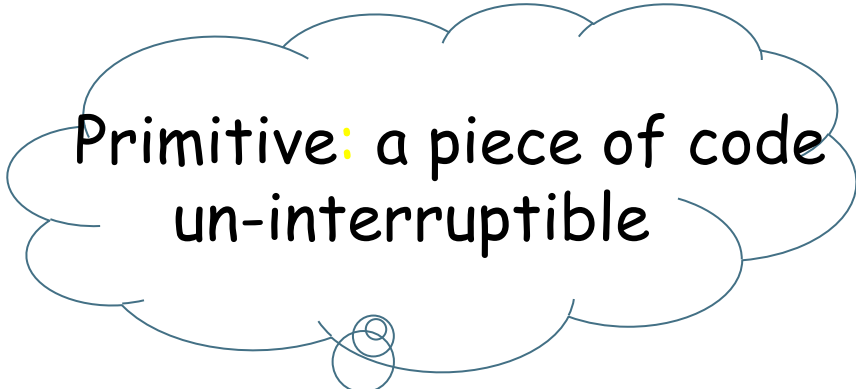
Procedure V(var s:semaphore)

 s.value:=s.value+1;

 If s.value<=0 Then

 wakeup(s.queue)

End



Primitive: a piece of code
un-interruptible

wakeup(s.queue)

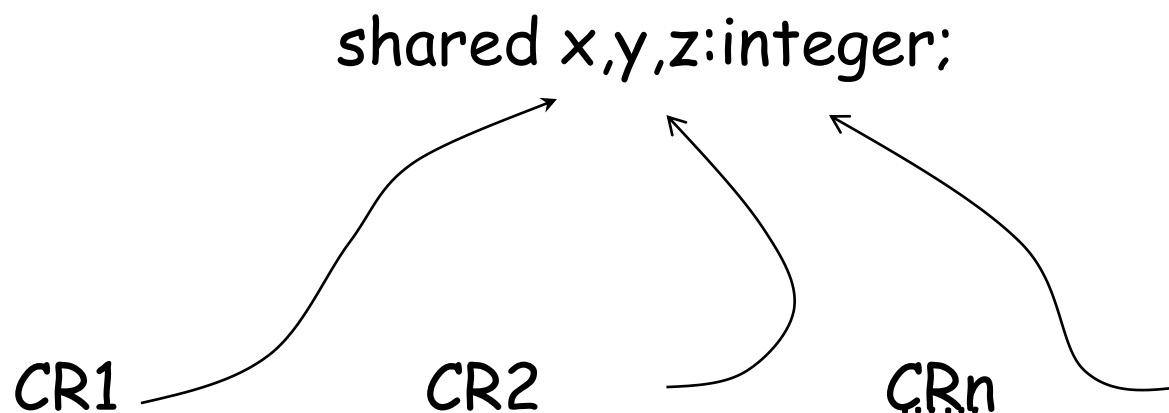
s.queue链头PCB出等待队列，进入就绪队列（状态改为就绪）

规定和结论

- 对于信号量变量的规定
 - 必须置一次初值，只能置一次初值，初值 ≥ 0
 - 只能执行P操作和V操作，所有其它操作非法
- 几个有用的结论
 - 当 $s.value \geq 0$ 时， $s.queue$ 为空
 - 当 $s.value < 0$ 时， $|s.value|$ 为队列 $s.queue$ 的长度
 - 当 $s.value_{初}=1$ 时，可以实现进程互斥
 - 当 $s.value_{初}=0$ 时，可以实现进程同步
 - 当 $s.value$ 的初值为非1的正整数时，可以用来管理同种组合资源，申请时执行一次P操作，归还时执行一次V操作

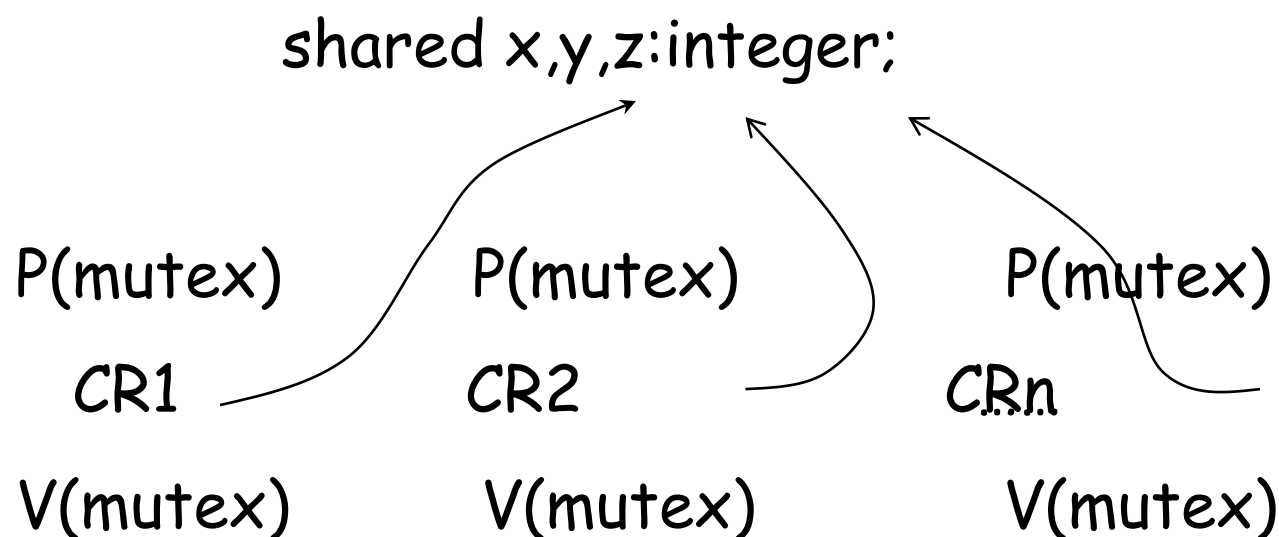
用信号量实现进程互斥

Var mutex: semaphore; (初值=1)



用信号量实现进程互斥

Var mutex: semaphore; (初值=1)



互斥例子：借书系统(revisited)

- Var mutex:semaphore; (initial value is 1)
- 终端1: 终端2:
- CYCLE CYCLE
- 等待借书者; 等待借书者;

- IF $x \geq 1$ Then IF $x \geq 1$ Then
- Begin Begin
- $x := x - 1$; $x := x - 1$;
- 借书
- 借书
- End End
- Else 无书; Else 无书;
- End End

互斥例子：借书系统(revisited)

Var mutex:semaphore; (initial value is 1)

终端1:

CYCLE

等待借书者;

P(mutex)

IF $x \geq 1$ Then

Begin

$x := x - 1$;

V(mutex)

借书

End

Else V(mutex);无书;

End

终端2:

CYCLE

等待借书者;

P(mutex)

IF $x \geq 1$ Then

Begin

$x := x - 1$;

V(mutex)

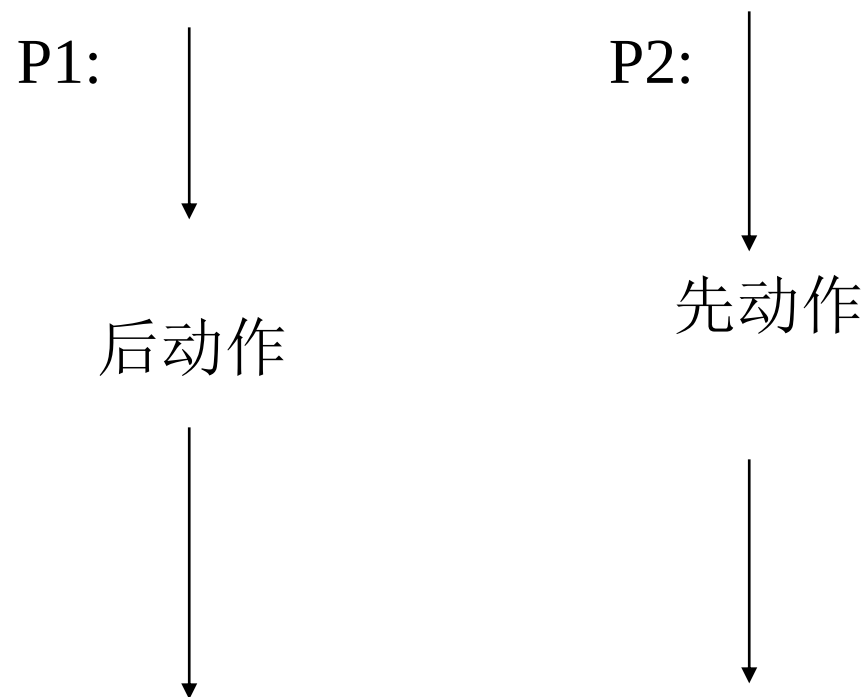
借书

End

Else V(mutex);无书;

End

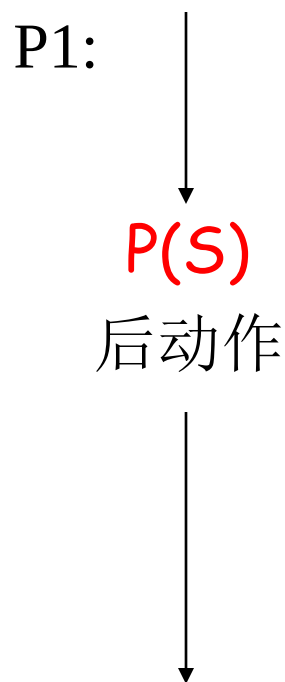
用信号量实现进程同步



用信号量实现进程同步 (Cont.)

General Case:

VAR S:semaphore; (initial value 0)



用信号量实现进程同步（Cont.）

- 例子：司机-售票员问题：

- 司机活动：

- Do {
 - 启动车辆
 - 正常行驶
 - 到站停车
- } While (1)

- 售票员活动：

- Do {
 - 关车门
 - 售票
 - 开车门
- } While (1)

用信号量实现进程同步（Cont.）

例子：司机-售票员问题：

VAR s1, s2: semaphore; (initial value 0)

司机活动：

Do {

P(S1)

启动车辆

正常行驶

到站停车

V(S2)

} While(1)

售票员活动：

Do {

关车门

V(S1)

售 票

P(S2)

开车门

} While(1)

用信号量实现资源管理

预备知识：

组合资源：若干相对独立的资源构成的资源集合，其中每个相对独立的资源称为子资源。

同种组合资源：相同类型的子资源构成组合资源。

管理：Var S:semaphore; (初值=子资源个数)

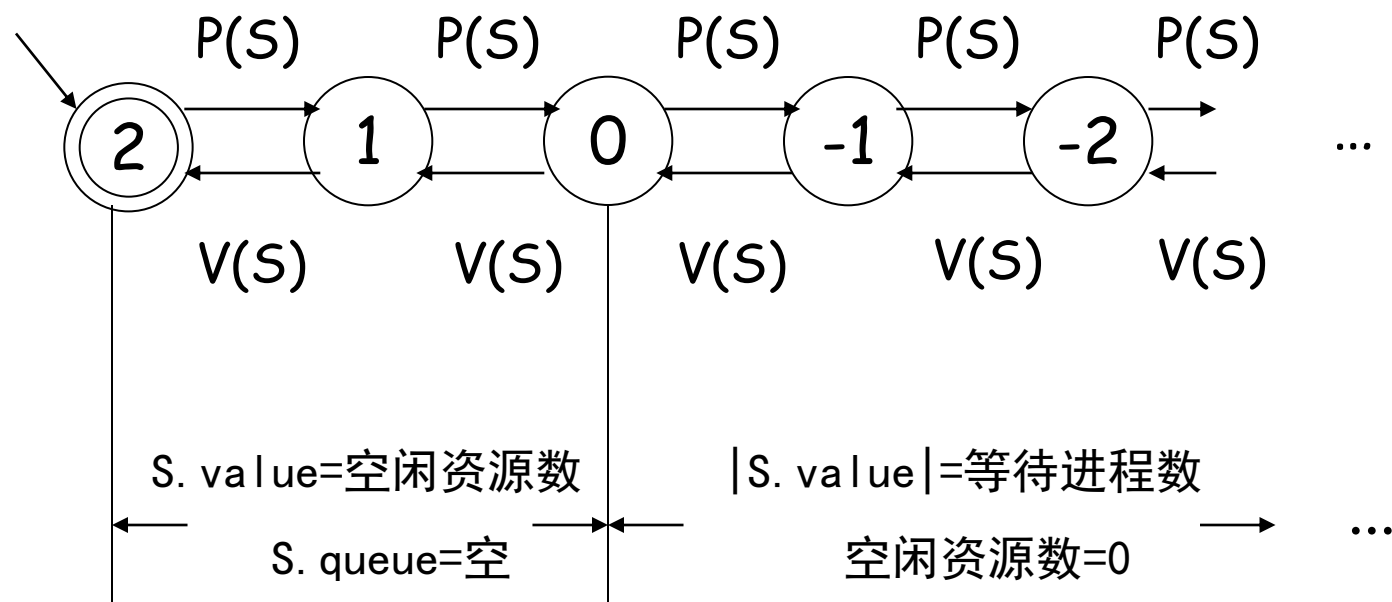
例子：2台打印机

Var S:semaphore; S.value=2;

申请：P (S) ;

释放：V (S) ;

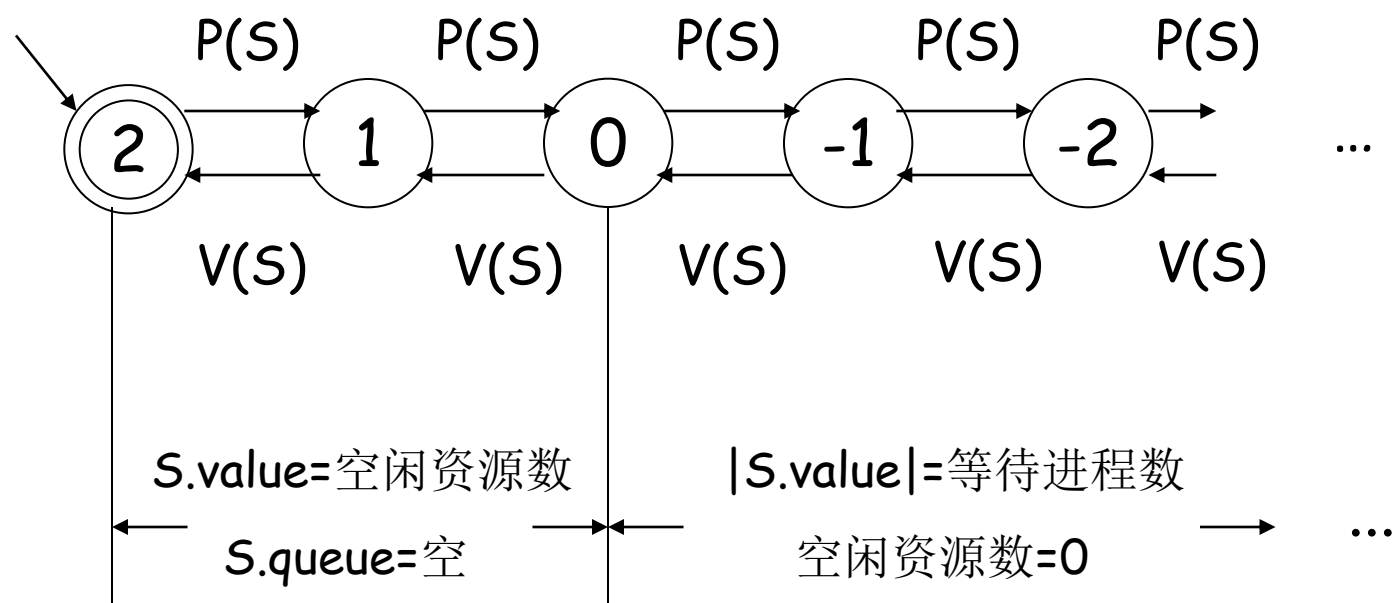
自动机描述



$P(S)$: $S > 0$ 时, 申请一台打印机, 分配, 空闲打印机减1

$P(S)$: $S \leq 0$ 时, 申请一台打印机, 等待, 等待进程数加1

自动机描述 (Cont.)



$V(S)$: $S < 0$, 释放一台打印机, 唤醒一个等待者, 打印机分给被唤醒进程, 等待进程数减1

$V(S)$: $S \geq 0$, 释放一台打印机, 空闲打印机数量增1

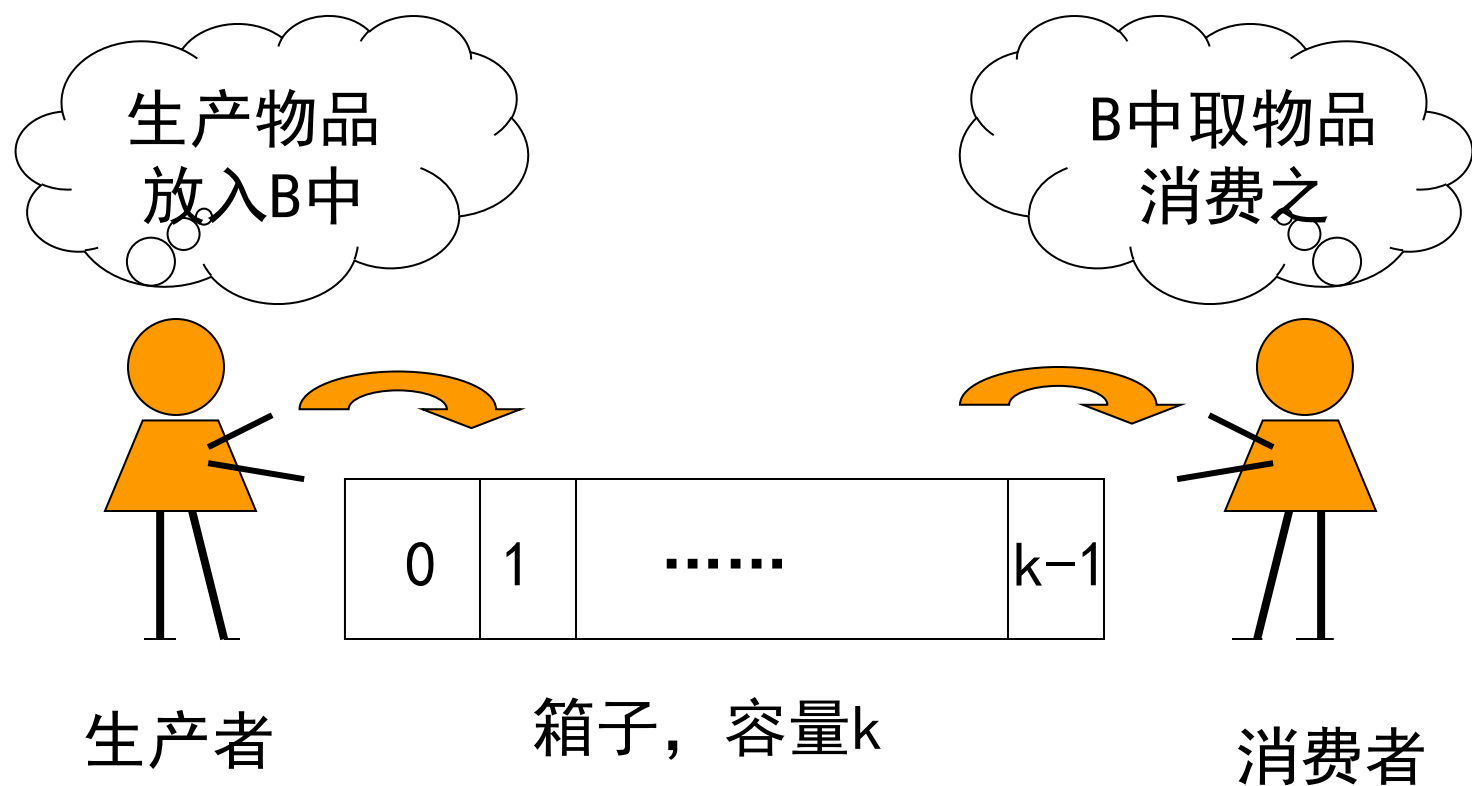
■ Remarks

- P操作相当于申请资源，V操作相当于释放资源。
- P操作和V操作在系统中总是成对出现的，它们即可以出现在同一进程中，也可以出现在不同的进程中
- 同步进程之间具有某种合作关系，如在执行时间上必须按一定的顺序协调进行，或者共享某种资源
- 互斥进程彼此在逻辑上完全无关，它们的运行不具有次序的特征

Classical synchronization problems

1. Producers and consumers problem
 2. Readers and writers problem
 3. Dining philosophers problem
 4. Cigarette smokers problem
 5. Sleepy barbers problem
- etc.

例1. 生产者/消费者问题

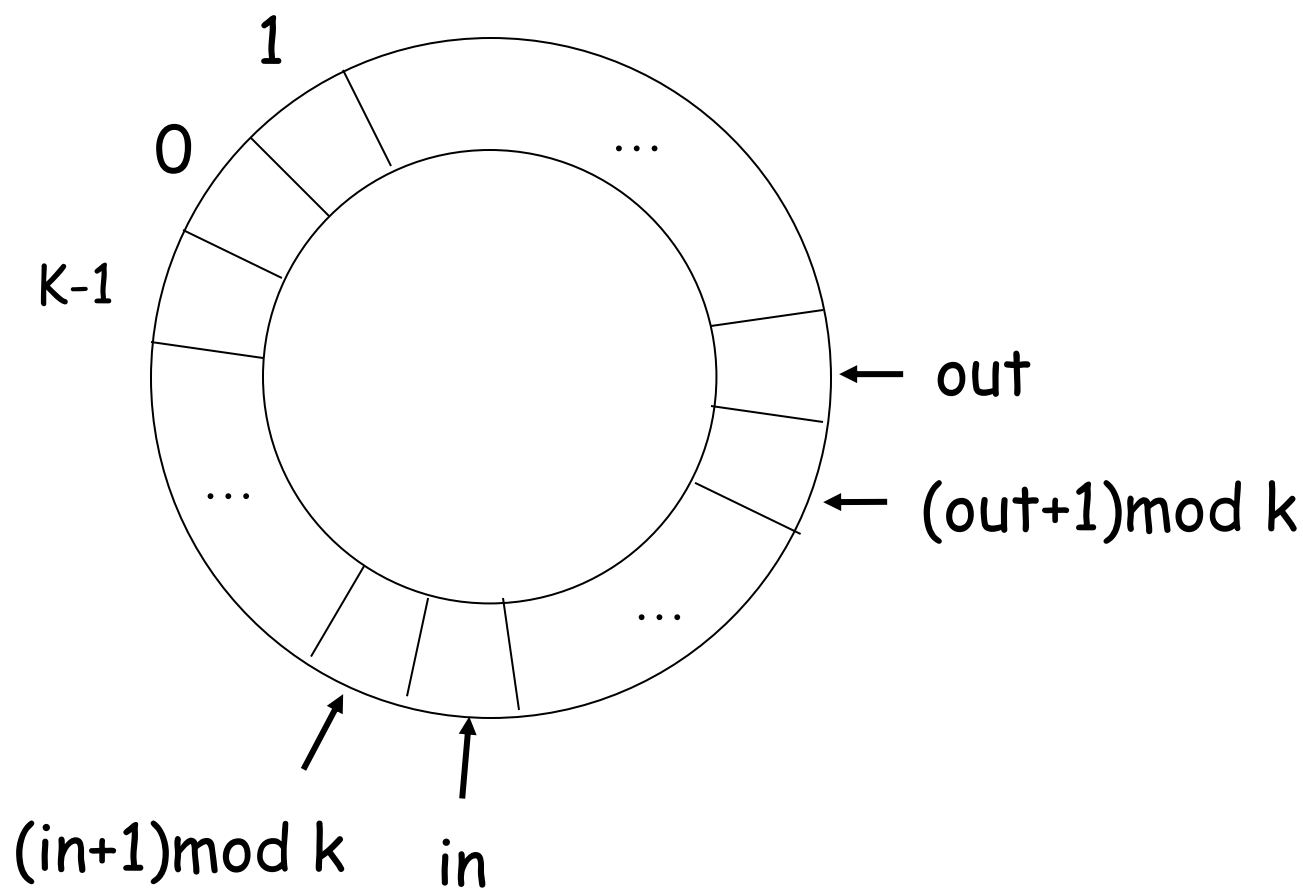


$B: \text{Array}[0..k-1] \text{ of item}$

箱子可以看做是一个缓冲区，且这个缓冲区是有限的，编号从0到k-1，容量是k。所以生产消费问题也叫有界缓冲区问题。

环形缓冲区

为了方便编程，将缓冲区设置为环形。In：放入指针，out：取出指针



问题分析

生产者活动:

do{

 加工一件物品

 物品放入箱中

}while(1)

资源: 空位 (同种组合)

Var S1:semaphore;

 S1.value=k;

放前: P (S1)

放后: V (S2)

消费者活动:

do{

 箱中取一物品

 消耗这件物品

}while(1)

资源: 物品 (同种组合)

Var S2:semaphore;

 S2.value=0;

取前: P (S2)

取后: V (S1)

同步

生产者活动:

Do{

加工一件物品

P(S1)

物品放入箱中

V(S2)

}While(1)

消费者活动:

Do{

P(S2)

箱中取一物品

V(S1)

消耗这件物品

}While(1)

箱子是临界资源，需实现互斥

Var mutex:semaphore; (mutex.value=1)

互斥

生产者活动:

```
Do{  
    加工一件物品  
    P(S1)  
    P(mutex)  
    物品放入箱中  
    V(mutex)  
    V(S2)  
}While(1)
```

消费者活动:

```
Do{  
    P(S2)  
    P(mutex)  
    箱中取一物品  
    V(mutex)  
    V(S1)  
    消耗这件物品  
}While(1)
```

程序

Program producers_consumers;

Var B:Array[0,...,k-1]Of item;

S1,S2,mutex:semaphore;

in,out:0..k-1;

Procedure producer

cycle

produce a product

P(S1);

P(mutex);

B[in]:=product;

in:=(in+1)mod k;

V(mutex);

V(S2)

end

Procedure consumer

cycle

P(s2);

P(mutex);

x:=B[out];

out:=(out+1)mod k;

V(mutex);

V(S1);

consume x;

end;

程序 (Cont.)

Begin

S1.value:=k; S2.value:=0;

mutex.value:=1;

in:=0; out:=0;

Cobegin

P1: producer; , Pm: producer;

C1: consumer;, Cn: consumer;

Coend;

End.

并发性提高策略

生产者和消费者：不操作B的相同分量（还没生产，消费者取）。

生产者的共享变量： 消费者的共享变量：

$B[in]$, in

$B[out]$, out

$in=out$ ：满或空

Var mutex1, mutex2: semaphore; (init 1)

并发性提高策略 (Cont.)

生产者活动:

Repeat

加工一件物品

P(S1)

P(mutex1)

物品放入箱中

V(mutex1)

V(S2)

Until false

消费者活动:

Repeat

P(S2)

P(mutex2)

箱中取一物品

V(mutex2)

V(S1)

消耗这件物品

Until false

例2. 读者/写者问题

P. T. Courtois 1971

Communication of the ACM, Vol.14, 667-669.

ACM: Association for Computing Machinery

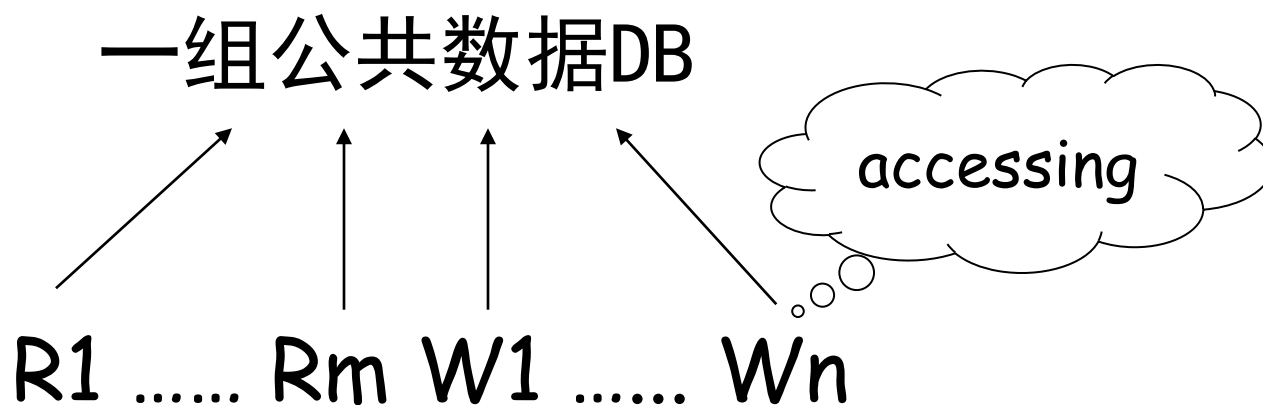
解法1：写者可能饿死

解法2：公平性解法

解法3：写者优先

例2. 读者/写者问题 (Cont.)

Problem Statement: 一组公共变量(数据)，有2组进程访问，它们都要访问这组公共数据，一组对公共数据进行读，一组对公共数据进行写。有m个读者，n个写者。这些读者读取这些数据，但是不修改数据。写者访问数据时可以修改数据。



例2. 读者/写者问题 (Cont.)

为了保证共享数据的完整性，要求：

- (1) R-R可以同时：读者不修改数据，不影响其它进程，可同时
- (2) R-W不可同时：有读者，不能有写者；有写者，不能有读者
- (3) W-W不可同时：如果同时写，数据是不确定的

Solution1: 考虑R-R互斥

Var r_w_w:semaphore; (init value: 1)

Reader:

P(r_w_w);

{读操作}

V(r_w_w);

Writer:

P(r_w_w)

{写操作}

V(r_w_w)

分析: (1)写者活动正确; (2)R-R不能同时

改进: 最先进入的R执行P; 最后离开的R执行V

Solution2: 考虑R-R不互斥 (Cont.)

Var read_count:integer; (initial value is 0)

Reader:

read_count:=read_count+1;

If read_count=1 Then P(r_w_w);

{读操作}

read_count:=read_count-1;

If read_count=0 Then V(r_w_w);

问题：为了记录读者个数，引入了Read_count变量，该变量是所有读者都需要访问的，需对Read_count操作实现互斥。

Solution3: 正确解法

```
Var  r_w_w, mutex: semaphore; (initial value is 1)
```

```
int  read_count:=0;
```

```
Reader:
```

```
    P(mutex);
```

```
    read_count:=read_count+1;
```

```
    If read_count=1 Then P(r_w_w);
```

```
    V(mutex);
```

```
    {读操作}
```

```
    P(mutex);
```

```
    read_count:=read_count-1;
```

```
    If read_count=0 Then V(r_w_w);
```

```
    V(mutex);
```

读者/写者程序

```
Program readers_writers;  
  Var r_w_w: semaphore;  
      mutex: semaphore;  
      read_count: integer;  
  
  procedure writer;  
  begin  
    P(r_w_w);  
    { write ops }  
    V(r_w_w)  
  end;
```


读者/写者程序 (Cont.)

```
Procedure reader;  
begin  
    P(mutex);  
    read_count:=read_count+1;  
    If read_count=1 Then P(r_w_w);  
    V(mutex);  
    { read ops }  
    P(mutex);  
    read_count:=read_count-1;  
    If read_count=0 Then V(r_w_w);  
    V(mutex);  
end;
```

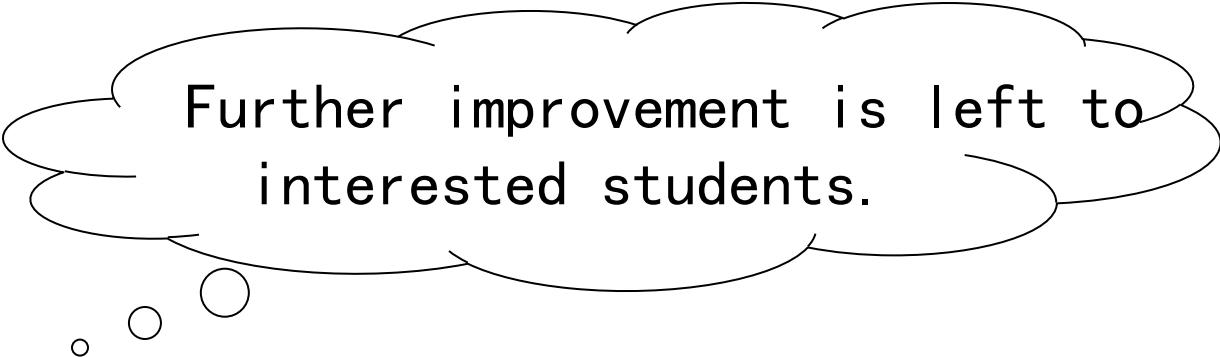
读者/写者程序 (Cont.)

```
begin
  read_count:=0;
  r_w_w.value:=1;
  mutex.value:=1;
  cobegin
    r1: reader; .....; rm: reader;
    w1: writer; .....,; wn: writer
  coend
end.
```

分析

问题：读者源源不断，`read_count`不归0，写者会被饿死。

策略：公平性解法，写者优先解法。



Further improvement is left to interested students.

写者优先

```
Semaphore r_w_w =1; //用于控制互斥访问数据区  
Semaphore rmutex =1; //用于读者互斥访问read_count  
Semaphore wmutex =1; //用于写者互斥访问write_count  
Semaphore read_q =1; //用于表示当前是否有写者  
int read_count=0, write_count=0;
```

一旦有写者等待，新到达读者等待，正在读的读者都结束后，写者进入。

写者优先

Reader:

P(read_q); //检测是否有写者存在，无写者时进入

P(rmutex);

read_count:=read_count+1;

If read_count=1 Then P(r_w_w);

V(rmutex);

V(read_q); //释放read_q，允许其它读者或写者占用
{读操作}

P(rmutex);

read_count:=read_count-1;

If read_count=0 Then V(r_w_w);

V(rmutex);

写者优先

writer;

P(wmutex);

write_count:=write_count+1;

If write_count=1 Then P(read_q); //阻止其他读者进入

V(wmutex);

P(r_w_w); //阻止其它写者进入

{ write ops }

V(r_w_w)

P(wmutex);

write_count:=write_count-1;

If write_count=0 Then V(read_q); //若是最后一个写者，

V(wmutex); 则允许读者进入

例3. 障栅（barrier）问题

- 问题：若干进程并发执行，向着一个障栅推进，每个进程到达障栅时将等待，直到所有进程均到达障栅，之后所有进程越过障栅继续向前推进。
- 解决：把障栅之前的活动看作一道工序 $W1$ ，障栅之后的活动看作下一道工序 $W2$ ，

例3. 障栅（barrier）问题

```
int count;(0)
semaphore barrier,mutex;(0, 1)
Pi进程活动:
  工序W1;
  P(mutex);
  count++;
  if count==n
    for(i=1;i<=n;i++)
      V(barrier);
    endfor
  endif
  V(mutex)
  P(barrier);
  工序W2;
```


例4. 3台打印机管理

设有3台类型相同的打印机，其编号分别为1、2、3，试编写一个申请函数`require`和一个释放函数`return`。当有打印机空闲时，返回分得的打印机的编号；当无打印机空闲时则等待，被唤醒后返回分得的打印机的编号。`Return`用于释放指定编号的空闲打印机，当有申请者等待时就将其一唤醒。

例4. 3台打印机管理(Cont.)

Var lp:array[1..3]of (free,used); (initial value is free)

S: semaphore; (initial value is 3)

mutex: semaphore; (initial value is 1)

procedure require(i:1..3);

 P(S);

 P(mutex);

 for i:=1 to 3 do

 if lp[i]=free then

 break;

 lp[i]:=used;

 V(mutex);

 return(i);

procedure return(i:1..3);

 P(mutex);

 lp[i]:=free;

 V(mutex);

 V(S);

例5. 吸烟者问题

- Cigarette Smokers' Problem
 - Patil S. S. Limitations and Capabilities of Dijkstra's semaphore primitives for coordination among processes. MIT project MAC Computation Structure Group Memo 57, MIT, Cambridge, Mass, 1977.

吸烟者问题-problem statement

3 supplier processes:

X: supplies tobacco and match;

Y: supplies match and wrapper;

Z: supplies wrapper and tobacco.

3 smoker processes:

A: possess tobacco;

B: possess match;

C: possess wrapper.

(1) only one of X,Y,Z can supply at a time;

(2) X,Y,Z can proceed only after consumption.

Traditional semaphore:

Var t,m,w,s:semaphore; (0,0,0,1)

Process X

P(s);

V(t);

V(m);

Process A

P(m);

P(w);

smoke;

V(s);

process Y

P(s);

V(m);

V(w);

Process B

P(w);

P(t);

smoke;

V(s);

process Z

P(s);

V(w);

V(t);

process C

P(t);

P(m);

smoke;

V(s);

Traditional semaphore:

Solution1存在的问题: 供应者进程都供应特定的两种材料, 并且分别供应给固定的吸烟者进程, **Solution1** 存在另外两个吸烟者进程取单一材料的情况。

解决方法(1): 某供应者进程执行(提供特定两种材料)时, 应通知相应吸烟者进程;

解决方法(2): 引进并发 P、V 操作 SP和SV。

解决方法(1):

```
semaphore mutex_C;    /* Initial 0 */  
semaphore mutex_A;    /* Initial 0 */  
semaphore mutex_B;    /* Initial 0 */
```

Traditional semaphore:

process X

```
P(mutex);  
V(tobacco);  
V(match);  
V(mutex_C);
```

process Y

```
P(mutex);  
V(match);  
V(wrapper);  
V(mutex_A);
```

process Z

```
P(mutex);  
V(wrapper);  
V(tobacco);  
V(mutex_B);
```

process C

```
P(mutex_C);  
P(tobacco);  
P(match);  
smoke;  
V(mutex);
```

process A

```
P(mutex_A);  
P(match);  
P(wrapper);  
smoke;  
V(mutex);
```

process B

```
P(mutex_B);  
P(wrapper);  
P(tobacco);  
smoke;  
V(mutex);
```

Simultaneous P-operation

```
SP( $S_1, t_1, d_1; \dots; S_n, t_n, d_n$ ):  
if  $S_1 \geq t_1$  and ... and  $S_n \geq t_n$  then  
  for  $I:=1$  to  $n$  do  $S_i := S_i - d_i$  endfor  
else
```

- (1) 将运行进程的PCB连到第一个 $S_i < t_i$ 的队列中,
- (2) 将该进程的指令计数器内容设置为SP操作的起始位置, 使得当该进程重新执行时可以对所有等待条件重新进行评估

Simultaneous V-operation

$SV(S1,d1;...;Sn,dn)$

{

for($i=1; i \leq n; i++$)

$S_i = S_i + d_i;$

将 S_i 队列上的进程**PCB**取出，连到就绪队列中；

}

吸烟者问题-Solution

Shared t, w, m : semaphore; (0,0,0)

s : semaphore; (1)

Process X

loop

$P(s);$

$SV(t, 1; m, 1);$

endloop

Process Y

loop

$P(s);$

$SV(m, 1; w, 1);$

endloop

Process Z

loop

$P(s);$

$SV(w, 1; t, 1);$

endloop

吸烟者问题-Solution (Cont.)

Process A

loop

$SP(m,1,1;$
 $w,1,1);$

smoke;

$V(s);$

endloop

Process B

loop

$SP(w,1,1;$
 $t,1,1);$

smoke;

$V(s);$

endloop

Process C

loop

$SP(t,1,1;$
 $m,1,1);$

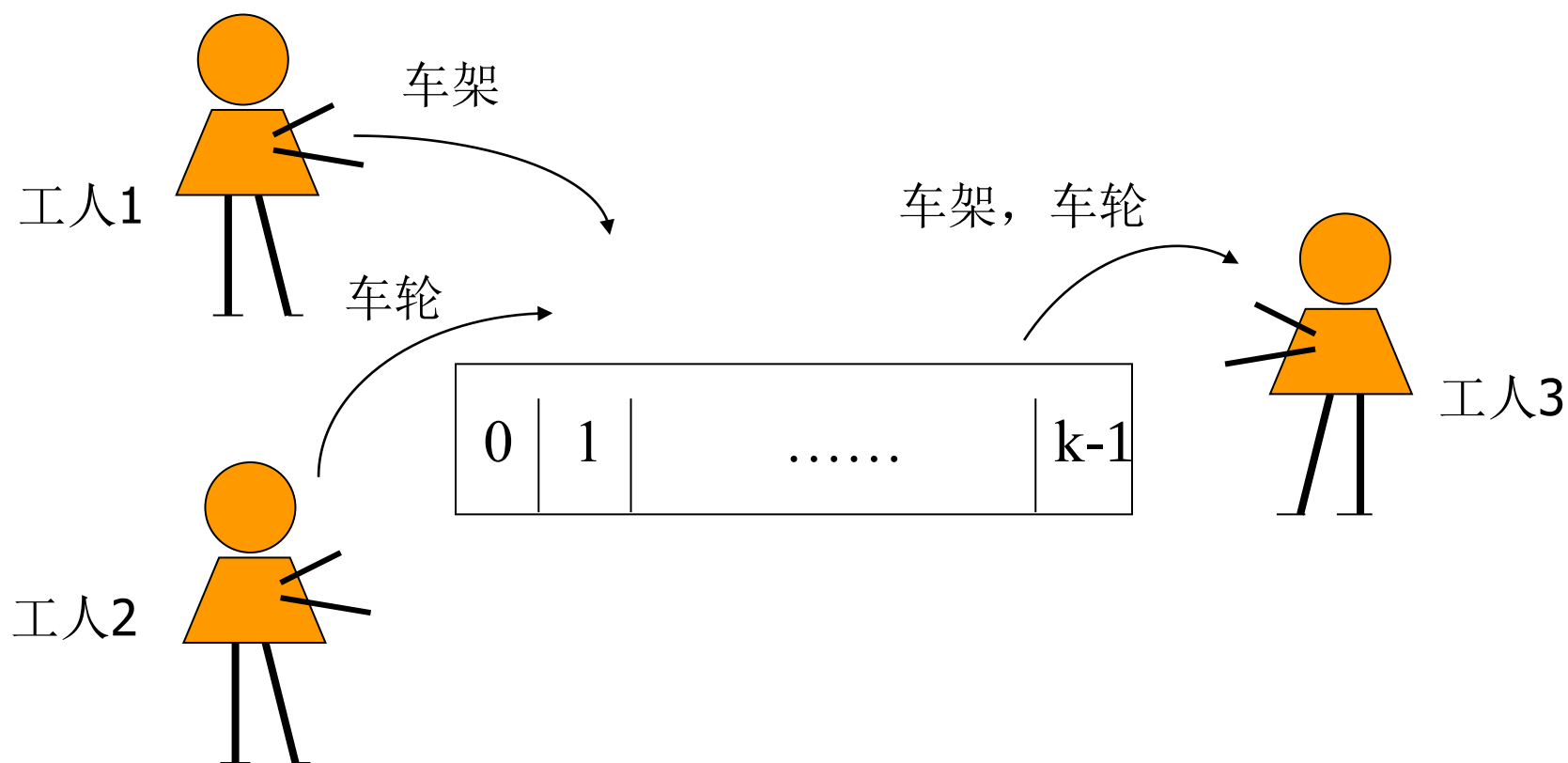
smoke;

$V(s);$

endloop

例6. 生产线问题

自行车生产线上有一只箱子，其中有 k 个位置，每个位置可以存放一个车架或一个车轮，有3个工人，活动如下：



例6. 生产线问题 (Cont.)

工人1活动:

Do{

加工一个车架

车架放入箱中

}while(1)

工人2活动:

Do{

加工一个车轮

车轮放入箱中

}while(1)

工人3活动:

Do{

箱中取一车架

箱中取二车轮

组装成一台车

}while(1)

semaphore empty; (空位, k)

semaphore frame; (车架, 0)

Semaphore wheel; (车轮, 0)

例6. 生产线问题 (Cont.)

工人1活动:

```
Do{  
    加工一个车架  
    P(empty)  
    车架放入箱中  
    V(frame)  
}while(1)
```

工人2活动:

```
Do{  
    加工一个车轮  
    P(empty)  
    车轮放入箱中  
    V(wheel)  
}while(1)
```

工人3活动:

```
Do{  
    P(frame)  
    箱中取一车架  
    V(empty)  
    P(wheel)  
    P(wheel)  
    箱中取二车轮  
    V(empty)  
    V(empty)  
    组装成一台车  
}while(1)
```

死锁情况: K个车轮, k个或k-1个车架

死锁防止: 车架不超过k-2个 (S1)

车轮不超过k-1个 (S2)

例6. 生产线问题 (Cont.)

工人1活动:

```
Do{  
    加工一个车架  
    P(S1)  
    P(empty)  
    车架放入箱中  
    V(frame)  
}while(1)
```

工人2活动:

```
Do{  
    加工一个车轮  
    P(S2)  
    P(empty)  
    车轮放入箱中  
    V(wheel)  
}while(1)
```

工人3活动:

```
Do{  
    P(frame)  
    箱中取一车架  
    V(empty)  
    V(S1)  
    P(wheel)  
    P(wheel)  
    箱中取二车轮  
    V(empty)  
    V(empty)  
    V(S2)  
    V(S2)  
    组装成一台车  
}while(1)
```

semaphore S1; (初值k-2, 车架)

semaphore S2; (初值k-1, 车轮)

例7. 哲学家就餐问题

Dining Philosophers Problem

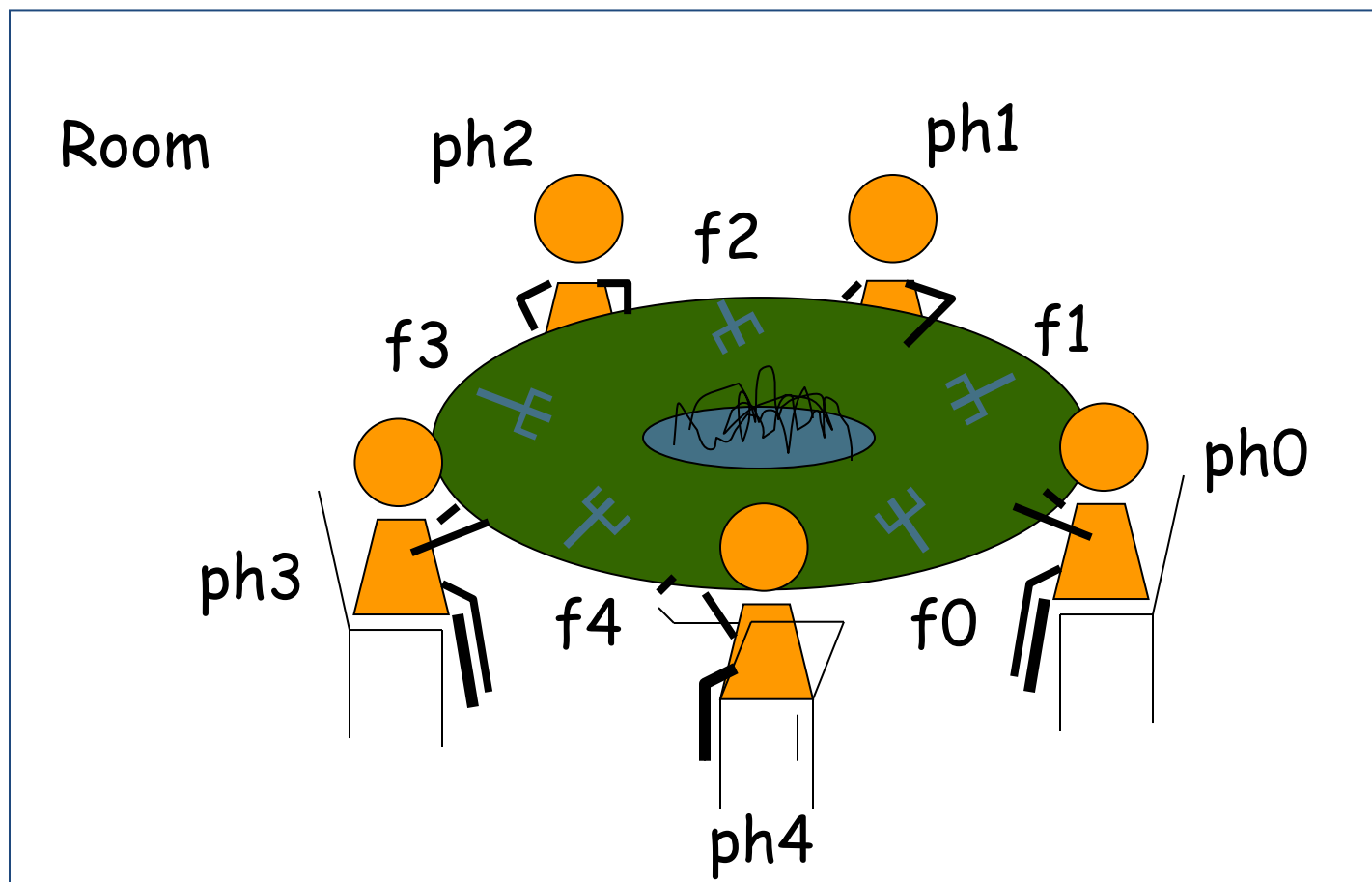
Proposed and solved by
E.W. Dijkstra, in 1965

例7. 哲学家就餐问题(Cont.)

■ 问题描述

- 有一个房间，其中有5位哲学家，他们的生活就是思考和进食。房间中摆着一张圆桌，就餐的桌子，圆桌的中间放着一个盘子，盘子中装有通心粉，这是哲学家进食时所吃的食物。这里假定盘子中的食物是无限多的，哲学家永远吃不完
- 有5个哲学家，哲学家在就餐时坐在自己的座位上(位置不能乱串)。当哲学家思考时，他可以离开自己的座位在房间内走动，但是当他想要进食时，则必须做到自己的座位上。在哲学家就餐时，需要一个工具，这个工具就是叉子
- 在圆桌上，每2个哲学家之间摆放一把叉子。当哲学家进食时需要使用其左边和右边的2把叉子
- 叉子不是同种组合资源，因为涉及到放的位置不同

例7. 哲学家就餐问题(Cont.)



例7. 哲学家就餐问题(Cont.)

哲学家活动:

```
Do{  
    思考  
    进食  
}While(1)
```

进食: 需要“叉子”

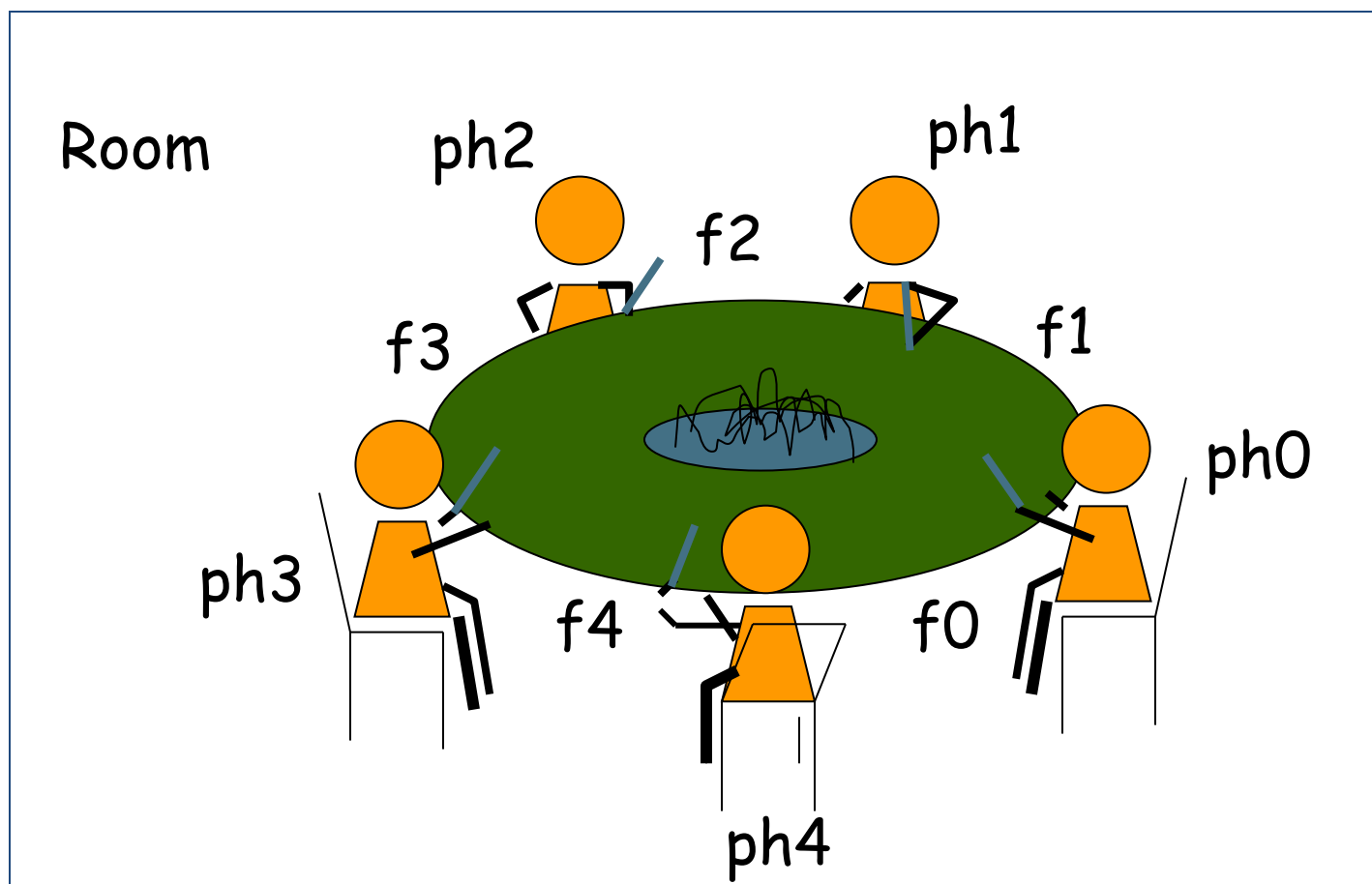
叉子: 不同种组合资源

哲学家活动 (包含资源活动)

```
Do{  
    思考  
    取左叉, 取右叉  
    进食  
    放左叉, 放右叉  
}While(1)
```

例7. 哲学家就餐问题(Cont.)

死锁情况：每位哲学家拿到左叉，等待右叉



死锁预防策略

死锁解决方法：同时申请左、右两把叉子(预先分配)。

哲学家状态描述(增加了hungry状态)：仅在饥饿的时候才申请叉子，且同时申请左、右两把叉子

State: Array[0..4]Of (thinking, hungry, eating);

Self: Array[0..4]Of semaphore; (initial value is 0);

就餐条件测试

测试I号哲学家是否具备进食条件：

```
Procedure test(I:0..4)
```

```
    If (state[I]=hungry) and
```

```
        (state[(I-1)mod 5] <> eating) and
```

```
        (state[(I+1)mod 5] <> eating)
```

```
    Then state[I]:=eating; V(self[I])
```

Remark:

如果i号哲学家处于饥饿状态，且左右两边的哲学家都不处于进食状态，那么将i号哲学家状态改为进食状态，同时唤醒i号哲学家。

未考虑互斥问题

I号哲学家活动:

1. Repeat
2. 思考
3. $state[I] := hungry;$
4. $test(I);$
5. $P(self[I]);$
6. 取左叉, 取右叉;
7. 进食
8. 放左叉, 放右叉;
9. $state[I] := thinking;$
10. $test((I-1) \bmod 5);$
11. $test((I+1) \bmod 5)$
12. Until false;

共享变量: $state;$

临界区域: 3-4, 9-11

Var mutex: semaphore; (1)

考虑互斥问题 (Cont.)

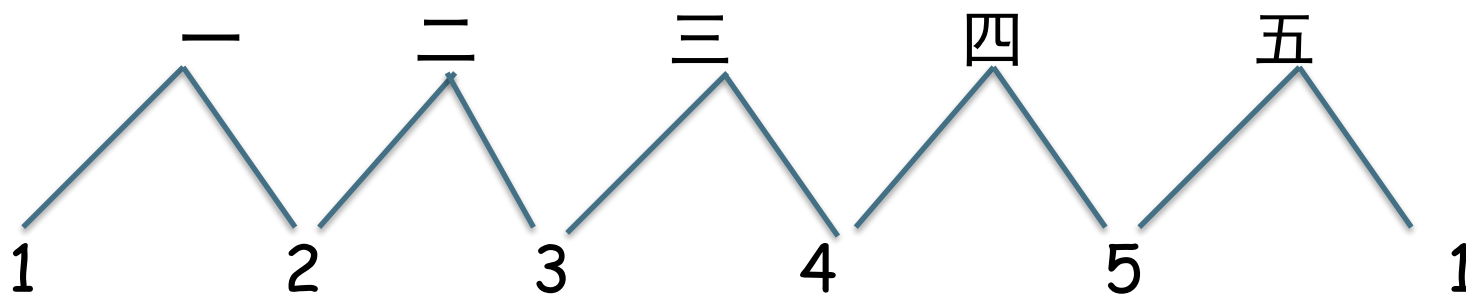
I号哲学家活动:

1. Repeat
2. 思考
3. P(mutex);
4. state[I]:=hungry;
5. test(I);
6. V(mutex);
7. P(self[I]);
8. 取左叉, 取右叉;
9. 进食
10. 放左叉, 放右叉;
11. P(mutex);
12. state[I]:=thinking;
13. test((I-1) mod 5);
14. test((I+1) mod 5);
15. V(mutex)
16. Until false;

问题

饿死情况：

某哲学家左右邻居至少有一个处于eating状态。



程序

```
Program dining_philosophers
var state:array[0..4]of (thinking,hungry,eating);
    self:array[0..4]of semaphore;
    mutex:semaphore;
procedure test(I:0..4)
begin
    if (state[I]=hungry)and
        (state[(I-1)mod 5]<>eating)
        and(state[(I+1)mod 5]<>eating) then
        begin state[I]:=eating; V(self[I]) end
end;
```

程序(Cont.)

```
Procedure philosopher(I:0..4)
begin
  cycle
    {thinking}
    P(mutex);
    state[I]:=hungry;
    test(I);
    V(mutex);
    P(self[I]);
```

程序(Cont.)

```
pick_up_fork(I);pick_up_fork((I+1)mod 5);  
{Eating};  
put_down_fork(I);put_down_fork((I+1)mod 5);  
P(mutex);  
state[I]:=thinking;  
test((I-1)mod 5);  
test((I+1)mod 5);  
V(mutex);  
end  
end
```

程序(Cont.)

```
begin
  for I:=0 to 4 do
    begin state[I]:=thinking; self[I].value:=0 end;
  mutex.value:=1;
  cobegin
    ph0: philosopher(0);
    .....
    Ph4: philosopher(4);
  coend
end.
```



例7. 资源调度问题

- 三组进程A、B、C，互斥使用某资源R，使用前申请，使用后释放。资源分配原则：

- 当只有一组申请进程时，该组申请进程依次获得R
- 当有两组申请进程时，组间交替，组内依次
- 当有三组申请进程时，组间轮流，组内依次

用信号量和PV操作分别给出各组进程的申请活动程序段、释放活动程序段

- 变量定义

- `int free; (init 1)` //设备状态标识，为1表示设备空闲
- `semaphore mutex; (init 1)` //互斥
- `semaphore qa, qb, qc; (init 0)` //各组等待队列
- `int counta, countb, countc; (init 0)` //等待队列长度

例7. 资源调度问题 (Cont.)

A组申请:

```
P(mutex)
if (free==1) {
    free=0;
    得到资源
    V(mutex);
}else{
    counta++;
    V(mutex);
    P(qa);
}
```

A组释放:

```
P(mutex)
if (countb>0) {
    countb--;
    V(qb);
}else if(countc>0) {
    countc--;
    V(qc);
}else if(counta>0) {
    counta--;
    V(qa);
}else free=1;
V(mutex)
```

仿A组进程活动可以给出B组和C组进程活动

例8. 系统与网络打印机问题

- 设有网络打印机和系统打印机各一台。网络打印机由网络用户使用；系统打印机一般由系统用户使用，但当其空闲时也可由网络用户使用。试用信号量与PV 操作实现对两台打印机的管理
- 变量定义
 - `int sys_free, net_free; (1, 1)`
 - `semaphore sys_q, (0) //系统用户等待`
 - `semaphore net_q; (0) //网络用户等待`
 - `int sys_wait; (0) //系统用户等待队列长度`
 - `int net_wait; (0) //网络用户等待队列长度`
 - `semaphore mutex; (1)`

例8. 系统与网络打印机问题（Cont.）

```

sys_apply //系统用户申请
{
    P(mutex);
L1: If(sys_free){
    sys_free=0;
    V(mutex);
}else{
    sys_wait++;
    V(mutex);
    P(sys_q);//系统用户等待
    goto L1;
}
}

```

```

net_apply //网络用户申请
{
    P(mutex);
L2: if(net_free) {//网络打印机空闲
    net_free=0;
    V(mutex);
    return(net);
}else if(sys_free){//系统打印机空闲
    sys_free=0;
    V(mutex);
    return(sys);
}else{
    net_wait++;
    V(mutex);
    P(net_q);//网络用户等待
    goto L2;
}
}
}

```

例8. 系统与网络打印机问题（Cont.）

```
sys_return{ //归还系统打印机
    P(mutex);
    sys_free=1;
    if(sys_wait>0) {
        sys_wait--;
        V(mutex);
        V(sys_q); //唤醒系统用户
    }else{
        if(net_wait>0){
            net_wait--;
            V(mutex);
            V(net_q);
        }else{
            V(mutex);
        }
    }
}
```

```
net_return{ //归还网络打印机
    P(mutex);
    net_free=1;
    if(net_wait>0){
        net_wait--;
        V(mutex);
        V(net_q);
    }else{
        V(mutex);
    }
}
```

例9. 2009研究生入学考试题

- 三个进程P1、P2、P3互斥使用一个包含N ($N>0$) 个单元的缓冲区。P1每次用produce () 生成一个正数并用put () 送入缓冲区某一空闲单元中；P2每次用getodd () 从该缓冲区中取出一个奇数并用countodd () 统计奇数个数；P3每次用geteven () 从该缓冲区中取出一个偶数并用counteven () 统计偶数个数。请用信号量机制实现三个进程的同步与互斥活动，并说明所定义信号量的含义
- 变量定义
 - semaphore empty; (init N)
 - semaphore odd, even; (init 0)
 - semaphore mutex; (init 1)

例9. 2009研究生入学考试题（Cont.）

```
main()
{
    process P1
    do{
        n=produce();
        P(empty);
        P(mutex);
        put();
        V(mutex);
        if(n%2==0)
            V(even);
        else
            V(odd);
    }while(true)

    process P2
    do{
        P(odd);
        P(mutex);
        getodd();
        V(mutex);
        V(empty);
        countodd();
    }while(true)

    process P3
    do{
        P(even);
        P(mutex);
        geteven();
        V(mutex);
        V(empty);
        counteven();
    }while(true)
}
```

例10. 寺庙问题

某寺庙有老和尚、小和尚若干。庙内有一水缸，由小和尚提水入缸，供老和尚饮用。水缸可容纳30桶水，每次入水、取水仅为一桶，不可同时进行。水取自同一井中，井口狭窄，每次只能容纳一个水桶取水。设有5只水桶，老和尚与小和尚共用。试用信号灯与PV操作给出老和尚和小和尚的活动。

■ 变量定义

- semaphore empty; (30) //水缸容量
- semaphore full; (0) //当前水量
- semaphore bucket; (5) //水桶数
- semaphore mutex_bigjar;
- semaphore mutex_well;

例10. 寺庙问题 (Cont.)

```
Young_monk ()
{
    do {
        P(empty);
        P(bucket);
        走到井边;
        P(mutex_well); //井口互斥
        井中取水;
        V(mutex_well);
        走到寺庙;
        P(mutex_bigjar); //水缸互斥
        水入缸中;
        V(mutex_bigjar);
        V(bucket);
        V(full); //水缸水量增加
    } while(1)
}
```

```
old_monk ()
{
    do {
        P(full); //从水缸取水
        P(bucket);
        P(mutex_bigjar);
        缸中取水;
        V(mutex_bigjar);
        喝水;
        V(bucket);
        V(empty); //水缸容量增加
    }
}
```

例10. 寺庙问题 (Cont.)

- P操作的顺序不当会引起死锁
- 上述解法是无死锁解法
- 若在上述解法中存在多个小和尚、老和尚时，P操作的不当顺序会引起死锁。假设先P(bucket)然后再P(empty)就会出现死锁
- 死锁一般都是在极限状态的时候判断

进程的同步与互斥

使用信号量和PV操作的规则

- ① 分清哪些是互斥问题（互斥访问临界资源的），哪些是同步问题（具有前后执行顺序要求的）
- ② 对于互斥问题要设置互斥信号量，不管有互斥关系的进程有几个或几类，互斥信号量的个数只与临界资源的种类有关。通常，有几类临界资源就设置几个互斥信号量，且初值为1，代表临界资源可用
- ③ 对于同步问题要设置同步信号量，通常同步信号量的个数与参与同步的进程种类有关，即同步关系涉及几类进程，就有几个同步信号量。同步信号量表示该进程是否可以开始或该进程是否已经结束
- ④ 在每个进程中用于实现互斥的PV操作必须成对出现；用于实现同步的PV操作也必须成对出现，但是，它们分别出现在不同的进程中；在某个进程中如果同时存在互斥与同步的P操作，则其顺序不能颠倒。必须先执行对同步信号量的P操作，再执行对互斥信号量的P操作。但是，V操作的顺序没有严格要求

调换同步互斥顺序发生错误

生产者活动:

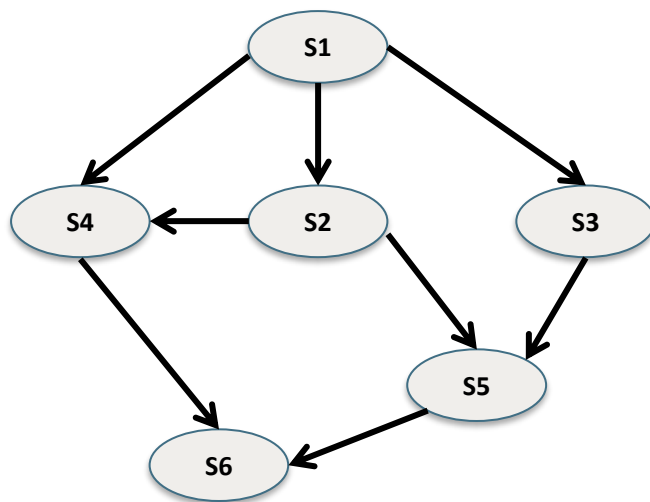
```
Do{  
    加工一件物品  
    P(mutex)  
    P(S1)  
    物品放入箱中  
    V(mutex)  
    V(S2)  
}While(1)
```

消费者活动:

```
Do{  
    P(S2)  
    P(mutex)  
    箱中取一物品  
    V(mutex)  
    V(S1)  
    消耗这件物品  
}While(1)
```

课堂练习

- 使用信号量和PV操作实现进程间的并发关系



4.3.4 条件临界区

Conditional Critical Region

- 背景
 - PV操作低级, 容易用错
- 条件临界区形式
 - `region r when b do s`
- 执行*s*条件
 - 没有其它进程处于与*r*相关的条件临界区中
 - 进入*s*时*b*为true

4.3.4 条件临界区

Conditional Critical Region

■ **Condition critical regions work as follows:**

1. A process wants to enter a region for S must obtain the mutex lock.
If it cannot, then it is queued in *main queue*.
2. Once the lock is obtained, the boolean expression B is tested.
If B evaluates to *TRUE* then the process proceeds, otherwise it releases the lock, and is queued in *event queue*.
When it next gets the lock it must retest B.
3. Each shared variable has two queues associated with it,
eg. the *main queue* and the *event queue*.

条件临界区

- **实现互斥**
 - `region v when true do s`
- **CCR与PV操作的等价性(证明从略)**
 - 用CCR可以实现PV操作
 - 用PV操作可以实现CCR
- **P.B.Hansen设计的Edison语言提供了CCR同步机制.**
- **CCR的实现效率比较低**

条件临界区的实现效率问题

- `region  $r$  when  $b$  do  $s$`
- 条件临界区的实现效率是比较低的
 - 这主要是条件表达式 b 的计算, 由于 b 中可能包含进程局部信息, 每个欲进入条件临界区的进程必须自己计算 b 的值(而不能由调度程序统一计算).
 - 因为进入条件临界区必须同时满足互斥和 b 为真两个条件, 任何一个条件不满足都将使进程等待, 这样在条件临界区的入口处会形成一个等待队列.
 - 当处于条件临界区内的进程执行完 s 后, 全局变量将发生变化, 可能使某些条件临界区语句的 b 变为真, 这时需要唤醒等待进程并由被唤醒的进程重新计算其 b 的值, 而计算结果有可能仍为假, 进程将重新回到等待状态. 本质上来说, 这也是一种忙式等待.

例4-6 简单批处理

作业由读入、执行、打印结果三个相对独立的部分组成，显然三者需要同步，用条件临界区实现的程序如下：

```
PROGRAM batch;  
  TYPE buffer=RECORD  
    slots: ARRAY[0..n-1]of T;  
    head,tail: 0..n-1 initial (0,0);  
    size: 0..n initial (0);  
  END;  
  VAR inbuff: buffer(cardimage);  
      outbuff: buffer(lineimage);  
  RESOURCE ib: inbuff; ob: outbuff;
```

例4-6 简单批处理

COBEGIN

PROCESS reader;

VAR card: cardimage

LOOP

Read card from cardreader;

REGION ib WHEN inbuff.size<n DO

inbuff.slots[inbuff.tail]:=card;

inbuff.size := inbuff.size+1;

inbuff.tail := (inbuff.tail+1) MOD n;

ENDDO

ENDLOOP

ENDPROCESS


```
PROCESS executer;  
  VAR card:cardimage;  
      line:lineimage;  
LOOP  
  REGION ib WHEN inbuff.size>0 DO  
    card := inbuff.slots[inbuff.head];  
    inbuff.size := inbuff.size-1;  
    inbuff.head := (inbuff.head+1)MOD n  
  ENDDO  
  Process card and generate line  
  REGION ob WHEN outbuff.size<n DO  
    outbuff.slots[outbuff.tail] := line;  
    outbuff.size := outbuff.size+1;  
    outbuff.tail := (outbuff.tail+1)MOD n;  
  ENDDO  
ENDLOOP  
ENDPROCESS
```

例4-6 简单批处理

```
PROCESS printer;  
  VAR line:lineimage;  
  LOOP  
    REGION ob WHEN outbuff.size>0 DO  
      line := outbuff.slots[outbuff.head];  
      outbuff.size := outbuff.size-1;  
      outbuff.head := (outbuff.head+1)MOD n;  
    ENDDO  
    Print line on lineprinter;  
  ENDLOOP  
ENDPROCESS  
COEND.
```

4.3.5 管程（Monitor）

- PV操作

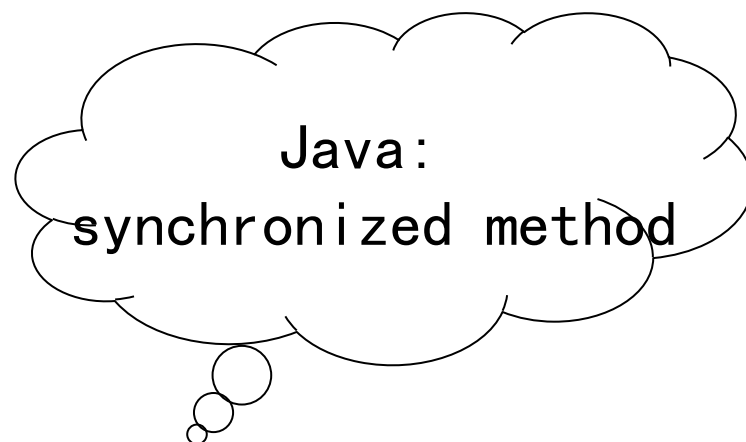
- 分散式同步机制
- 缺点
 - 可读性差
 - 正确性不易保证
 - 不易修改和维护
- 优点
 - 高效，灵活

- 管程

- 集中式同步工具
- 优点
 - 可读性好
 - 正确性易于保证
 - 易于修改
- 缺点
 - 不甚灵活，效率略低

4.3.5.1 管程的提出

- 70年代初, By
 - E. W. Dijkstra, C. A. R. Hoare, P. B. Hansen.
- 背景: Structured programming
- 基于管程的语言
 - Concurrent Pascal (Hansen)
 - Modula (With)
 - Mesa
 - Mod*
 - Concurrent Euclid
 - XCY



4.3.5.1 管程的提出(Cont.)

- 构造操作系统的PCM方法
 - P: process
 - C: class
 - M: monitor
- Example system
 - solo

4.3.5.1 管程的提出(Cont.)

- 基本思想：将共享变量及其所有操作集中在一个模块中。
- 目的：
 - 把分散在各进程中的临界区集中起来进行管理；
 - 防止进程有意或无意的违法同步操作；
 - 便于用高级语言来书写程序，也便于程序正确性验证；
 - 共享性、安全性、互斥性。

4.3.5.2 管程形式

Type monitor_name=MONITOR (形参表)

共享变量说明;外部不可见

define 本管程内定义, 本管程外使用的子程序名表;

use 本管程外定义, 本管程内使用的子程序名表;

Procedure 过程名 (形参表)

局部变量说明

Begin 语句序列 **End**;

.....

4.3.5.2 管程形式(Cont.)

Function 函数名（形参表）：返回值类型；

局部变量说明

Begin 语句序列 **End**;

.....

Begin 共享变量初始化语句序列 **End**;

从语言角度看，管程特点：

- (1) modularized
- (2) abstract data type
- (3) information hiding

4.3.5.3 管程语义(Cont.)

■ Remarks

- 管程的共享变量在管程外部不可见，外部只能通过调用管程中的子程序访问共享变量
- 为保证对共享变量操作的数据完整性，规定管程互斥进入，**每次仅允许一个进程在管程内执行某个内部过程**
- 进程互斥地通过调用内部过程进入管程，当某进程在管程内执行时，其它想进入管程的进程必须等待
- 管程是一种特殊的数据类型，它将共享变量以及对共享变量的操作封装到一起，是一种集中式的同步机制

4.3.5.3 管程语义(Cont.)

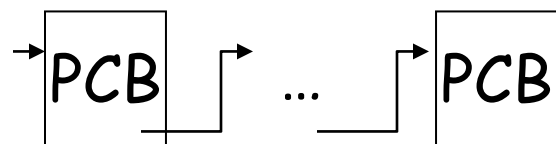
- 管程中的等待/唤醒机制，等待时释放管程的互斥权，唤醒时(P唤醒Q)
 - P紧急等待，Q继续，直到Q退出或等待 (Hoare)
 - Signal and urgent wait
 - 进程切换比较频繁，实现效率较低
 - Q等待，P继续，直到P退出或等待 (Java)
 - Signal and continue
 - 被唤醒进程需要重新检查等待条件, 可能再次等待
 - 唤醒是管程中可执行的最后一个操作 (Hansen)
 - Signal and leave

4.3.5.3 管程语义(Cont.)

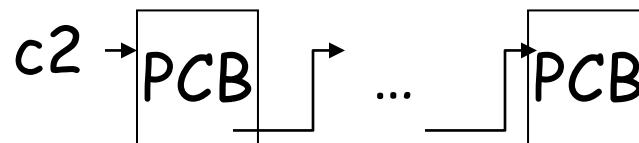
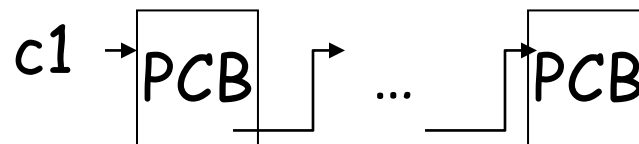
- 入口等待队列
 - 每个管程变量一个，用于排队进入
- 紧急等待队列
 - 每个管程变量一个，用于唤醒等待
- 条件等待队列
 - `var c: condition;` 可根据需要定义多个，用于设置等待条件

4.3.5.3 管程语义(Cont.)

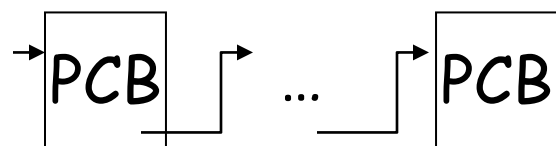
管程用于管理资源，具有3种等待队列



入口队列



条件等待队列

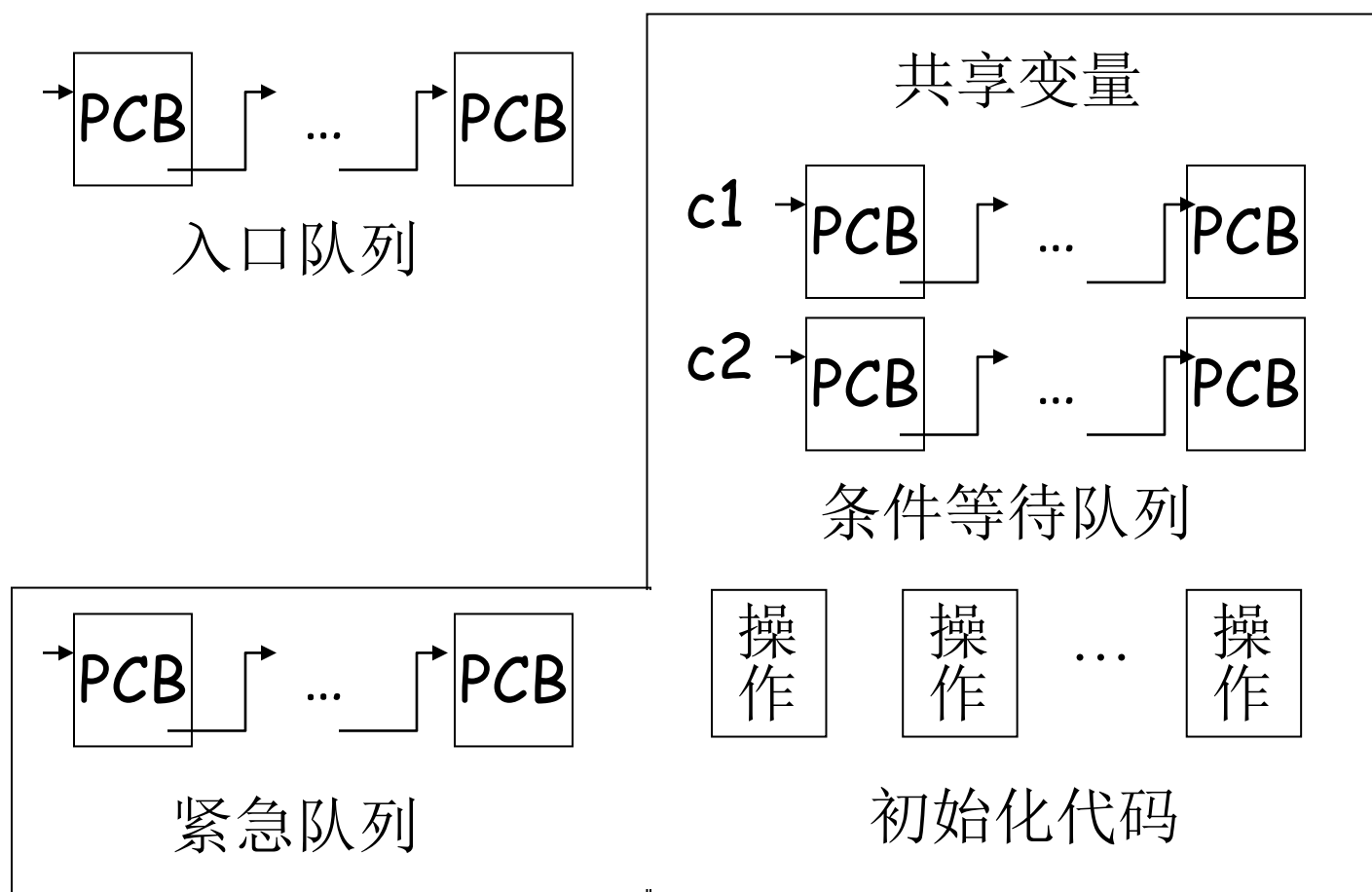


紧急队列

Monitor

4.3.5.3 管程语义(Cont.)

管程成分



4.3.5.3 管程语义(Cont.)

- 进入管程
 - 申请管程互斥权
- 离开管程
 - 如紧急等待队列非空，唤醒第一个等待者
 - 否则开放管程

条件变量操作

- `Var c:condition; // 指针，指向PCB队列头部`
- `wait(c)`
 - 执行此操作的进程(线程)进入c链尾
 - 如紧急队列非空，唤醒第一个等待者，否则唤醒入口等待队列中的一个进程，即释放管程互斥权
- `signal(c)`
 - 如c链空，相当空操作
 - 否则唤醒第一个，执行此操作的进程(线程) 有3种方式，分为Hoare管程、Hansen管程和Java管程

条件变量操作（Cont.）

- Hoare管程
 - 指从条件队列中被唤醒的进程继续执行，执行唤醒操作的进程进入到紧急等待队列。当它从紧急队列被唤醒后，继续执行管程内的其它代码
- Hansen管程
 - 被唤醒的进程继续执行，执行唤醒操作的进程离开管程，Signal是管程中的最后一条指令
- Java管程
 - 执行唤醒操作的进程继续执行，被唤醒的进程等待，当执行唤醒操作的进程执行完毕后，被唤醒的进程开始执行

例. 在Hoare管程中，设某管程当前入口等待队列EQ中有进程P0、紧急等待队列UQ中有进程P1、条件变量C的等待队列CQ中有进程P2，进程P3拥有该管程的互斥权。当依次：进程P4要申请该管程互斥权、P3执行signal(C)后，该管程各队列中的进程和运行进程是

- A. EQ中有P0，UQ中有P4，CQ中有P2、P3；P1运行
- B. EQ中有P0、P4，UQ中有P1，CQ中有P3；P2运行
- C. EQ中有P0、P4，UQ中有P2、P3，CQ为空；P1运行
- D. EQ中有P0、P4，UQ中有P1、P3，CQ为空；P2运行

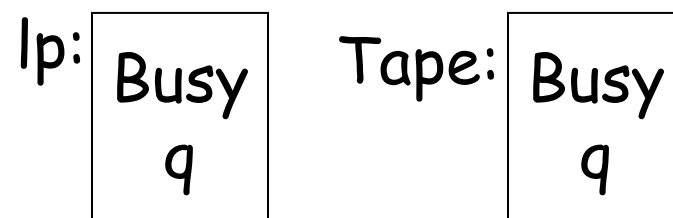
答案 D

例. 单一资源管理

- Type single_resource=MONITOR;
- Var busy:boolean;
- q:condition;
- Define require,release;
- Procedure require;
- Begin
- If busy Then
- wait(q);
- busy:=true
- End;

例. 单一资源管理

- Procedure release;
- Begin
- busy:=false;
- signal(q);
- End;
- Begin
- busy:=false
- End;
- Var lp,tape:single_resource;



申请lp: lp.require

释放lp: lp.release

申请tape: tape.require;

释放tape: tape.release

4.3.5.4 管程的使用

- 生产者/消费者问题
- 读者/写者问题（写优先）
- 哲学家就餐问题（another approach）
- Sleepy barber' s problem
- Disk head scheduler
- Single resource management

例1.生产消费问题

```
Type producer_consumer = MONITOR
Var B:Array[0..n-1]Of integer;
    count, in, out: integer;
    pq, cq:condition;
define put_in, get_out;
Procedure put_in(item:integer);
    Begin
        If(count==n)Then
            wait(pq);
        B[in]:=item;
        in:=(in+1)%n;
        count++;
        signal(cq);
    End;
```

例1.生产消费问题 (Cont.)

```
Procedure get_out(Var item:integer)
```

```
  Begin
```

```
    If(count==0)Then
```

```
      wait(cq);
```

```
      item:=B[out];
```

```
      out:=(out+1)%n;
```

```
      count--;
```

```
      signal(pq);
```

```
  End;
```

```
Begin
```

```
  in:=0; out:=0; count:=0;
```

```
End;
```

例1.生产消费问题 (Cont.)

Var **pc**: producer_consumer;

生产者:

```
{produce an item;  
pc.put_in(item);  
}while(1)
```

消费者

```
{pc.get_out(item);  
consume the item;  
}while(1)
```

例2. 读者/写者问题

写者优先

```
Type readers_writers = Monitor;  
  Var rq,wq: condition;  
    reading_count,write_count:integer;  
  Define start_read, finish_read,  
    start_write, finish_write;
```


Procedure **start_read**;

Begin

If write_count > 0 Then

wait(rq);

reading_count++;

signal(rq);

End;

Procedure **start_write**

Begin

write_count++;

If (write_count > 1) or

(reading_count > 0)

Then

wait(wq)

End;

Procedure **finish_read**;

Begin

reading_count--;

If reading_count = 0

Then signal(wq)

End;

Procedure **finish_write**;

Begin

write_count--;

If write_count > 0

Then signal(wq)

Else signal(rq)

End;

例2. 读者/写者问题 (Cont.)

Begin

 reading_count:=0;

 write_count:=0;

End;

Var rw:readers_writers;

读者:

rw.start_read;

{reading}

rw.finish_read;

rw:

reading_count
write_count
rq
wq

写者:

rw.start_write;

{writing}

rw.finish_write;

例3. 哲学家就餐问题

PV操作解决哲学家就餐问题

哲学家活动:

Repeat

 思考

 L: 取左叉

 If 右叉空闲 Then

 Begin 取右叉, 进食, 放左叉, 放右叉 end;

 Else Begin 放左叉, goto L end;

Until false

例3. 哲学家就餐问题（Cont.）

```
Type dining-philosophers=MONITOR
  Var fork:Array[0..4]Of(free,used);
      q:Array[0..4]Of condition;
  define pick_up, try_pick_up, put_down;
  Procedure pick_up(I:0..4);
    Begin
      If fork[I]=used Then
        wait(q[I]);
        fork[I]:=used
      End;
```

例3. 哲学家就餐问题（Cont.）

```
Function try_pick_up(I:0..4):boolean;
```

```
  Begin
```

```
    If fork[I]=used Then
```

```
      try_pick_up:=false
```

```
    Else Begin
```

```
      fork[I]:=used;
```

```
      try_pick_up:=true
```

```
    End
```

```
End;
```

例3. 哲学家就餐问题 (Cont.)

```
Procedure put_down(I:0..4);  
  Begin  
    fork[I]:=free;  
    signal(q[I])  
  End;  
Procedure init;  
  Var I:0..4;  
  Begin  
    For I:=0 to 4 Do fork[I]:=free;  
  End;  
Begin init End;
```

例3. 哲学家就餐问题 (Cont.)

Var forks:dining_philosophers;

I号哲学家活动:

Repeat

{thinking}

with_two_forks:=false;

Repeat

forks.pick_up(I);

If forks.try_pick_up((I+1)mod 5)Then

with_two_forks:=true

Else forks.put_down(I)

Until with_two_forks; //为true跳出循环

例3. 哲学家就餐问题 (Cont.)

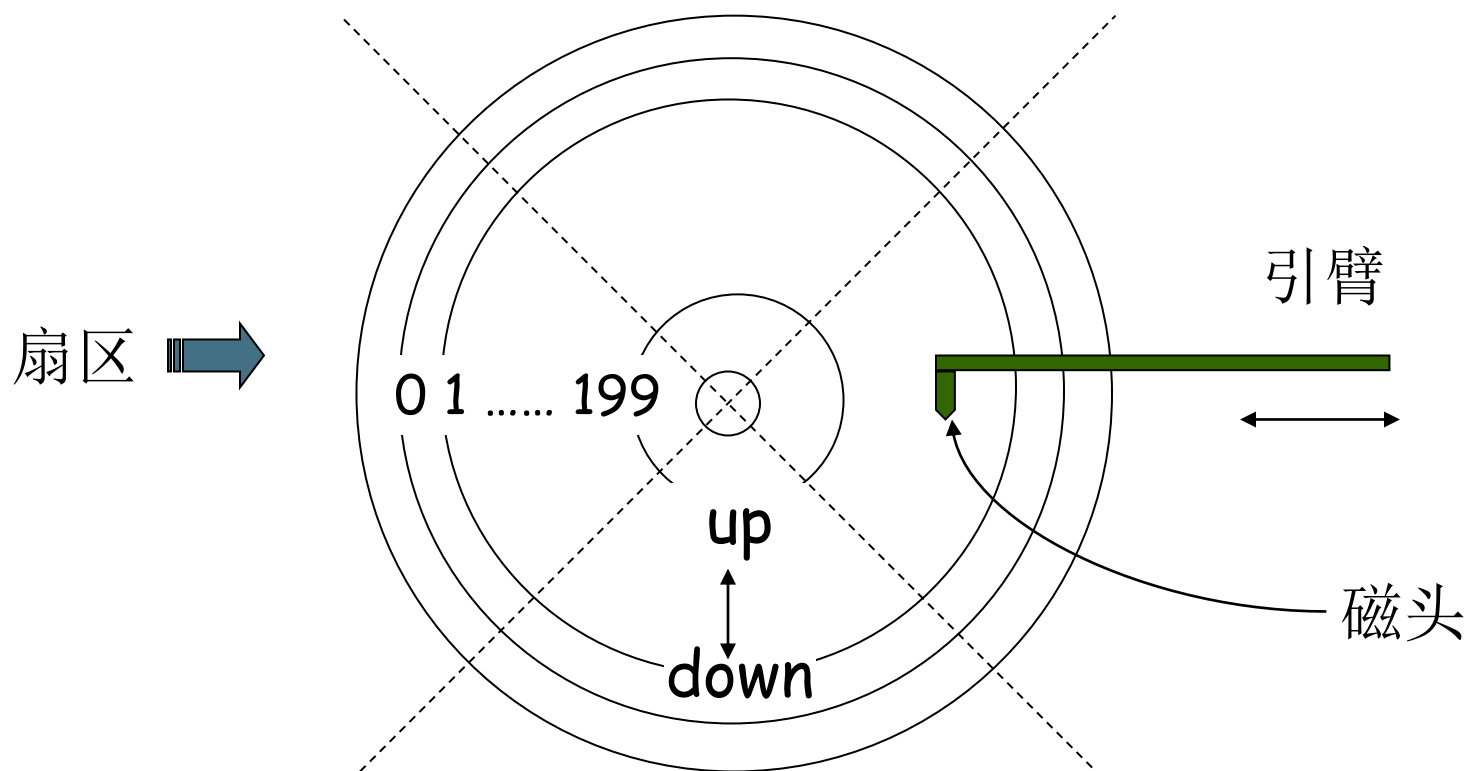
{Eating}

forks.put_down(I);

forks.put_down((I+1)mod 5)

Until false;

例4. 磁头引臂调度问题

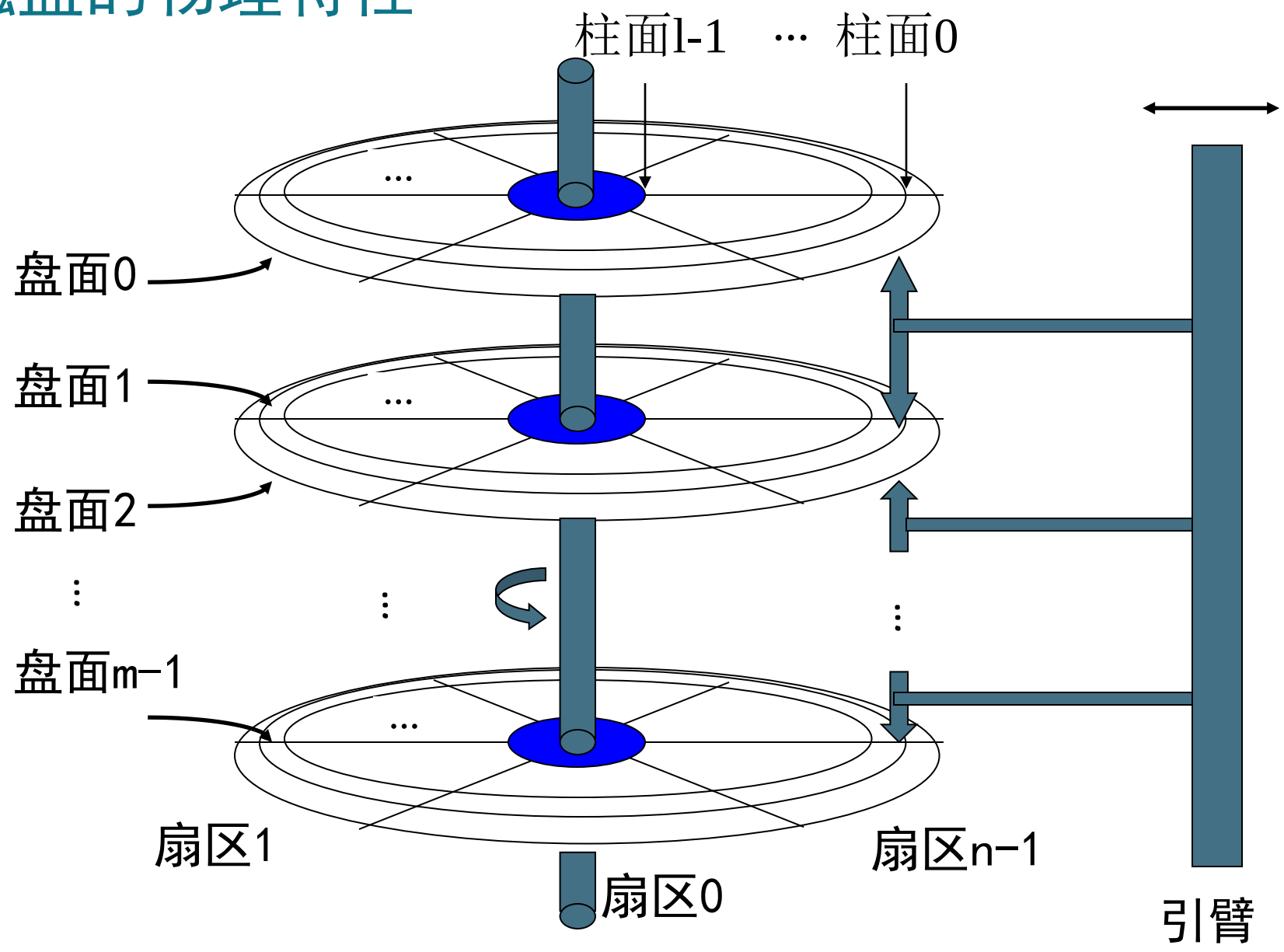


当有多个磁盘块的访问请求时，采用电梯算法进行调度

磁盘的物理特性



磁盘的物理特性



调度算法

- FCFS(first come first serve)
 - 公平
 - 效率低
- SSTF(shortest seek time first)
 - 效率高
 - 磁道歧视
- SCAN(elevator algorithm)
 - 效率较高, 较公平

调度算法

- 电梯算法思想（SCAN）
 - 当无访问请求时，磁头引臂停止不动
 - 当有访问请求时，磁头引臂按照电梯移动规律运动，并为路经柱面的访问请求服务
 - 起始时，磁头由最外柱面向内柱面移动，并为路经的请求服务。一旦内柱面没有访问请求，则改变移动方向(如果外柱面有请求)或者停止移动(外柱面也无请求)

Hansen管程实现电梯算法

外部过程：

申请：`require(dest:0..199)` //申请磁头引臂

释放：`release()` //释放磁头引臂

使用方式：

`require(dest)` ; //dest表示申请的磁道编号

{IO操作} -- (管程外操作)

`release()` ;

Hansen管程实现电梯算法 (Cont.)

```
Type diskhead=MONITOR
```

```
  Var busy:boolean;//磁头是否忙碌
```

```
    headpos:0..199;//磁头当前位置
```

```
    direction:(up,down);//磁头移动方向
```

```
    cylinder:Array[0..199]Of condition;
```

```
    count:Array[0..199]Of integer;
```

```
  Define require, release;
```

Hansen管程实现电梯算法 (Cont.)

```
Procedure require(dest:0..199);  
  Begin  
    If busy Then //磁头非空闲，请求进程等待  
      Begin  
        count[dest]:=count[dest]+1;  
        wait(cylinder[dest])  
      End;  
    busy:=true;  
    If dest<headpos Then  
      direction:=down  
    Else If dest>headpos Then  
      direction:=up;  
    headpos:=dest; //磁头指向请求磁道  
  End;
```


Hansen管程实现电梯算法 (Cont.)

```
Procedure release;//释放磁头引臂
Begin
    busy:=false;
    If direction=up Then
        Begin
            upscan;downdscan
        End
    Else
        Begin
            downdscan;upscan
        End
    End;
End;
```

Hansen管程实现电梯算法 (Cont.)

```
Procedure upscan; //从当前位置向上(内)扫描磁道
  Var I:0..200;
  Begin
    I:=headpos;
    //向内依次扫描每个磁道，当发现磁道上有等待进程时停止
    While (I<=199)and(count[I]=0) Do
      I:=I+1;
      If I<=199 Then
        Begin
          count[I]:=count[I]-1;
          signal(cylinder[I])
        End
      End
    End;
  End;
```

Hansen管程实现电梯算法 (Cont.)

```
Procedure downscan; // 从当前位置向下(外)扫描磁道
  Var I:-1..199;
  Begin
    I:=headpos;
    // 向外依次扫描每个磁道，当发现磁道上有等待进程时停止
    While (I>=0) and (count[I]=0) Do
      I:=I-1;
      If I>=0 Then
        Begin
          count[I]:=count[I]-1;
          signal(cylinder[I])
        End
      End
    End;
  End;
```

Hoare管程实现电梯算法(一)

```
Procedure upscan;  
  Var I:0..200;  
  Begin  
    I:=headpos;  
    flag:=false;  
    While (I<=199)and(count[I]=0) Do  
      I:=I+1;  
      If I<=199 Then  
        Begin  
          count[I]:=count[I]-1;  
          signal(cylinder[I])  
          flag=true ;  
        End  
      End  
    End;  
  End;
```

加上flag是为了判断是否执行过唤醒动作Signal

Hoare管程实现电梯算法(一)

```
Procedure downscan;  
  Var I:-1..199;  
  Begin  
    I:=headpos;  
    flag:=false;  
    While (I>=0)and(count[I]=0) Do  
      I:=I-1;  
      If I>=0 Then  
        Begin  
          count[I]:=count[I]-1;  
          signal(cylinder[I])  
          flag=true ;  
        End  
      End  
    End;  
  End;
```

Hoare管程实现电梯算法(一)

Procedure release;

Begin

busy:=false;

If direction=up Then

Begin

upscan;

if flag!=true then downscan

End

Else

Begin

downscan;

if flag!=true then upscan

End

End;

Hoare管程实现电梯算法(二)

```
Procedure upscan;  
  Var I:0..200;  
  Begin  
    I:=headpos;  
    While (I<=199)and(count[I]=0) Do  
      I:=I+1;  
      If I<=199 Then  
        Begin  
          count[I]:=count[I]-1;  
          signal(cylinder[I])  
        End  
      else downscan;  
    End;  
End;
```

Hoare管程实现电梯算法(二)

```
Procedure downscan;  
  Var I:-1..199;  
  Begin  
    I:=headpos;  
    While (I>=0)and(count[I]=0) Do  
      I:=I-1;  
    If I>=0 Then  
      Begin  
        count[I]:=count[I]-1;  
        signal(cylinder[I])  
      End  
    else upscan;  
  End;
```

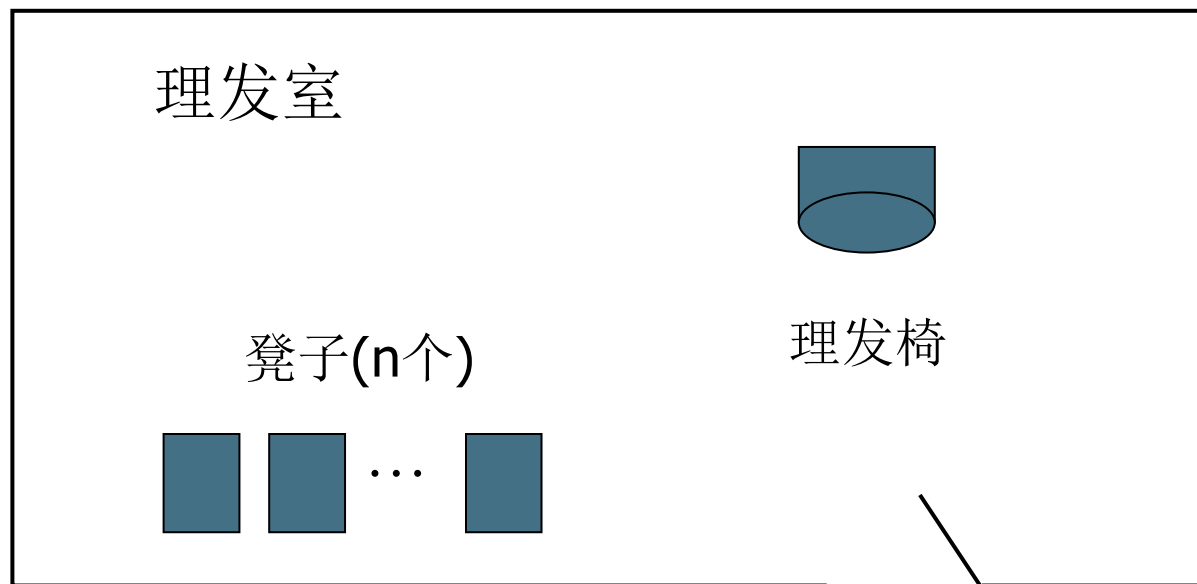

Hoare管程实现电梯算法(二)

```
Procedure release;  
  Begin  
    busy:=false;  
    If direction=up Then  
      Begin  
        upscan;  
      End  
    Else  
      Begin  
        downscan;  
      End  
    End;  
  End;
```

例6 嗜眠理发师问题 (Sleepy barber's problem)

- 一把理发椅，顾客理发时坐在理发椅上，由理发师为其理发。没有顾客时，理发师在理发椅上睡觉；
- N 个凳子，顾客等待理发用。凳上坐满顾客时，新进入的顾客退出理发室。

例6 嗜眠理发师问题 (Cont.)



例6 嗜眠理发师问题 (Cont.)

```
TYPE sleepybarber=MONITOR;  
  VAR chair:condition;  
      stool:condition;  
      count:0..n+1;  
  Define sleep, enter, leave;  
  Procedure sleep;  
    Begin  
      If count=0 Then  
        wait(chair)  
    End;
```

例6 嗜眠理发师问题（Cont.）

```
Procedure enter(var full:Boolean);  
Begin  
  If count=n+1 Then  
    full:=true;  
  Else Begin  
    full:= false;  
    count:=count+1;  
    If count =1 Then  
      signal(chair);  
    Else wait(stool);  
  End;  
End;
```

例6 嗜眠理发师问题（Cont.）

Procedure leave;

Begin

count:=count-1;

If count<> 0 Then

signal(stool)

End;

Begin count:=0; End

例6 嗜眠理发师问题（Cont.）

理发师活动:

Repeat

 sleepybarber.sleep;

 Cuthair;

Until false;

顾客活动:

Repeat

 sleepybarber.enter(full);

Until not full;

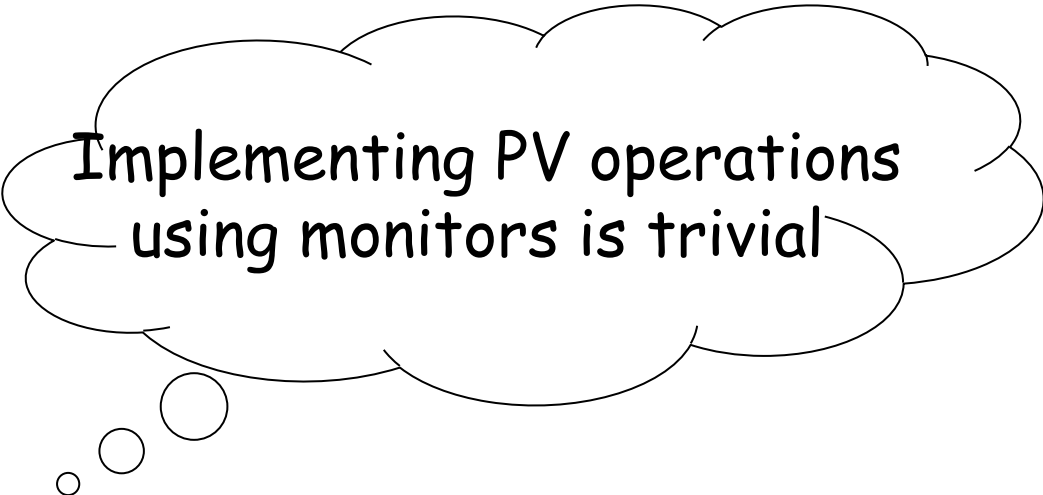
Cuthair

Sleepybarber.leave

4.3.5.5 管程与PV操作的等价性

用PV操作可以构造管程

用管程可以构造PV操作



Implementing PV operations
using monitors is trivial

用管程构造PV操作

```
TYPE semaphore=MONITOR(init_value)
```

```
  VAR c:condition;//条件变量
```

```
      count: integer;//整形变量
```

```
DEFINE P,V;//定义2个外部函数
```

```
//用管程构造P操作
```

```
PROCEDURE P;
```

```
  BEGIN
```

```
    count:=count-1;
```

```
    IF count<0 THEN
```

```
      wait(c);
```

```
  END;
```

```
//用管程构造V操作
```

```
PROCEDURE V;
```

```
  BEGIN
```

```
    count:=count+1;
```

```
    IF count <=0 THEN
```

```
      signal(c);
```

```
  END;
```

```
//初始化
```

```
BEGIN
```

```
  count:=init_value;
```

```
END;
```

用管程构造PV操作(Cont.)

信号灯与PV操作	管程
VAR s: semaphore;	TYPE s=MONITOR;
s.value:=init_value;	Init s(init_value);
P(s);	s.P;
V(s);	s.V;

用PV操作构造管程

管程	信号灯与PV操作
一个管程实例	一个入口等待队列mutex: semaphore (1) 一个紧急等待队列urgent: semaphore (0) 一个紧急等待计数urgent_count: integer (0)
一个条件变量 c: condition;	一个信号量变量s: semaphore (0) 一个计数变量count: integer (0)
等待操作 wait(c);	count:=count+1; IF urgent_count>0 THEN BEGIN urgent_count:=urgent_count-1; V(urgent); ELSE V(mutex); P(s);

用PV操作构造管程

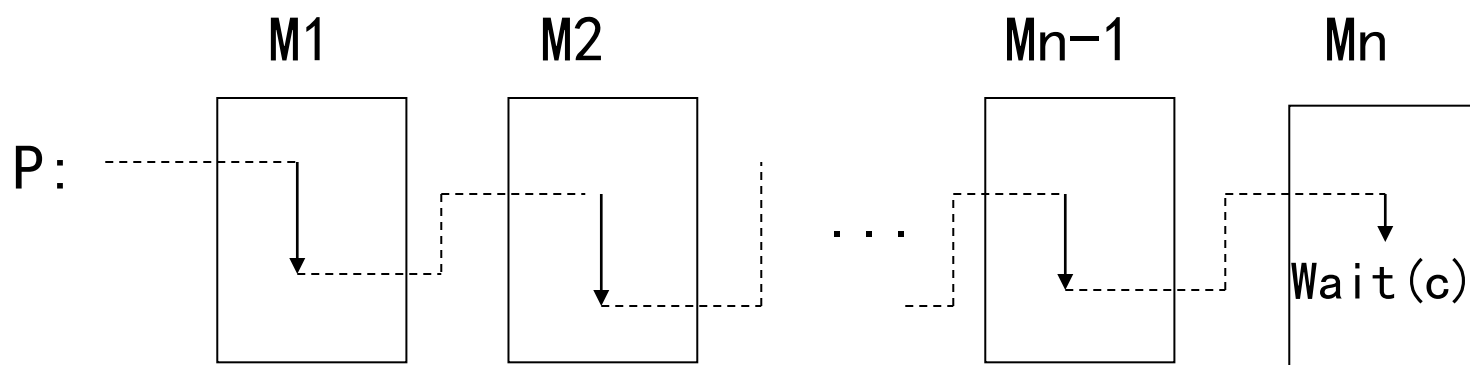
管程	信号灯与PV操作
唤醒操作 Signal (c)	<pre>IF count>0 THEN BEGIN count:=count-1; urgent_count:=urgent_count+1; V(c); P(urgent); END</pre>
进入管程	<pre>P(mutex);</pre>
离开管程	<pre>IF urgent_count>0 THEN BEGIN urgent_count:=urgent_count-1; V(urgent); END ELSE V(mutex);</pre>

思考

- 管程的Signal操作与信号量机制中的V操作是否相同？

4.3.5.6 管程的嵌套调用问题

- 管程嵌套：
 - 一个进程在执行过程中调用某一管程中定义的外部函数，该函数在执行过程中可能又调用另一个管程中的外部函数



4.3.5.6 管程的嵌套调用问题（Cont.）

问题：1. 是否允许嵌套

2. 若允许，如何处理嵌套调用时管程的互斥权？

方法：1. 禁止嵌套，简单易行，但是过于严格的限制；

2. 允许嵌套，等待时释放当前管程互斥权

这是容易保证程序正确性的处理策略，而且开销比较小，但并发性差；

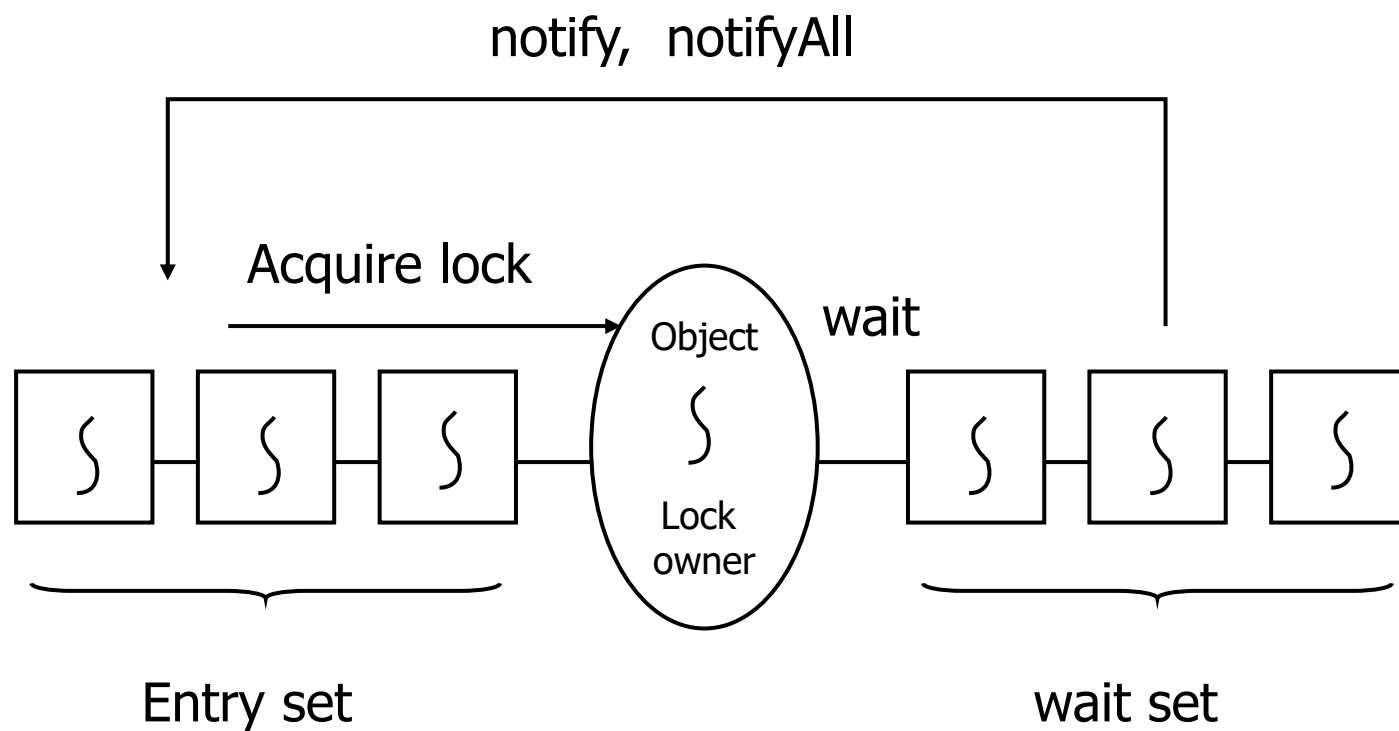
3. 允许嵌套，等待时释放所有管程互斥权。并发性好，但是一旦等待结束，需要重新获得路经管程的互斥权，实现困难，很难保证语义上的正确性；

4. 允许嵌套，调用时释放路经管程互斥权。并发性好，但保证语义上的正确性更加困难。

Java语言中的“管程”

- 类似管程的对象object
 - 每个object有一个互斥锁lock，一般未用
 - 说明为synchronized的method启用互斥锁
 - 每个object内部有一个等待队列
 - wait() method
 - 锁的持有者执行wait操作，状态改为blocked，释放所持有的锁，进入wait set；若entry set非空，选择其一进入
 - notify() method
 - 在wait set中取任意线程，移到entry set，状态改为runnable. (Signal and continue)
 - notifyAll() method
 - Wait set所有线程移到entry set，状态改为runnable

Entry set and wait set



例1：生产/消费问题--Java

```
public class Bounded Buffer {  
    public BoundedBuffer() {    //缓冲区变量定义  
        count=0; in=0; out=0;  
        buffer=new Object[BUFFER_SIZE];  
    }  
}
```

例1：生产/消费问题--java

```
public synchronized void enter(Object item) {  
    while (count==BUFFER_SIZE) {  
        try { wait(); //被唤醒后重新检测等待条件  
        } catch (InterruptedException e){ }  
    }  
    ++count;  
    buffer[in]=item;  
    in=(in+1)% BUFFER_SIZE;  
    notify();  
}
```

例1：生产/消费问题--java

```
public synchronized Object remove() {  
    while (count==0) {  
        try { wait();  
        } catch (InterruptedException e){ }  
    }  
    --count;  
    item=buffer[out];  
    out=(out+1)% BUFFER_SIZE;  
    notify();  
    return(item);  
}
```

例1：生产/消费问题--java

```
private static final int NAP_TIME=5;  
private static final int BUFFER_SIZE=5;  
private int count in,out;  
private Object[] buffer;  
}
```

例2读者 / 写者问题-Java solution

Multiple notification

`notifyAll()` -- 唤醒wait set集合中所有线程, 进入entry set。

```
public class Database {  
    public Database() {  
        Readcount=0;  
        dbReading=false;  
        dbWriting=false;  
    }  
    public synchronized int startRead() {  
        //  
    }
```

读者 / 写者问题-Java solution

```
public synchronized int endRead() {  
    //  
}  
public synchronized startWrite() {  
    //  
}  
public synchronized endWrite() {  
    //  
}  
private int readerCount;  
private boolean dbReading;  
private boolean dbWriting;  
}
```

读者 / 写者问题-Java solution

```
public synchronized void startRead() {  
    while (dbWriting==true) {  
        try {  
            wait();  
        } catch (InterruptedException e) {}  
        ++readCount;  
        if (readCount==1)  
            dbReading=true;  
    }  
}
```


读者 / 写者问题-Java solution

```
public synchronized void endRead() {  
    --readCount;  
    if(readCount==0)  
        dbReading=false;  
    notifyAll();  
}
```

读者 / 写者问题-Java solution

```
public synchronized void startWrite() {  
    while(dbReading==true||dbWriting==true) {  
        try{  
            wait();  
        }catch(InterruptedException e){ }  
    }  
    dbWriting=true;  
}  
public synchronized void endWrite(){  
    dbWriting=false;  
    notifyAll();  
}
```

Java 同步规则

■ Remarks

- 拥有一个对象锁的线程可进入同一对象的另一个同步方法
- 一个对象的同步方法可嵌套调用另一对象的同步方法，因而一个线程可同时拥有多个对象的互斥锁
- 同一对象中未说明synchronized的方法与说明synchronized的方法可以同时进入
- 当一个对象的wait set为空时，notify()和notifyAll()没有作用

Windows互斥同步机制

- Mutex (OS API)
 - CreateMutex, //创建互斥对象, 返回句柄
 - OpenMutex, //打开并返回存在对象句柄
 - ReleaseMutex; //释放对互斥对象的占用
- 保证只有一个线程或进程可以申请到该对象
- 可以跨进程使用
- 可以有名称
- 互斥量比临界区要耗费更多资源, 速度慢

Windows互斥同步机制（Cont.）

- semaphore (OS API)
 - CreateSemaphore //创建信号量
 - OpenSemaphore //返回已存在信号量句柄
 - ReleaseSemaphore //释放对信号量的占用
- 允许指定数目的多个线程/进程访问临界区
- 一种资源计数器，用于限制并发线程的数量
- 初始值可设为N，表示允许N个进程/线程并发访问资源
- 信号状态
 - 信号量的值大于0时，有信号状态
 - 信号量的值小于等于0，为无信号状态

Windows互斥同步机制（Cont.）

- Event（OS API）：用于通知一个或多个线程某事件出现或标识某操作已经完成
 - CreateEvent
 - OpenEvent
 - SetEvent
 - ResetEvent
 - PulseEvent

Windows互斥同步机制（Cont.）

- CRITICAL_SECTION类型
 - InitializeCriticalSection: 初始化
 - EnterCriticalSection: 等待进入
 - TryEnterCriticalSection: 非等待, 失败返回0
 - LeaveCriticalSection: 离开
 - DeleteCriticalSection: 删除
- 在进程内使用, 保证仅一个线程可以申请到该对象
- 临界区内是临界资源的访问

提纲

4.1 并发进程(concurrent processes)

4.2 进程互斥(mutual exclusion)

4.3 进程同步(synchronization)

4.4 进程高级通讯 (communication)

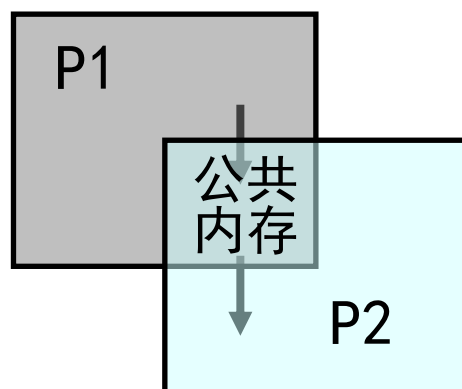
4.4 进程高级通讯

4.4.1 进程通讯概念

- 进程通讯：进程之间的相互作用
 - 低级通讯（简单信号）
 - 进程互斥
 - 进程同步
 - 高级通讯（大宗信息）
- 高级通讯
 - memory sharing vs. message passing
 - direct vs. indirect
 - symmetric vs. non-symmetric
 - buffering vs. non-buffering

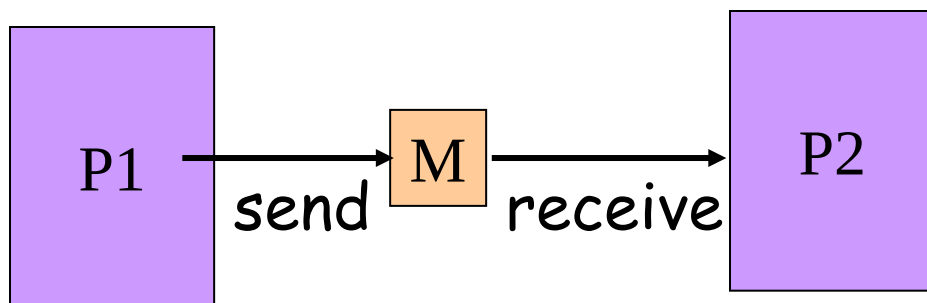
4.4.2 进程通讯模式

- 共享内存模式 (shared memory)
 - OS提供
 - 公共内存
 - 互斥同步机制



4.4.2 进程通讯模式(Cont.)

- 消息传递模式 (message passing)
 - OS提供两个基本的系统调用命令
 - 发送命令
 - 接收命令

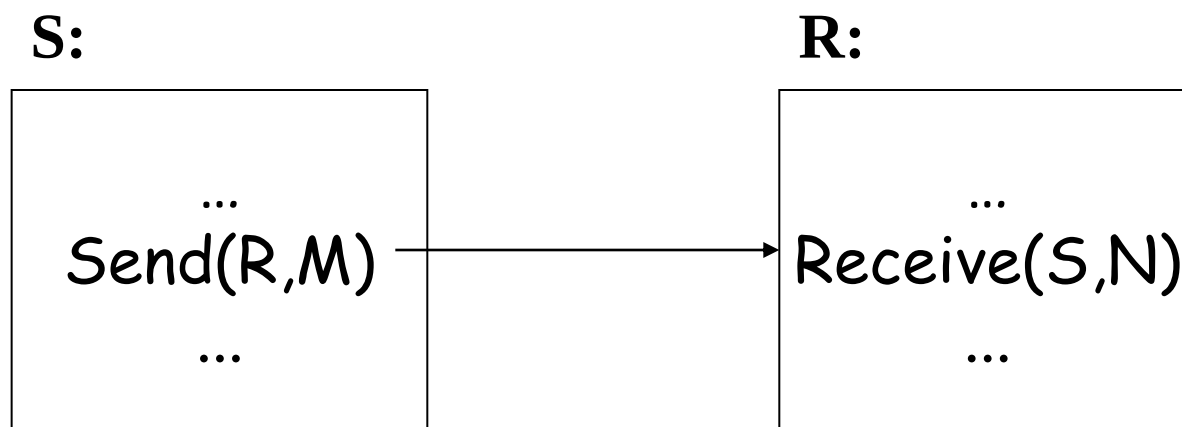


直接：进程-进程

间接：进程-信箱-进程

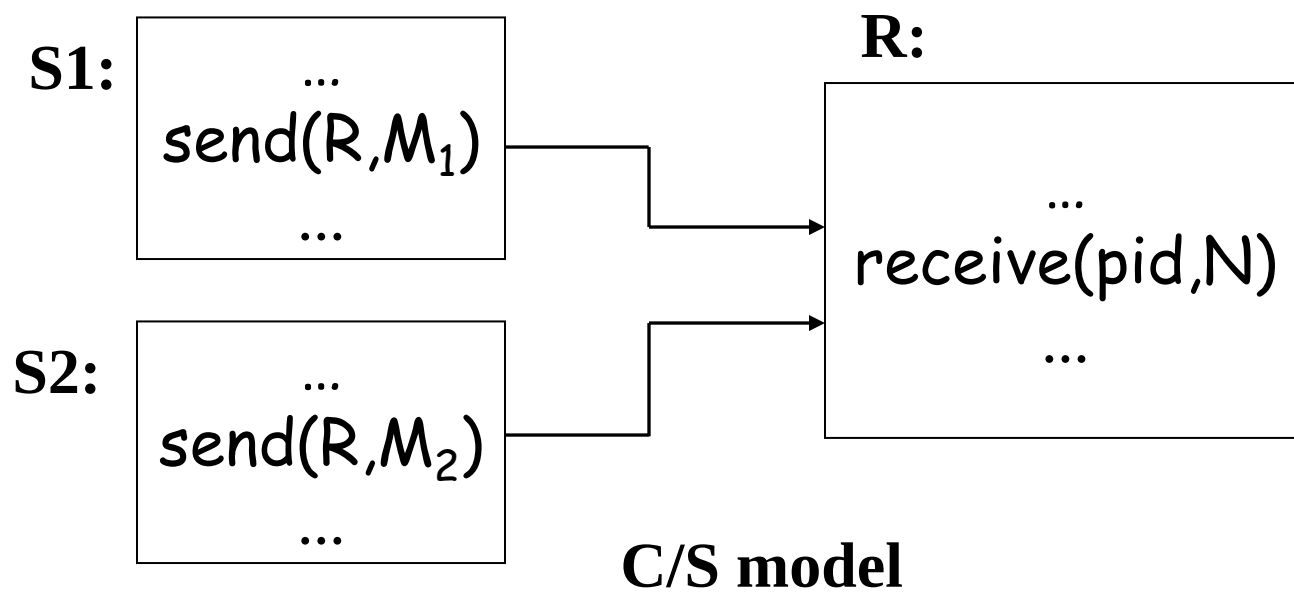
4.4.3 直接方式

- 对称形式：一对一，发送者在发送时指定接收者的名字，接收者在接收时要指定发送者的名字
 - `send(R, message)`
 - `receive(S, message)`



4.4.3 直接方式(Cont.)

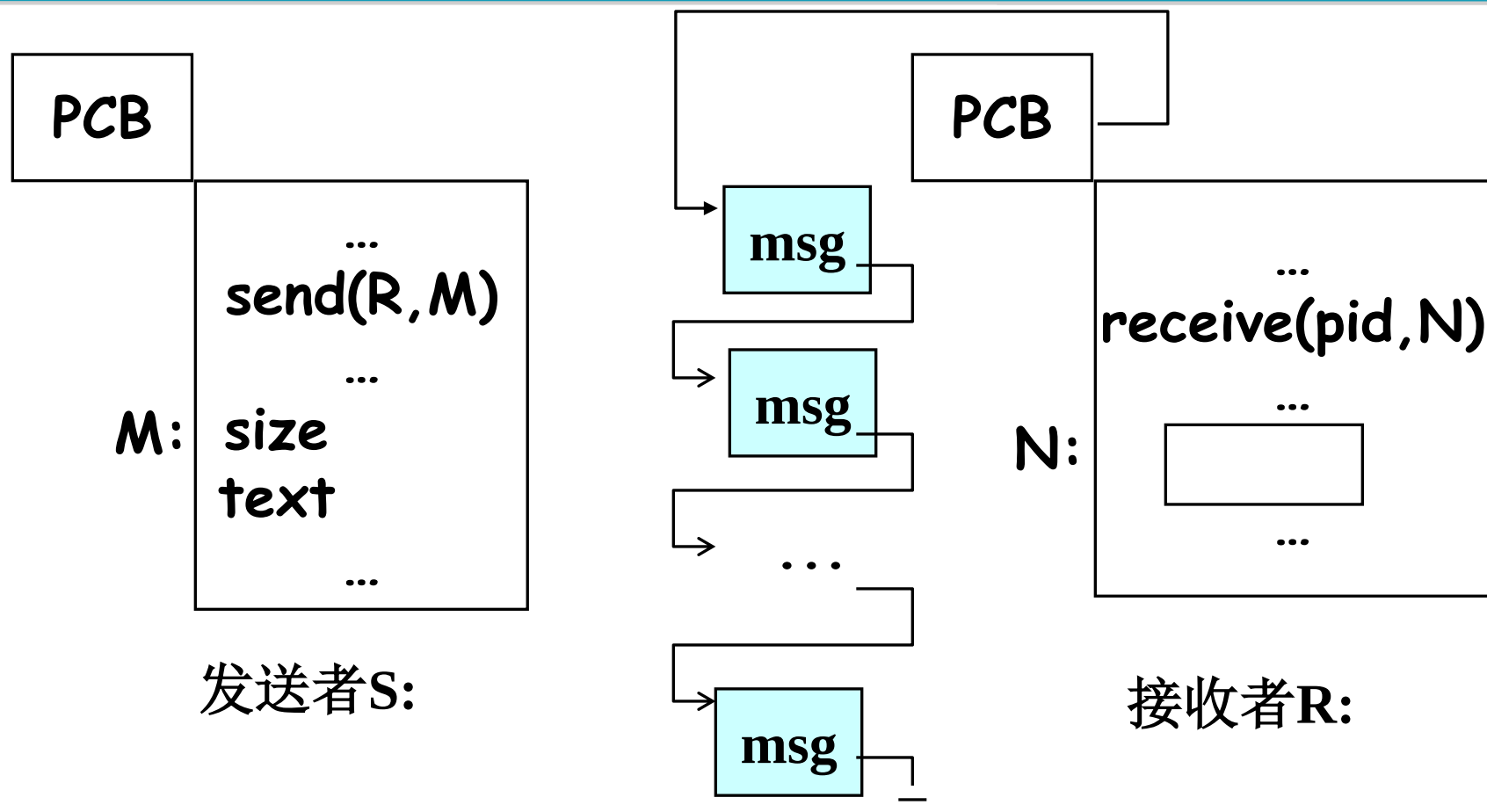
- 非对称形式：多对一，发送者在发送时指定唯一的接收者，接收者在接收时不指定具体的发送者
 - `send(R, message)`
 - `receive(pid, message)`



4.4.3 直接方式(Cont.)

- 实现方式
 - 有缓冲途径
 - 无缓冲途径

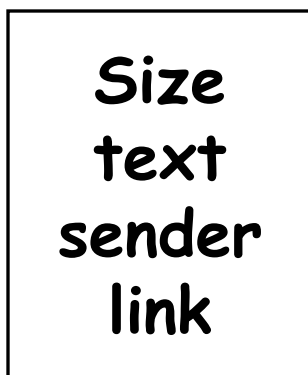
4.4.3.1 有缓冲途径 (消息传递模式,直接方式,非对称形式)



消息在发送者和接收者的传输经过一次缓冲，提高了系统的并发性。

Message passing, direct, non-symmetric, buffering

载有消息的缓冲:



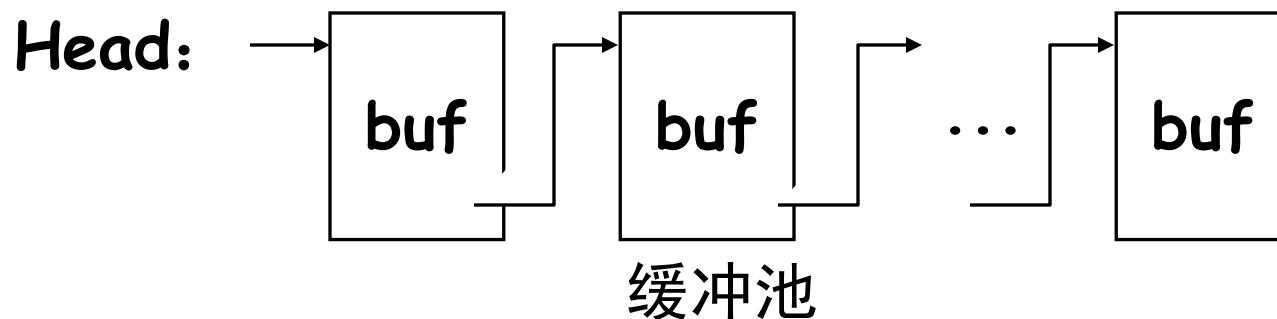
进程消息队列管理:

```
Var Sm: semaphore; (0)  
    收取消息前: P (Sm) ;  
    消息入队后: V (Sm);
```

消息队列互斥:

```
Var m_mutex: semaphore; (1)  
    P (m_mutex) ;  
    入列(出列) 动作;  
    V (m_mutex) ;
```


Buffer pool management



Var Sb, b_mutex:semaphore; (k,1)

申请: P(Sb);

P(b_mutex);

头缓冲出链;

V(b_mutex);

释放: P(b_mutex);

缓冲入链头;

V(b_mutex);

V(Sb);

发送-接收原语

Send (R, M)

根据R找接收者，否则带错
返回；

P (Sb) ;

P (b_mutex) ;

取一空buf;

V (b_mutex) ;

size, text, sender => buf;

P (m_mutex) ;

将载有消息的缓冲区挂到接
收进程消息链的尾部；

V (m_mutex) ;

V (Sm) ;

Receive (pid, N)

P (Sm) ;

P (m_mutex) ;

将消息链中的第一个载有消息的
缓冲区由消息链取出；

V (m_mutex) ;

size, text=> N;

sender => pid;

P (b_mutex) ;

将空闲的消息缓冲区挂到消息链
的尾部；

V (b_mutex) ;

V (Sb) ;

Remarks

- `Send/receive` 为高级通讯原语，可用低级原语实现
- `Send/receive`不是真正意义的原语，可以被中断
- 对于发送进程来说，信息需要由发送进程空间复制到消息缓冲区中。对于接收进程来说，信息需要由消息缓冲区复制到接收进程空间
- 发送进程空间和接收进程空间均属于用户区，而消息缓冲区属于操作系统区

4.4.3.2 无缓冲途径 (消息传递模式,直接方式,非对称形式)

- 发送-接收都发生，信息由发送进程空间复制到接收进程空间，传送由操作系统完成
 - 当发送进程执行到send系统调用命令时，如果接收进程尚未执行receive系统调用命令，则发送进程将等待
 - 当接收进程执行到receive系统调用时，如果发送进程尚未执行send系统调用命令，则接收进程将等待
 - 当发送进程执行到send系统调用命令且接收进程执行到receive系统调用命令时，信息传输才真正开始，此时消息以字为单位由发送进程空间传送到接收进程空间中，由操作系统完成复制，传输时可以使用寄存器

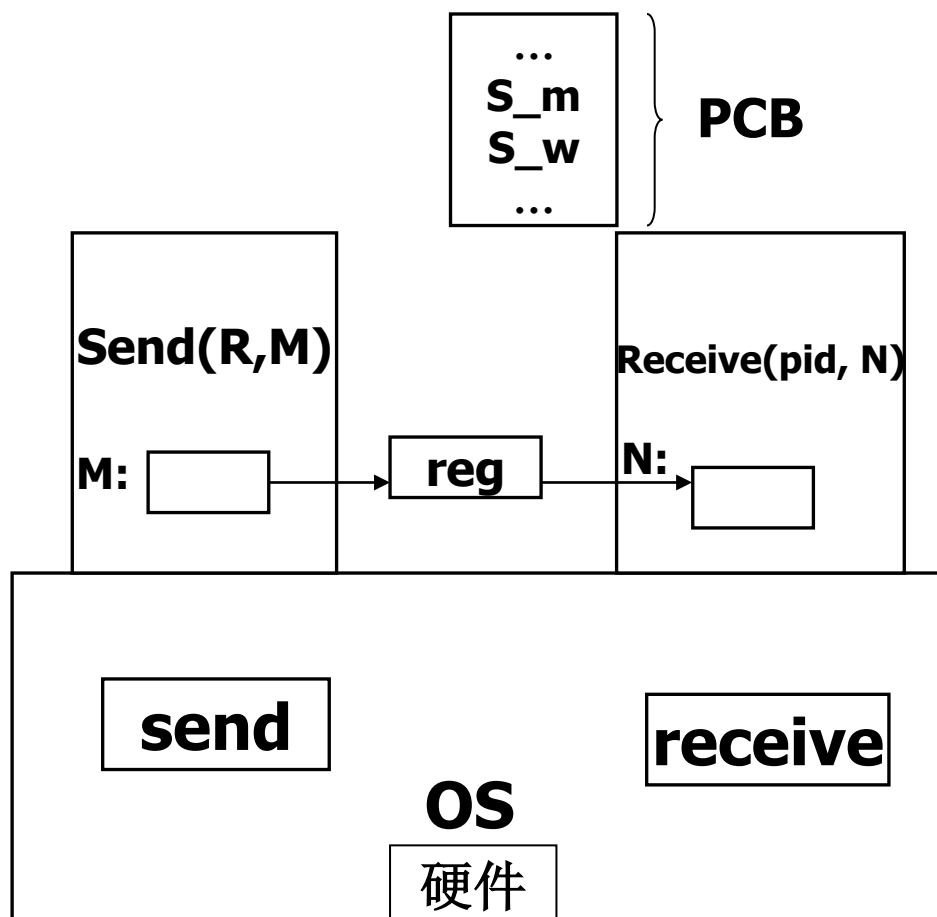
4.4.3.2 无缓冲途径 (消息传递模式,直接方式,非对称形式)

- PCB中两个信号量, S_m , S_w , 初值0
 - S_m : 接收进程等待
 - S_w : 发送进程等待

4.4.3.2 无缓冲途径

- 发送过程: `send(R, M)`
 - 根据R找到消息接收者
 - 发送消息增1, 如接收进程等待将其唤醒, 即执行 $V(S_m)$
 - 等待消息传送完毕, 即执行 $P(S_w)$
- 接收过程: `receive(pid, N)`
 - 等待消息到达, 即执行 $P(S_m)$
 - 消息由发送进程空间复制到接收进程空间
 - 唤醒发送消息进程, 即执行 $V(S_w)$

4.4.3.2 无缓冲途径



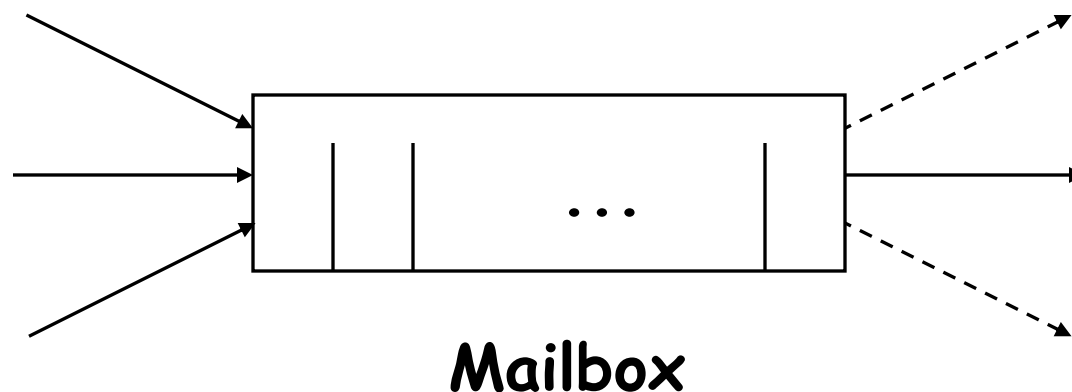
4.4.3.2 无缓冲途径

- 优点
 - 节省空间(不需要buffer)
- 缺点
 - 并发性差：发送进程需要等待接收进程执行receive把信息复制到接收进程空间后才能继续

4.4.4 间接方式

- 间接方式：
 - 指相互通信的进程之间在通信时不是直呼对方名字，而是指明一个中间媒体，即信箱，进程之间通过信箱来实现通信
- 发送和接收原语
 - `Send_MB (MB, M)` : 将消息M发送到信箱MB中
 - `Receive_MB (MB, N)` : 由信箱MB中接收消息至N

4.4.4 间接方式



`Send_mb(mb,m)`

`Receive_mb(mb,n)`

multi-sender -- multi-receiver

multi-sender -- one receiver

4.4.4.1 信箱属于操作系统空间

```
Type    mailbox=record
        in, out:0..k; //读写指针，初值为0
        s1, s2:semaphore; (k, 0) //协调发送和接收进程
        mutex:semaphore; (1) //用于对信箱操作互斥
        //信箱体，可以保存k封信件
        letter:array[0..k-1]of message
end;
Var mb:mailbox; //定义信箱类型的变量
```

4.4.4.1 信箱属于操作系统空间

- 有关信箱的4个系统调用命令
 - creat_MB
 - delete_MB
 - send_MB
 - receive_MB

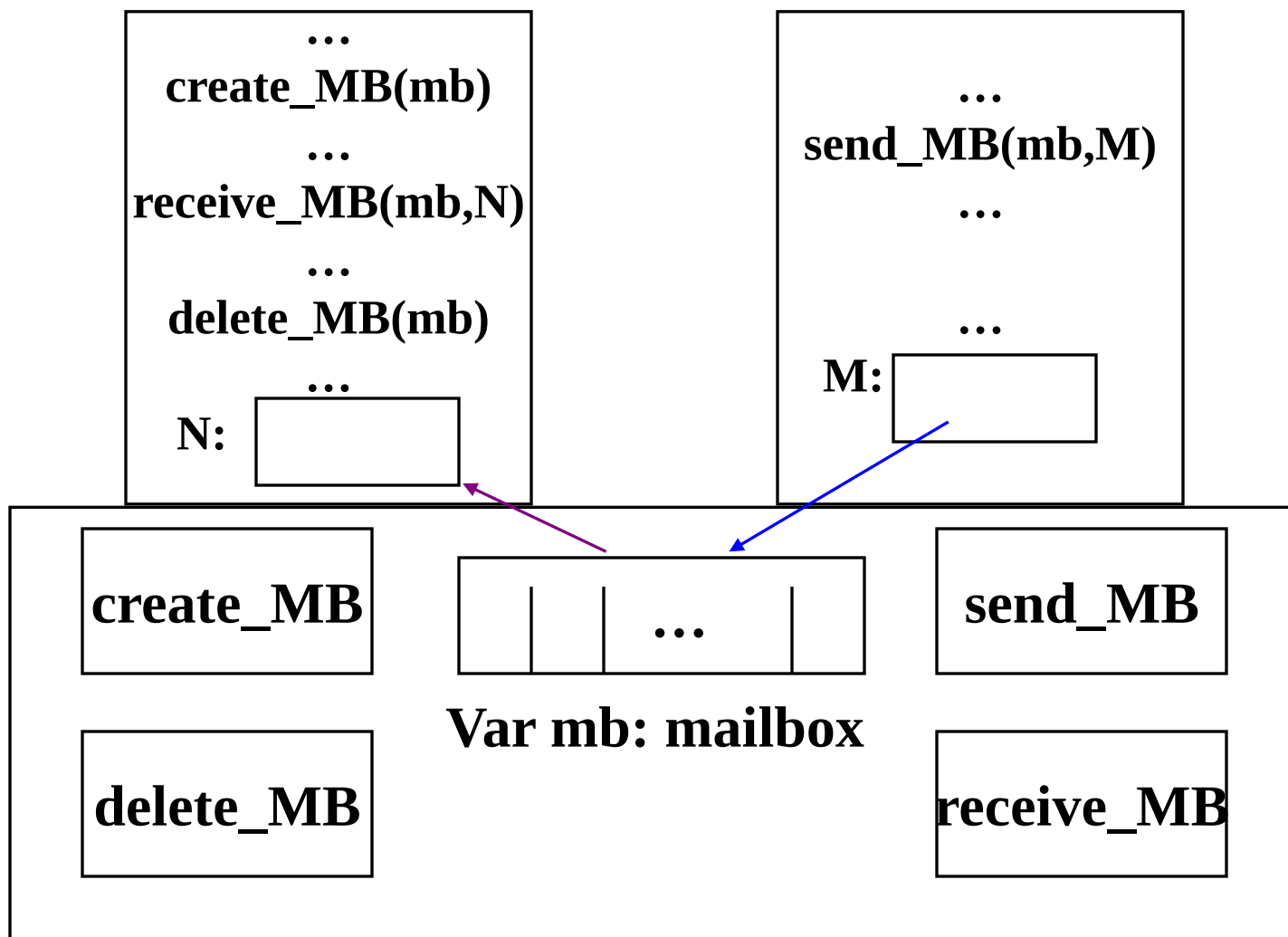
信箱通讯

```
Procedure send_mb(var mb:mailbox; m:message);  
begin  
    with mb do  
        begin  
            P(s1); //申请空位  
            P(mutex);  
            letter[in]:=m;  
            in:=(in+1)mod k;  
            V(mutex);  
            V(s2) //消息增加一个  
        end;  
    end;  
end;
```

信箱通讯

```
Procedure receive_mb(var mb:mailbox; var n:message);  
begin  
    with mb do  
        begin  
            P(s2); //申请消息  
            P(mutex);  
            n:=letter[out];  
            out:=(out+1)mod k;  
            V(mutex);  
            V(s1) //空位增加一个  
        end;  
    end;  
end;
```

属于操作系统空间的信箱



UNIX进程高级通讯机制

- 管道pipe：
 - 管道是一个特殊的文件，没有名字，只有两个文件描述符
 - 读文件描述符和写文件描述符, 分别用于管道文件的读写
 - 当一个进程创建管道文件后，其后建立的子进程自动地继承该管道文件
 - 使用管道进行通信的条件是通信进程之间具有相同的祖先进程，并且管道已经由祖先进程建立

UNIX进程高级通讯机制（Cont.）

Pipe: an unnamed file with two fds, one for write, and the other for read. The size of pipe file is limited to 4 blocks(one block is 512 bytes).

```
int fd[2];
```

```
pipe(fd);
```

功能：创建一个pipe文件

返回：fd[0]：读描述符；

fd[1]：写描述符。

pipe可以被子进程继承

子进程1：

```
write(fd[1], buf1, count1);
```

子进程2：

```
read(fd[0], buf2, count2)
```

UNIX进程高级通讯机制（Cont.）

- pipe作为文件，读写需要两次IO，但一般不会真正执行IO操作
 - 缓冲与延迟写(delayed write)，写到内存缓冲区，只要内存资源不紧张，缓冲区不另作它用，不会写到外存，读时可在内存缓冲区中得到
 - Pipe文件大小的限制（Eg. 2k），循环使用，避免缓冲资源紧张

UNIX进程高级通讯机制（Cont.）

管道读规则

- **管道写端不存在：** 所有进程都关闭了写端。
 - 则认为已经读到了数据的末尾，读函数返回的读出字节数为0；
 - 管道写端关闭后，写入的数据将一直存在，直到读出为止。
- **管道写端存在：**
 - 如果请求的字节数目大于PIPE_BUF，
 - 则返回管道中现有的数据字节数；
 - 如果请求的字节数目不大于PIPE_BUF，
 - 则返回管道中现有数据字节数
(此时，管道中数据量小于请求的数据量)；
 - 或者返回请求的字节数
(此时，管道中数据量不小于请求的数据量)。

UNIX进程高级通讯机制（Cont.）

管道写规则

- 向管道中写入数据时，linux将不保证写入的原子性；
- 管道缓冲区一有空闲区域，写进程就会试图向管道写入数据；
- 如果管道缓冲区满，且读进程不读走管道缓冲区中的数据，那么写操作将一直阻塞；
- 只有在管道的读端存在时，向管道中写入数据才有意义；否则，向管道中写入数据的进程将收到内核传来的SIFPIPE信号，应用程序可以处理该信号，也可以忽略（默认动作则是应用程序终止）。
- 在向管道写入数据时，至少应该存在某一个进程，其中管道读端没有被关闭，否则就会出现上述错误。

UNIX进程高级通讯机制（Cont.）

- 优点
 - 实现简单
 - 界面统一
 - 节省内存
- 缺点
 - 速度较慢

UNIX信号通讯机制-Signal

- 信号是进程间一种重要的通信机制
- 信号是向进程发送的一个通知，通知某个事件已发生
- 收到信号的进程可以立即执行指定的操作
- 信号的发送可以是进程，也可以是系统（含硬件）

UNIX信号通讯机制（Cont.）

- 终端上使用信号机制的例子
- 例1：键盘上按下Ctrl+C，杀死一个进程
说明：1) Ctrl+C，产生信号**SIGINT**
2) 进程收到**SIGINT**信号，执行默认操作（结束进程）
- 例2：键盘上按下Ctrl+Z，挂起（暂停）一个进程
说明：1) Ctrl+Z，产生信号**SIGSTP**
2) 进程收到**SIGSTP**，执行默认操作（挂起进程）
- 例3：终端命令kill-9，杀死一个进程
说明：1) kill-9，产生信号**SIGKILL**
2) 进程收到**SIGKILL**，执行默认操作（强制结束进程）

UNIX信号通讯机制（Cont.）

■ 信号的产生

- 来源1：键盘输入特殊组合键产生信号，例Ctrl+C
- 来源2：执行终端命令产生信号，例Kill系列命令
- 来源3：程序中调用函数产生信号，例kill(), abort()
- 来源4：硬件异常或内核产生相应信号。例：内存访问出错

UNIX信号通讯机制（Cont.）

- 信号的定义：64种信号，信号用整数1-64表示

信号编号	信号名称	说明
2	SIGINT	进程结束（按键 CTRL+C 能产生）。
6	SIGABRT	进程结束（调用 abort 函数产生）。
9	SIGKILL	进程强制结束（用户不能捕获该信号）。
11	SIGUSR1	用户自定义信号 1。
14	SIGALRM	定时器时间到的信号。
19	SIGTSTP	进程暂停执行（用户不能捕获该信号）。

UNIX信号通讯机制（Cont.）

中断与信号相同点：

- 都采用异步通信方式
- 当检测到信号或中断时，暂停当前程序的执行，转去执行相应处理程序
- 处理完后回到断点
- 信号和中断都可进行屏蔽

中断与信号不同点：

- 中断有优先级，信号没有
- 信号处理在目态，中断处理在管态
- 中断响应是及时的，信号响应通常都有较大的延迟。

UNIX信号通讯机制（Cont.）

- **Signal 发生时的三种选择：**
- **忽略Signal：**大部分Signal都可以Ignore，
 - 除了SIGKILL和SIGSTOP；
- **捕捉Signal：**可以让内核来调用我们所指定的函数。
 - (SIGKILL和SIGSTOP无法捕捉.)
- **执行缺省行为：**如果不做任何处理，则执行缺省动作。
 - 大部分Signal的缺省行为都是中止进程。

UNIX信号通讯机制（Cont.）

```
struct proc{
    ...
    char p_sig;//接收到的信号
    ...
}proc[NPROC]

struct user{
    ...
    //信号处理程序入口
    int u_signal[NSIG];
    ...
}
```

（用户处理的中断）

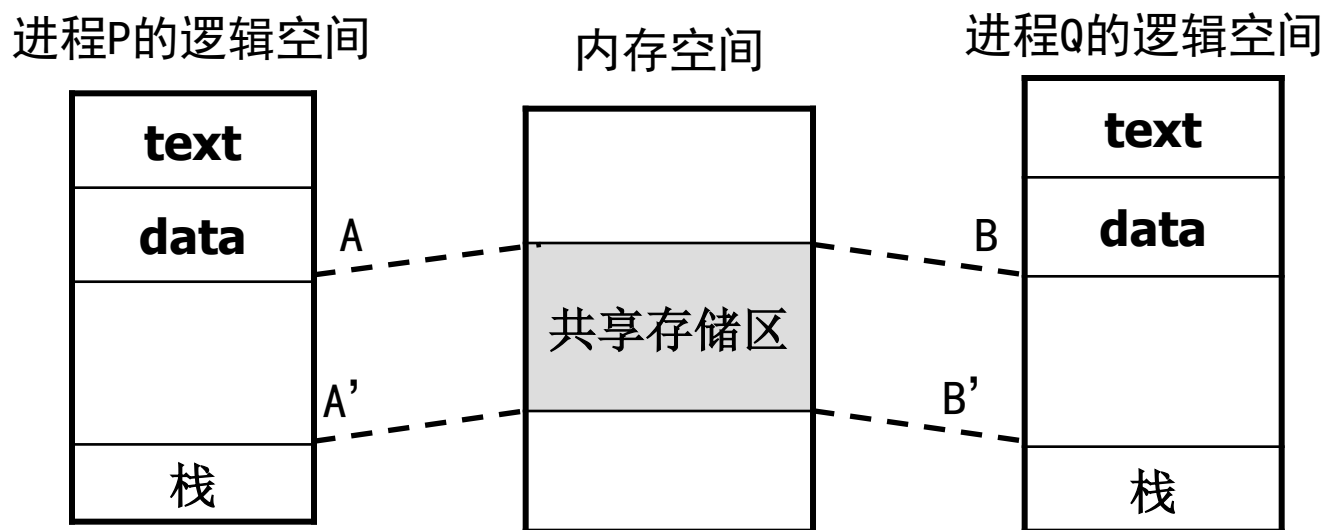
- 系统调用signal(sig, func)
预置信号处理程序
 - u_signal[sig]=func;
 - Func=0时接收信号进程终止自己;
 - Func=奇数时忽略;
 - Func=偶数时为处理程序入口.
 - 进程初创时继承其父进程信号处理程序
- 信号发送与接收
 - 任何时候都可接收, 记录在p_sig中
 - 进程收到信号后执行signal函数

Linux共享内存通讯

■ 共享内存

- 共享内存是内存中的一个公共区域，可以通过系统调用命令 **shmget** 建立。此后，进程可以通过系统调用命令 **shmat** 将共享内存附接至自己的逻辑地址空间
- 进程相应逻辑空间的读写访问将反映在共享存储区中，如此多个进程便可以相互交换信息
- 多个进程对共享内存的访问需要考虑互斥和并发控制问题
- **Shmdt** 用于将共享内存与进程逻辑空间分离

Linux共享内存通讯 (Cont.)



限制：只能用于同一站点通讯

注意：互斥问题需要用户自己控制

Linux共享内存通讯 (Cont.)

■ 描述共享存储区的数据结构

```
struct shmid_ds {  
    struct ipc_perm shm_perm;    //operation perms  
    int  shm_segsz;    // size of segment (bytes)  
    __kernel_time_t  shm_atime;    // last attach time  
    __kernel_time_t  shm_dtime;    // last detach time  
    __kernel_time_t  shm_ctime;    // last change time  
    __kernel_ipc_pid_t  shm_cpid;    // pid of creator  
    __kernel_ipc_pid_t  shm_lpid;    // pid of last operator  
    unsigned short  shm_nattch;    // no. of current attaches  
    unsigned short  shm_unused;    // compatibility  
    void *shm_unused2;    // ditto - used by DIPC  
    void *shm_unused3;    // unused  
}
```

Linux共享内存通讯（Cont.）

■ 相关系统调用

- `shmget(key, size, permflg)` //共享存储区的获取或创建
- `shmat(shmid, shmaddr, shmflg)` //共享存储区的附接
- `shmdt(virtaddr)` //共享存储区的剥离
- `shmctl(shmid, cmd, buf)` //共享存储区的控制

Linux共享内存通讯 (Cont.)

■ 共享存储区的获取或创建

■ 调用原型:

- `int shmids=int shmget (key_t key, int size, int shmflag);`

■ 参数:

- Key: 共享存储区的名字
- size: 共享存储区的大小(以字节为单位, 页面大小的整数倍)
- shmflg: (1) 权限(读写), (2) 标志 (a) IPC_CREATE不存在创建, 存在返回标识, (b) IPC_CREATE|IPC_EXCL不存在创建, 存在报错。

■ 返回值:

- shmids: 共享内存的标识(ID/描述符), 错误返回-1。

■ 调用过程

- 检查共享存储区表, 查找与key相同的表项, 若找到, 则说明共享存储区已经建立, 返回描述符shmids
- 若未找到, 且shmflg为IPC_CREATE, 且参数size在系统限制范围内, 则分配页表及相应页面, 并读入页面内容
- 在系统共享存储区表中, 为新建的空闲区域分配一个表项, 并填写相关信息, 最后返回共享存储区描述符shmids

Linux共享内存通讯 (Cont.)

- 共享内存的附接：将共享存储区附接到自己的虚拟地址空间上，此后，共享存储区便成为进程虚拟地址空间的一部分，对该空间的访问被映射到对共享存储区的访问
 - 调用原型
 - `*virtaddr=shmat(int shmid, char *shmaddr, int shmflag)`
 - 参数：
 - `shmid`：共享内存的ID，`shmget`函数的返回值；
 - `*shmaddr`：共享内存对应的进程虚拟空间的起始地址指针，通常为`空`，表示让系统来选择共享内存的地址。
 - `int shmflag`：操作模式，`SHM_RDONLY`只读模式，其它(0)读写模式。
 - 返回值：
 - 成功时，共享存储区对应进程虚拟空间的起始地址指针，失败时返回-1。
- 第一次创建完共享内存时，它还不能被任何进程访问，`shmat`函数的作用就是用来启动对该共享内存的访问，并把共享内存连接到当前进程的地址空间

Linux共享内存通讯 (Cont.)

- 脱离附接：当进程不再需要共享存储区时，将其与进程虚拟地址空间分离
 - 调用原型
 - `int shmdt(char *virtaddr)`
 - 参数：
 - `*virtaddr`：共享内存的虚拟地址指针(要分离的虚拟地址)
 - 返回值：
 - 成功时返回0。失败时返回-1

Linux共享内存通讯 (Cont.)

■ 共享内存控制

■ 调用原型

- `int shmctl(int shmid, int cmd, struct shmid_ds *buf)`

■ 参数

- shmid: 共享内存的ID
- cmd: 是控制命令(
 - IPC_STAT: 把shmid_ds结构中的数据设置为共享内存的当前关联值, 即用共享内存的当前关联值覆盖shmid_ds的值
 - IPC_SET: 如果进程有足够的权限, 就把共享内存的当前关联值设置为shmid_ds结构中给出的值
 - IPC_RMID: 删除共享内存段
- *buf: 指向shmid_ds(共享内存的数据结构)结构体指针

■ 返回值:

- 成功: 0, 失败: -1

Linux共享内存通讯(例1)

发送进程 sndshm.c

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<unistd.h>
#include<sys/types.h>
#include<linux/shm.h>

main()
{
    int shmid;
    char *viraddr;
    char buffer[BUFSIZ];
    shmid=shmget(1234, BUFSIZ, 0666|IPC_CREAT);
    viraddr=(char*)shmat(shmid, 0, 0);
```

Linux共享内存通讯(例1)

```
while(1)
{
    puts("Enter some text:");
    fgets(buffer, BUFSIZ, stdin);
    strcat(viraddr, buffer);
    if(strncmp(buffer, "end", 3)==0)
        break;
}
shmdt(viraddr);
exit(0);
}
```

Linux共享内存通讯(例1)

接收进程 rcvshm.c

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<unistd.h>
#include<sys/types.h>
#include<linux/shm.h>
main()
{
    int shmid;
    char *viraddr;
```

Linux共享内存通讯(例1)

```
    shmidx=shmget(1234, BUFSIZ, 0666|IPC_CREAT)
    viraddr=shmat(shmidx, 0, SHM_RDONLY);
    printf("Your message is : \n%s", viraddr);
    shmdt(viraddr);
    shmctl(shmidx, IPC_RMID, 0);
    exit(0);
}
```


Linux共享内存(例2)

```
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define KEY 1234
#define SIZE 1024
int main()
{
    int shmid;
    char *shmaddr;
    struct shmid_ds buf;
    shmid = shmget(KEY, SIZE, IPC_CREAT | 0600); /*建立共享内存*/
    if (fork() == 0)
    {
        shmaddr = (char *) shmat(shmid, 0, 0);
        strcpy(shmaddr, "Hi! I am Chiled process!\n");
        printf("Child: write to shared memery: /\"Hi! I am Chiled process!\"/\"/>\n");
        shmdt(shmaddr);
        return;
    }
}
```

Linux共享内存(例2)

```
else
{
    sleep(3);    /*等待子进程执行完毕*/
    shmctl(shmid, IPC_STAT, &buf);    /*取得共享内存的状态*/
    printf("shm_segsz = %d bytes/n", buf.shm_segsz);
    printf("shm_cpid = %d/n", buf.shm_cpid);
    printf("shm_lpid = %d/n", buf.shm_lpid);
    shmaddr = (char*) shmat(shmid, 0, SHM_RDONLY);
    printf("Father:  %s/n", shmaddr);    /*显示共享内存内容*/
    shmdt(shmaddr);
    shmctl(shmid, IPC_RMID, NULL);    /*删除共享内存*/
}
}
```

Linux共享存储通讯

- 几点说明
 - `fork()` 后子进程继承其父进程已连接的共享内存
 - 执行`exec`自动剥离共享存储
 - 执行`exit`自动剥离共享存储

总 结

- 并发进程 (concurrent processes)
- 进程互斥 (mutual exclusion)
- 进程同步 (synchronization)
- 进程高级通讯 (communication)

作业 (2)

1. Consider the following program:

```
var blocked: array[0..1] of boolean;  
    turn: 0..1;  
procedure P(id: integer);  
begin  
    repeat  
        blocked[id] := true;  
        while turn <> id do  
            begin  
                while blocked[1-id] do  
                    {nothing}  
                turn := id  
            end;  
            <Critical section>  
            blocked[id] := false;  
            <Remainder>  
        until false  
    end;
```

```
begin  
    blocked[0] := false;  
    blocked[1] := false;  
    turn := 0;  
    parbegin  
        P(0); P(1)  
    parend;  
end.
```

This is a software solution to the mutual exclusion problem proposed by Hyman. Find a counter example to demonstrate that this solution is incorrect. It is interesting to note that even the *Communication of the ACM* was fooled on this one.

- 2 设有一个可以装入A、B两种物品的仓库，其容量无限大，但是要求仓库A、B两种物品的数量满足下述不等式：

$$-m \leq A\text{物品数量} - B\text{物品数量} \leq n。$$

其中，m和n为正整数，试用信号量和PV操作描述A、B两种物品的入库过程。

- 3 假定系统有三个并发进程read, move和print。进程read负责从输入设备上读信息, 每读入一个记录后把它存放到缓冲区B1中; 进程move从缓冲区B1中取出一记录, 加工后存入缓冲区B2; 进程print将B2中的记录取出打印输出。缓冲区B1和B2都只能存放一个记录。要求打印输出的记录与读入的记录的个数、次序完全一样。用信号灯和P/V操作实现三个进程的同步。
- 4 某图书阅览室有50个座位, 进入阅览室的读者需要在登记簿上登记, 离开阅览室时注销登记。试用管程实现对阅览室的管理。